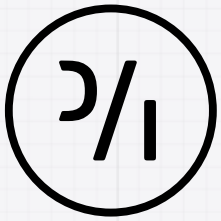


Knowledge Graph
Developer Handbook

Version 4.2

Mar 2026

Prevalent AI

1. Introduction 2
1.1. Key Features of Entity Inventory *2*
1.1.1. Entity Extraction 2
1.1.2. Entity Resolution 3
1.1.3. Relationship Extraction 3
1.1.4. Relationship Resolution 3
1.1.5. Post Analytics Writeback 3
1.1.6. Graph Olap Layer 3
1.1.7. Entity Explorer 3
1.2. Data Quality Features *3*
1.2.1. Data Quality Scoring 4
1.2.2. Core Data Quality Dimensions 4
1.2.3. Layer-Aware Quality Measurement 4
1.2.4. Entity-Level Quality Aggregation 4
1.2.5. Data Quality Evidence Persistence 4
1.2.6. Entity Enrichment with Quality Values 4
1.2.7. Data Quality Insights and Configurability 4
2. Configuring Entity Inventory 4
2.1. Global Entity Configuration *4*
2.1.1. Config Details 5
2.1.2. Field Description 5
2.1.3. Field Level Spec Details 5
2.2. Entity Extraction *6*
2.2.1. Key Functionalities 7
2.2.2. Entity ID 7
2.2.3. Entity Field Extraction Strategies 7
2.2.4. Smart Configuration Reorganizer 8
2.2.5. Controlling Persist/Update Behaviour 8
2.2.6. Lookup Enrichment 8
2.2.7. Lookup Enrichment / OS Extraction | EoL-EoS 9
2.2.8. Entity Extraction Using View Based Strategy 9
2.2.9. Job Execution Flow 9
2.2.10. Config Details 9
2.3. Mini SRDM Creation *9*
2.3.1. Mini SRDM Generation Logic and Configuration 9
2.4. Entity Resolution *9*
2.4.1. Disambiguation Group Identification 9
2.4.2. Attributes Resolution 9
2.4.3. Source Based Confidence Matrix 9
2.4.4. Controlling Persist/Update Behaviour 9
2.4.5. Entity Enrichment 9
2.4.6. Lookup Enrichment Support in Entity Resolution 9
2.4.7. Fragment and Resolver Table 9
2.4.8. Job Execution Flow 9
2.4.9. Config Details 9
2.4.10. Resolution Field Extraction Strategies 9
2.5. Entity Fragment Write *9*
2.5.1. Job Execution Flow 9
2.6. Relationship Extraction *9*

2.6.1. Relation Input Support 9
2.6.2. Relation Block Building 9
2.6.3. Lookup Enrichment / Software Normalisation 9
2.6.4. Job Execution Flow 9
2.6.5. Config Details 9
2.7. Relationship Resolution *9*
2.7.1. Disambiguation Group Identification 9
2.7.2. Attributes Resolution 9
2.7.3. Output Generation 9
2.7.4. Fragment and Resolver Table 9
2.7.5. Job Execution Flow 9
2.7.6. Config Details 9
2.8. Post Analytics Write Back *9*
2.8.1. Post Analytics Write Back Job 9
2.8.2. Post Analytics Write Back Configurations 9
2.9. Entity Relationship Enrichment *9*
2.9.1. Entity Relationship Enrichment Job 9
Job Execution flow of Enrichment Job 9
2.9.2. Entity Relationship Enrichment Configurations 9
2.10. Graph Olap Layer *9*
2.10.1. Graph Olap Layer Job 9
2.10.2. Graph Olap Layer Configurations 9
Graph Olap Config Element Details 9
2.10.3. Config Merger for Graph Olap Configs 9
2.11. Change Detection System(CDS) *9*
2.11.1. Execution Flow 9
2.11.2. Configuring Airflow Variables for CDS 9
2.12. Entity Explorer *9*
2.12.1. Data Dictionary Update Module 9
Data Type Mapping 9
Execution Flow 9
2.12.2. Data Dictionary Data Dictionary 9
2.12.3. Config Merger for data dictionary 9
2.12.4. UI Automation 9
2.12.5. Relevance of each Attribute from Data Dictionary on UI 9
2.13. Configuration Handler *9*
2.13.1. Entity extraction config merge 9
2.13.2. Entity Resolution Config Merge 9
2.13.3. Data Dictionary Config Merge 9
2.13.4. Relationship Config Update 9
3. Configuring Data Quality 9
3.1. Completeness Score Calculation *9*
3.1.1. Calculation Model 9
3.1.2. Formula and Aggregation Logic 9
3.1.3. Field-Level vs Record-Level Completeness 9
3.1.4. Threshold Configuration 9
3.1.5. Persistence and Consumption 9
3.2. Compaction Job *9*
3.2.1. Purpose of Compaction 9

3.2.2. Job Execution Flow of Compaction Job 9
3.2.3. Storage and Query Performance Impact 9
3.2.4. Integration with DQ Metrics 9
3.3. Dashboard Population (In-Depth Dashboard) *9*
3.3.1. End-to-End Data Retrieval Flow 9
3.3.2. Tables and Aggregation Logic 9
3.3.3. APIs and Service Interactions 9
3.4. Overview Dashboard Population *9*
3.4.1. How It Differs from In-Depth Dashboard 9
3.4.2. End to End Data Retrieval Flow 9
3.5. Config Details *9*

4. Orchestration Pipeline 9
4.1. Design *9*
4.1.1. Mediator Pattern: Orchestrating Solution Pipelines 9
4.1.2. Backfill DAG: Filling the missing dag runs 9
4.1.3. Generating dag tasks 9
4.2. Entity Extraction *9*
4.2.1. Calculating epoch parameter 9
4.2.2. Submitting spark job 9
4.3. Entity Resolution *9*
4.4. Entity Fragment Write *9*
4.5. Relationship Extraction *9*
4.6. Relationship Resolution *9*
4.7. Dynamic DAG Generator (Entity Relationship Enrichment & Publisher) *9*
4.8. Data Dictionary Mapping *9*
4.9. EI Auto Orchestration Config Generation *9*

5. Repository Management 9
5.1. EI Configs *9*
5.2. Config Merging *9*
5.2.1. Handling Array of objects 9
5.2.2. Shift Right Feature and Column Expression Merge Strategy of Config Merger 9
5.2.3. Config Merging for Each Folder 9
5.3 Config Context Variables *9*
5.4 Update Attribute in Data Dictionary *9*

6. Upgrade from beta 1.2 to 2.0 9
6.1. Entity Extraction *9*
6.2. Entity Resolution *9*
6.3. Relationship Extraction *9*
6.4. Relationship Resolution *9*
6.5. Enrichments *9*
6.6. Publisher *9*
6.7. Entity Explorer *9*
6.8. Orchestration Variables *9*
6.9. Context Variable *9*
6.10. Dockerfile *9*

7. Upgrade from beta 2.0 to 2.1 9
7.1. Entity Extraction *9*
7.2. Entity Resolution *9*

7.3. Publisher *9*
7.4. Context Variable *9*
8. Upgrade from 2.1 to 4.2 9
8.1. Entity Resolution *9*
8.2. Fragment Separation *9*
8.3. Mini SRDM Models *9*
8.4. Independent Deployment of KG Product Configurations *9*
8.5. Context *9*
9. Troubleshooting 9
9.1. General Spark Job Failures *9*
9.1.1. Config path does not exist. 9
9.2. General Orchestration Failures *9*
9.2.1. DAG import error in Airflow UI 9
9.2.2. Variable missing when pipeline starts running. 9
9.2.3. Task(s) not getting generated in airflow UI after deployment. 9
9.2.4. “None” value rendered for parsed-interval-start in orchestration variables. 9
9.2.5. Mediator dag: dag id not found. 9
9.2.6. Mediator dag: Multiple days run triggered simultaneously. 9
9.3. General UI Automation failures *9*
9.3.1. UI Rendering Issues 9
9.3.2. UI Automation Dag Failures 9

1. Introduction

Entity inventory is a one-stop solution for all the cyber analytical requirements where the user populates an entity inventory, creates relationships between entities, and use this data as a product to drive all the analytics. An inventory could be considered a knowledge base. It is meant to provide a single source of truth. It can facilitate bringing together knowledge of all assets into a single centralized location, to save having to go and consult many different records to provide a complete understanding of all assets.

The purpose of this document is to assist users in comprehending how entity inventory operates, configuring it, building it, and deploying it to a client environment.

1.1. Key Features of Entity Inventory

1.1.1. Entity Extraction

It refers to the procedure of extracting and enhancing entities with a comprehensive amount of information derived from various sources.
Example: It is possible to extract the Person entity from HR records, VPN records, event logs, and other sources. And can be supplemented with details such as date of entry, date of exit, address, location, social security number, login time, logon location, and so forth.

1.1.2. Entity Resolution

Entity resolution refers to the process of identifying and consolidating references to the same entity from various sources. When working across several data sources, it is common to encounter variances, inconsistencies, or duplications in the representation of entities. The objective of entity resolution is to rectify these inconsistencies and provide an accurate, unified representation for each unique entity. Various attributes or characteristics associated with the entities, including names, addresses, identifiers, or other relevant data, are assessed, and correlated in the procedure.

1.1.3. Relationship Extraction

Relations define the associations that exist between entities. There may be properties defined for a relationship that offer a further description of the relationship. This assists in acquiring source-level relationship information.
Relationships must be designated with a distinct name that specifies or categorizes the nature of the relationship. An inverse relationship ought to exist for each relationship. The terms "source entity" and "target entity" will be changed in this context to retain the same relationship name.

1.1.4. Relationship Resolution

Relationship resolution refers to the process of identifying and consolidating references to the same relationship from various sources. When working across several data sources, it is common to encounter variances, inconsistencies, or duplications in the representation of relationships. The objective of relationship resolution is to rectify these inconsistencies and provide an accurate, unified representation for each unique relation. Various attributes or characteristics associated with the relationship are assessed and correlated in the procedure.

1.1.5. Post Analytics Writeback

There will be instances in which the solution or client needs to enrich an entity based on their analytics after it has been extracted. Enrichments can be derived solely by inspecting the attributes of the same entity, from multiple rows of records from the source, external information, or from the attributes of other entities traversed via relationships.

The use of post-analytics writeback, helps Entity Inventory to store the enrichment acquired from the post-analytics, which are manifested as enrichment fields.

1.1.6. Graph Olap Layer

The Graph OLAP layer serves as an intermediary between the Puppy Graph and publisher layer, referencing original data files from publisher tables without duplicating data. This system implements a time-based data retention strategy that intelligently maintains snapshots at different intervals, keeping daily snapshots for recent data, transitioning to weekly snapshots for intermediate-age data, and retaining only monthly snapshots for older data.

The Graph OLAP layer significantly reduces storage costs and improves query performance by maintaining historical data at appropriate granularities while eliminating data duplication.

1.1.7. Entity Explorer

The Entity Explorer is a user interface (UI) that has been designed to enhance the exploration of entities and their relationships within the Entity Inventory. The Entity Explorer tool offers users the ability to analyse the attributes linked to entities, relationships, and enrichments from post analytics.

The Entity Explorer is powered by the Data Dictionary, which has comprehensive information regarding all the properties that are supported by the EI. The information encompasses various aspects, such as the display label that will be presented in the user interface (UI), the visibility of the attribute in the UI, the data type associated with the property, a description elucidating its purpose, and any additional pertinent details.

1.2. Data Quality Features

1.2.1. Data Quality Scoring

Data quality scoring provides a standardized way to assess the reliability and usability of entity attributes generated at different processing layers.
Example: attributes such as hostname, ip_address, and owner can be scored to indicate whether they are complete, valid, and trustworthy for downstream analytics.

1.2.2. Core Data Quality Dimensions

The framework evaluates data using well-defined dimensions Completeness to capture different quality characteristics for each attribute.

1.2.3. Layer-Aware Quality Measurement

Data quality is measured independently for each processing layer (Entity Extraction, Entity resolution….), enabling visibility into how quality improves or degrades through the pipeline.

1.2.4. Entity-Level Quality Aggregation

Dimension scores are aggregated into an overall data quality score for each entity, providing a single quality indicator that can be used for filtering, prioritization, and monitoring.

1.2.5. Data Quality Evidence Persistence

Detailed and summary quality results persisted in dedicated Data Quality tables so that historical trends, audits, and root-cause analysis can be performed consistently.

1.2.6. Entity Enrichment with Quality Values

Computed quality scores can be enriched back into entity records, allowing consumers to use quality indicators directly in graph analytics, search, and downstream applications.

1.2.7. Data Quality Insights and Configurability

Data quality rules are configuration-driven, allowing teams to tune dimensions, scoring logic, and thresholds based on entity type and source-specific behavior. These results can be surfaced through Data Quality Insights dashboards to help users quickly identify low-quality attributes and track improvements over time.The Data Quality configuration files define which quality dimensions are evaluated, how completeness is scored, which attributes are excluded, and where quality scores are enriched in entity and relationship flows.

2. Configuring Entity Inventory

2.1. Global Entity Configuration

Global entity configurations, as their name implies, pertain to entity-level settings. The primary purpose of implementing global entity configurations was to reduce the number of times the same configuration was used for sources of a particular entity. Additionally, it may be necessary to recalculate certain properties of an entity once entity resolution is complete; this can be accomplished via global entity configuration.

For example, the expression utilized to obtain the display label for an entity should be uniform throughout all loaders linked to that entity. Furthermore, the display label should be regenerated after entity resolution to include the entity's disambiguated attributes.

To propagate properties from the global entity configuration to all other configurations, a configuration handler script is executed against the configurations. Now if there is a case where the expression for a property is different for some of the sources/loaders, then it is possible to override the property in the corresponding loader configuration.

2.1.1. Config Details

Config Format

```
{
    "entityClass": "Name of the entity class",
    "commonProperties": [
        {
            "colName": "name of the field",
            "colExpr": "expression to derive the field",
            "fieldsSpec": {
                "isInventoryDerived": (true/false),
                "convertEmptyToNull": (true/false),
                "aggregateFunction": "name of aggregate function",
                "replaceExpression": (true/false),
               “caseSensitiveExpression": (true/false),
                "computationPhase": "all"
            }
        }
    ],
    "entitySpecificProperties": [
        {
            "colName": "name of the field",
            "colExpr": "expression to derive the field",
            "fieldsSpec": {
                "isInventoryDerived": (true/false),
                "convertEmptyToNull": (true/false),
                "aggregateFunction": "name of aggregate function",
                "replaceExpression": (true/false),
```

```
                "caseSensitiveExpression": (true/false),
                "computationPhase": "all"
            }
        }
    ],
    "lastUpdateFields": ["field1","field2"]
}
```

2.1.2. *Field Description*

Attribute	Usage	Is Mandatory
entityClass	Specifies the name of the entity	Yes
commonProperties	List of common fields and its extraction logic.	No
entitySpecificProperties	List of Entity specific fields and its extraction logic.	No
lastUpdateFields	List of fields for which last updated information is derived.	No

2.1.3. *Field Level Spec Details*

Attribute	Usage	Default	Is Mandatory
persistNonNullValue	The toggle feature for persisting and updating. If set to true, the job selects the most recent non-null value; otherwise, it proceeds with most recent value.	True, takes the latest non null val	No
isInventoryDerived	If set to true, the fields are calculated by the loader after the most recent inventory is made. Since SDM fields are not available in the inventory derived field calculation step, the attributes used in the expression must be in inventory.	False, by default fields are considered as sdm derived fields	No
convertEmptyToNull	Replaces the value of empty value fields in the expression to null of string type	True	No
replaceExpression	When enabled, the loader will check the configuration to see whether the attributes specified in the expression are defined there. If found, it will use the value from the configuration to replace the one in the expression. By doing so, the user can define the attribute once and then reuse it in another expression.	True	No

caseSensitiveExpression	In loaders, `spark.sql.caseSensitive` is set to true from orcehstration variables, implying that multiple cases of the same field are considered distinct. However, there are instances where a field may have values in one case but not in others. For fields derived from SRDM, the caseSensitiveExpression is defaultly set to false. This means that elements with different cases will be grouped together, and a coalesce function will be applied. If user need to uniquely map a specific field regardless of the presence of case-insensitive elements, it can be specified as true. Please note that case insensitive property will not be applicable to primary key, inventory derived and temporary properties fields.	False	No
computation_phase	The computation phase defines which stages of the processing pipeline (loader, inter, or all) should calculate a column expression to optimize performance.	loader	No
merge_strategy	The merge strategy allows you to combine column expressions from both client and solution configurations using COALESCE operations instead of completely replacing one with the other.	OVERRIDE	No

2.2. Entity Extraction

The entity extraction/loader job's primary objective is to retrieve the most recent information about an entity from a data source. To do this, the extraction job takes SRDM as input and does an aggregate on the primary key defined for that entity, which computes the most recent characteristics about the entity.

2.2.1. Key Functionalities

- Aggregation at the single entity level that aids in the extraction of information from a specific entity.
- Ability to persist the attribute if the latest value is null.
- Controlling persist/update behavior down to the field level.
- Capability to determine when a field was last updated and what its previous value was.
- Capability to consume solely the observed changes in a source relative to the previous run.
- A variety of field extraction strategies.
- SRDM support.

2.2.2. Entity ID

Each uniquely identified entity will be assigned an entity id to refer to. The generation of an entity id in each loader is guaranteed to be unique by employing the hash value of the class, the primary_key, the origin, and the attribute name of the primary_key.

Entity ID Generation Expression

```
SHA2(
 NULLIF(
   CONCAT_WS(
    '||',
    IFNULL(NULLIF(TRIM(CAST(primary_key AS STRING)), ''), '^^'),
```

```
    IFNULL(NULLIF(TRIM(CAST(origin AS STRING)), ''), '^^'),
    IFNULL(NULLIF(TRIM(CAST(class AS STRING)), ''), '^^'),
    IFNULL(NULLIF(TRIM(CAST(primary_key_attribute AS STRING)), ''), '^^')
   ),
 '^^||^^'
 ),
256
)
```

2.2.3. Entity Field Extraction Strategies

Following are the entity field extraction strategies available in entity inventory.

Strategy	Field Spec	Usage
Latest Non-Null Value	persistNonNullValue = true	Finds the latest non null value of an attribute by comparing the latest SDM data and previous inventory.
Latest Value	persistNonNullValue = false	Find the latest value of an attribute by comparing the latest SDM data and previous inventory and accepts it even if the value is null.
Aggregation	aggregateFunction	Runs the spark aggregation function against the attribute, here the aggregation is run against the latest data and the previous inventory. e.g.: min, max.
Inventory Derive Fields	isInventoryDerived	This is useful for calculating the value once the most recent inventory is built. One such application would be calculating a person's full name from their first and last names. It is important to note that only inventory fields can be referred in the derivation logic; SDM fields are not accessible during the inventory derived fields computation phase.
Merge	merge_strategy	The merge strategy allows you to combine column expressions from both client and solution configurations using COALESCE operations instead of completely replacing one with the other.
Case Sensitive Expression	caseSensitiveExpression	In loaders, spark.sql.caseSensitive is set to true from orcehstration variables, implying that multiple cases of the same field are considered distinct. However, there are instances where a field may have values in one case but not in others. For fields derived from SRDM, the caseSensitiveExpression is defaultly set to false. This means that elements with different cases will be grouped together, and a coalesce function will be applied. If user need to uniquely map a specific field regardless of the presence of case-insensitive elements, it can be specified as true. Please note that case insensitive property will not be applicable to primary key, inventory derived and temporary properties fields.
Convert Empty To Null	convertEmptyToNull	Replaces the value of empty value fields in the expression to null of string type.

It is important to remember that a field can only employ a single strategy, i.e., the aggregate function and inventory derived field strategies cannot be used for the same field.

2.2.4. *Smart Configuration Reorganizer*

Entity inventory loader configuration defines the properties that must be extracted from data sources; to do so, the user writes the corresponding extraction logic as spark expressions in the config. In most cases, there will be dependencies between the fields; one can only derive a property based on the value of another property. To address this challenge, the entity loader finds these attribute dependencies and rearranges the field order based on their logical ordering, allowing the user to apply the extraction logic without worrying about the logical ordering of the config attributes.

2.2.5. *Controlling Persist/Update Behaviour*

Entity allows the user to select the persist/update behaviour of a field, the user can set the persist logic in each loader or it can be set at a entity level (See *global entity config*).

Setting Persist behavior in a loader

```
{
    "colName": "name of the field",
    "colExpr": "expression to derive the field",
    "fieldsSpec": {
        "persistNonNullValue": (true/false)
    }
}
```

2.2.6. *Lookup Enrichment*

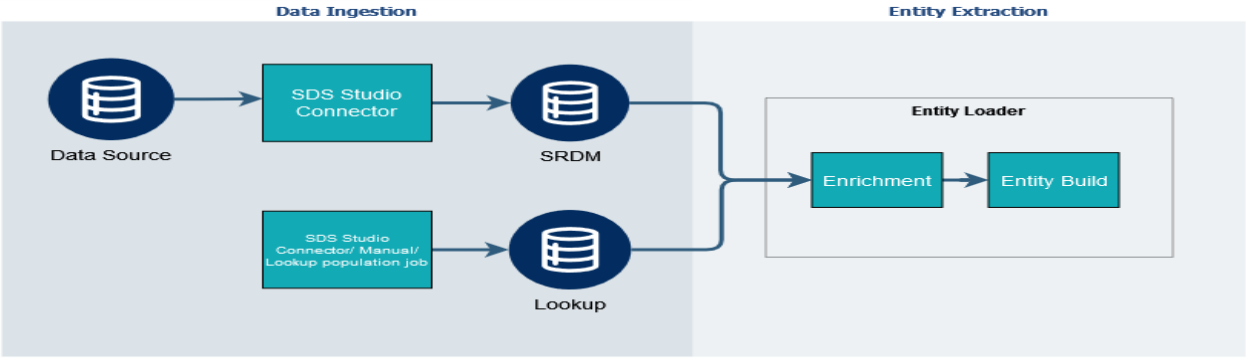
Lookup based enrichment in the context of entity inventory refers to the process of retrieving information from a reference dataset based on a key attribute in the inventory. The lookup sources contributed to the entity will have a field called `origin_lookup_enrich`. If the lookup table has an `updated_at` field, the job automatically filters the records with the current `updated_at` before joining with the enrich table.

Example:

> IP Geolocation Lookup - Finding the geographical location associated with an IP address using an IP geolocation data, this IP address is an attribute present in an entity (A Host entity may contain an IP)
> Country Code Mapping - Translating country codes (e.g., 'US', 'IN') to country names (e.g., 'United States', 'India') using a reference table.

Lookup Enrichment Support in Entity Loader

Lookup-based enrichment support in the loader enables users of the entity inventory to enhance the attributes of an SRDM.



Config Format

```
{
 "enrichments":[
        {
            "lookupInfo":{
                "tableName": "<%EI_LOOKUP_SCHEMA_NAME%>.eol_lookup",
                "preTransform": [
                    {
                        "colName": "eol_status",
                        "colExpr": "normalization_result.is_eol"
                    },
                    {
                        "colName": "eol_date",
                        "colExpr": "case when normalization_result.eol_date='NO
MATCH' then NULL else
UNIX_MILLIS(TIMESTAMP(to_timestamp(normalization_result.eol_date))) end"
                    },
                    {
                        "colName": "normalized_os",
                        "colExpr": "normalization_result.normalised_name"
                    }
                ],
                "enrichmentColumns": [
                    "eol_status", "eol_date","normalized_os"
                ]
            },
            "joinCondition": "lower(s.os)=lower(e.os_string)"
        }
]
}
```

Field Description

Attribute	Usage	Is Mandatory
preTransform	Array of expressions that need to be computed in the lookup before the join.	No
sourcePreTransform	An array of expressions to be computed in enriching table before performing the join.	No
`enrichmentColumns`	The column need to be enriched from to lookup from source	yes
`joinCondition`	Expression used to define the join condition.	yes

2.2.7. Lookup Enrichment / OS Extraction | EoL-EoS

OS Extraction PySpark job standardises and enriches operating system (OS) information from multiple source tables, ensuring consistent and accurate OS representation across the data pipeline. It uses the pai-os-normaliser library to clean and normalise raw OS strings, then augments them with End-of-Life (EoL) and End-of-Service (EoS) details to support the knowledge graph enrichment process. The job is designed to run as part pipeline it should come before the inter jobs

The normalisation process follows these key steps:

- Family & Version Identification – Extract the OS family from family_map.json and detect the major version where possible (e.g., "2019" from "Windows Server 2019"). Noise words are removed, and formatting is standardised.
- Candidate Filtering – Select potential matches from the product database based on OS family, and further narrow the list by version to avoid mismatches (e.g., preventing "Windows Server 2012" from matching "Windows Server 2019").
- Fuzzy Matching & Scoring – Compare the cleaned OS string to candidates using the rapidfuzz library, keeping only matches above a confidence threshold (default: 50).
- Result Normalisation & Enrichment – Output the standardised OS name along with EoL/EoS status, dates, and other relevant metadata.

Config Format

```
{
    "enrichments":[
        {
            "lookupInfo":{
                "tableName": "name of the input table",
                "name": "Name of the lookup"
                "filter": "filter to apply on the lookup table",
                "preTransform": [
                    {
                        "colName": "name of the column",
                        "colExpr": "expression"
                    }
                ],
                "enrichmentColumns": ["list of final columns to take from lookup table"]
            },
            "sourcePreTransform": [
                {
                    "colName": "name of the column",
                    "colExpr": "expression"
                }
            ],
            "joinType": "LEFT|INNER",
            "joinCondition": "Join expression"
        }
    ]
}
```

2.2.8. Entity Extraction Using View Based Strategy

- The entity extraction module processes and merges data from three primary sources: Incremental SRDM, which includes filtered records from the source for the current run; Previous Inventory, representing the latest known state of the data from the last successful run; and Historical SRDM, which is used when changes are made to the logic of SRDM-derived fields, ensuring accurate re-evaluation using past data.
- Combining these inputs, the Knowledge Graph extracts the most current representation of the source data. After extraction, it computes several categories of fields Common Fields such as recency (the difference between updated_at and last_found timestamps), Last Updated Fields which track changes between runs, and Inventory-Derived Fields which are user-defined transformations configured via files (typically comprising 40–60% of the total fields).

- Optimize performance and reduce storage overhead, these final derivations are implemented as views rather than materialized tables. This view-based approach significantly reduces read/write operations on extracted tables while maintaining flexibility and efficiency in the pipeline.
- The entity extraction job generates a base table named by appending "__srdm_inv" to the configured output table name. The final output is created as an Iceberg view on top of this base table, using the original output table name from the config as the view name.
- Iceberg view creation is supported starting from Spark version 3.5.5 and Iceberg version 1.9. These views behave like regular tables and can be queried seamlessly via both Spark shell and Thrift interfaces.
- One of the key benefits of using a view is that, as mentioned, it is a spark SQL definition built on top of the base table. Since it doesn't rely on partitions like updated_at_ts, any changes made to the logic for fields such as last updated fields, common fields, or inventory-derived fields will automatically apply across the entire partition of data.
- The default behavior of entity extraction is to write the output as a view.

2.2.9. Job Execution Flow

- Read SRDM/datasource and filters the latest day data using parsed interval timestamp
- Reads previous inventory
- Add lookup enrichment columns from lookup table
- Generate fields from SRDM
    - Transform properties into case sensitive properties.
    - Converts all empty string fields from SRDM to null.
    - Converts struct fields from config into temporary fields and substitute it in properties as well.
    - Adds any missing fields to Data frame
    - Generate temporary fields defined on the SRDM
    - Generate Primary Key
    - Filters SRDM using event timestamp to get the latest day's data only
    - Apply user defined filter in the config and primary key null filter on the Data frame
    - Generate SRDM based transformation defined on the config.
    - Compares with previous inventory and apply the latest value finding strategies for each of the field. See *Entity Field Extraction Strategies*
    - Apply salting if it is defined
    - Remove the missing null type columns.
    - If written type is view create a base table ending with __srdm_inv, otherwise skip the write task.
- Generate Inventory Derived Fields
    - Add core fields and p_id calculation
    - Add missing fields to data frame
    - Converts struct fields from config into temporary fields and substitute it in properties as well.
    - Apply temporary properties defined as inventory derived fields
    - Generate the inventory derived fields
    - Finds recency, lifetime, observed_lifetime and recent activity fields
    - Finds the last updated fields compared to previous inventory
    - Generate the inventory derived fields which uses last updated attributes field
- Selects only the fields defined in the config and the inventory core fields and finally remove null type missing fields.
- Create a table or view on top of the base table created above based on the written mode defined in the configuration.

2.2.10. Config Details

Config Format

```
{
  "primaryKey": "primary key expression",
```

```
"operationalMode": "(dailyReset/carryForward)",
"filterBy": "any filter condition",
"origin": "'Origin Name'",
"dataSource": {
  "name": "source name",
  "feedName": "feed name",
  "srdm": "input source table",
  "dataIntervalTimestampKey":"timestamp column used for scanning data ranges.",
"dataEventTimestampKey": "timestamp column used for filtering scanned data.",
"uniqueRecordIdentifierKey ": "column used for uniquely identifying for each record",
},
"outputTableInfo": {"outputTableName": "output table name ",
                    "outputWrittenMode": "tableType/viewType"
},
"enrichments": [
  {
    "lookupInfo": {
      "tableName": "table name",
      "enrichmentColumns": [
        "array of enrichment columns"
      ],
      "preTransform": [
        {
          "colName": "name of the field",
          "colExpr": "expression to derive the field"
        }
      ]
    },
    "joinCondition": "s.source_column_name = e.enrichment_column_name",
    "sourcePreTransform": [
      {
        "colName": "name of the field",
        "colExpr": "expression to derive the field"
      }
    ]
  }
],
"temporaryProperties": [
  {
    "colName": "name of the field",
    "colExpr": "expression to derive the field",
    "fieldsSpec": {
      "isInventoryDerived": (true/false),
      "convertEmptyToNull": (true/false)
    }
  }
],
"commonProperties": [
  {
    "colName": "name of the field",
    "colExpr": "expression to derive the field",
    "fieldsSpec": {
      "isInventoryDerived": (true/false),
      "convertEmptyToNull": (true/false),
      "aggregateFunction": "name of aggregate function",
      "replaceExpression": (true/false),
```

```
        "persistNonNullValue": (true/false),
        "caseSensitiveExpression": (true/false)
      }
    }
  ],
  "entitySpecificProperties": [
    {
      "colName": "name of the field",
      "colExpr": "expression to derive the field",
      "fieldsSpec": {
        "isInventoryDerived": (true/false),
        "convertEmptyToNull": (true/false),
        "aggregateFunction": "name of aggregate function",
        "replaceExpression": (true/false),
        "persistNonNullValue": (true/false),
        "caseSensitiveExpression": (true/false)
      }
    }
  ],
  "sourceSpecificProperties": [
    {
      "colName": "name of the field",
      "colExpr": "expression to derive the field",
      "fieldsSpec": {
        "isInventoryDerived": (true/false),
        "convertEmptyToNull": (true/false),
        "aggregateFunction": "name of aggregate function",
        "replaceExpression": (true/false),
        "persistNonNullValue": (true/false),
        "caseSensitiveExpression": (true/false)
      }
    }
  ]
}
```

Config Element Details

Attribute	Usage	Is Mandatory
primaryKey	Spark expression for primary key field derivation; it can only reference fields defined in the temporary property portion of the configuration and the SDM.	Yes
operationalMode	The operational mode defines whether entities persist across multiple days in the inventory system, with "carry-forward mode" preserving entities until explicitly removed, while "daily-reset mode" removes all entities not present in each day's data, ensuring each day starts with only current entities.	No
filterBy	A filter condition that can be applied to a data source/SDM to retrieve only valid entries for the entity from a given source.	No

origin	A label to identify the data source, the value should be enclosed in a single quotation mark.	Yes
dataSource	The "dataSource" parameter contains information about the source table.	Yes
outputTableInfo	The "outputTableInfo" parameter specifies the output table details for the loader.	Yes
temporaryProperties	This section of the configuration can be used to define the temporary properties from which the entity properties can be derived. The fields defined in this section will not be included in the entity's attributes. The fields mentioned here can also be used during relationship building, allowing you to extract logic from srdm and utilize them as block variables for constructing relationships.	No
commonProperties	Defines the entity's common properties.	No
entitySpecificProperties	Defines the entity's entity specific properties.	No
sourceSpecificProperties	Defines the entity's source specific properties.	No
enrichments	Contains information regarding enrichment columns to be added	No

See *Field Spec Details*

2.3. Mini SRDM Creation

Mini SRDM table is a mini version of SRDM tables that holds the latest information for each entity at the SRDM level. It is produced by the Mini SRDM Loader job, which reads from the full SRDM source, merges with prior inventory, and writes a reduced table that keeps only the most recent record per entity. Unlike the full entity-inventory tables, the mini SRDM table omits inventory-specific columns (such as origin, lifetime, recency, observed_lifetime, recent_activity, and similar fields) and instead exposes a lean, SRDM-style schema. It is currently used for the DQ completeness job.

2.3.1. Mini SRDM Generation Logic and Configuration

Do not maintain a separate configuration for the mini table. Instead, use the same loader configuration that is used for the main job. From this configuration, the job reuses the entity definition, primary key, filter, origin, and data source.

The list of output columns for the mini srdm table is not defined in the configuration. Instead, it is derived directly from the SRDM source, all columns from the source are included except the unique record identifier and the reserved primary key column. Map-type fields are stored as JSON, while the remaining fields are stored as-is. For each primary key, the job determines the latest value for each column, ensuring the mini srdm table always reflects the current SRDM structure without requiring each column to be explicitly listed.

Only temporary properties that are not inventory-derived are used for the mini srdm table, inventory-derived temporary properties are excluded. The same logic used by the main loader is applied here, ensuring the mini table remains consistent with how the main inventory processes the same source.

The Mini SRDM generation job uses the same arguments as the Loader job. The output table name is taken from the loader configuration field `outputTableInfo.outputTableName`, while the schema is defined using the Spark configuration `spark.kg.mini_srdm_schema`. For example, if the configuration output table is `ei_temp.sds_ei__host__active_directory__object_guid` and `spark.kg.mini_srdm_schema` is set to `ei_util`, the mini SRDM table will be written to `ei_util.sds_ei__host__active_directory__object_guid`.

2.4. Entity Resolution

The entity resolution/disambiguation job reads data from various mini inventories and attempts to merge multiple entities into a single entity using common key properties that exist among them; these key attributes are referred to as candidate keys.

Entity resolution job is divided into two primary module.

- Disambiguation Group Identification.
- Attributes Resolution.

2.4.1. Disambiguation Group Identification

It is the process of grouping similar entities together based on their candidate key values. The entity resolution job finds a relationship between two entities based on their candidate key values; to accomplish this, the entity inventory uses a graph-based disambiguation approach. Additionally, these candidate keys are made case insensitive, meaning that all variations of the same key are considered identical, enhancing entity resolution.

Entity inventory creates a graph with the entity id and the lower of candidate key value as the vertex, and an edge is created between the candidate key and the entity id. Finally, the connected component algorithm is applied to this graph in order to group related entities together.

The entity inventory enables the user to manage the disambiguation classification through the use of the following controllers:

- Key to Key Match : To ensure maximum disambiguation, the entity inventory by default examines only the values of candidate keys when plotting the graph vertices; it does not evaluate which candidate key the value is originating from. The user can enable key to key match for a candidate key to instruct the resolver to consider the key as well when matching.
- Exception Filter : The user can instruct the resolver to disregard the value of a candidate key when a given condition is met.
- Case Sensitive : To ensure maximum disambiguation, candidate keys are made case insensitive by converting to lower case. This can be controlled explicitly by setting caseSensitive to true .
- Match Attribute List: Entity resolution module includes an option to invalidate a candidate key if it contains more than one value for an attribute in the fragments. For example, a user may wish to consider a host name for a candidate key only if it has a single fqdn associated with it. When match attribute list is set then key to key match must be true.

Type of the candidate key can be string or array of string, when a candidate key is given as an array value, the dismabiugation grouping modules explodes the value of the key and creates an vertex for each of the element and creates an edge to it's entity id.

Candidate Key and Match Attribute Evaluation for Disambiguation

- The values of the columns host_name, cloud_instance_id, and os start with specific prefixes:
  - host_name → prefix hst (examples: hst1, hst2, hst3)
  - cloud_instance_id → prefix cid (examples: cid1, cid2, cid3)
  - os → prefix os (examples: os1, os2, os3)
- The configuration explains how the system evaluates candidate keys (such as host_name and cloud_instance_id) together with match attributes (such as os and cloud_instance_id) to determine whether multiple records represent the same entity.
- If the candidate keys and match attributes indicate a relationship between records, those records are grouped together and unified under a single disambiguated ID.

Example configuration:

Candidate Keys: host_name, cloud_instance_id

matchAttributesList: ["os, cloud_instance_id"]

Scenario	Input Summary	Merge Rule	Expected Output
Same candidate key, different values for match attribute list columns.	3 records • hst1, cid1 • hst1, cid2 • hst1, cid2	Two records sharing the same non null cloud_instance_id are merged	• cid2 records are merged into 1. • cid1 stays separate. • Output contains 2 records
Null and non null combination of match attribute list with same candidate key.	5 records • hst2, cid3 • hst2, null • hst2, null • hst2 cid4 • hst2 cid4	• Two null values are merged • Two identical non nulls are merged	• Null records of cid are merged into 1 • cid3 stays separate • cid4 into one record • output contains 3 records
Two null and one non null combination of match attribute list	3 records • hst3, cid3 • hst3, null • hst3, null	Two nulls and a non-null value merged into 1.	Output contains 1 record
Combination with two match attribute list columns	4 records • hst3, null, os1 • hst3, null, os1 • hst3, cid3, null • hst3, cid4, os2	Two records with same os value and null cid are merged	• os1 values of os are merged into 1 • cid3/null and cid4/os2 stay as separate records • output contains 3 records

2.4.2. Attributes Resolution

Once related entities have been grouped together, the next step is to extract relevent information about the entity from all of the non-resolved entities in order to build a unified representation of the resolved entity.

The following are the different strategies available in the entity inventory for resolving attributes from unresolved entities:

2.4.3. Source Based Confidence Matrix

The source-based confidence matrix is the primary strategy that will be applied to resolve an attribute when no other strategy is defined for an attribute. In this context, a priority is assigned to each data source, and the value of an attribute is retrieved from the source that has the highest priority and returns it's latest non-null value, When the source contains only null values, the resolver proceeds to select the next source that is higher in the confidence order.

In certain situations, an entity may have multiple records with the same last found date from the same source. In such cases, the resolver selects the record containing the highest entity id (highest entity id is only considering the characters in the entity id and no other information about the entity is taken into account). Thus, the deterministic character of the entity inventory is ensured. The selection based on last_found_date can be overridden by utilizing temporalConfidenceMatrix. This feature enables the retrieval of field values not only based on last_found_date but also allows for specifying other fields such as last_active_date, first seen_date, first found_date, and so on, as needed.

Priority resolution order

- Source Confidence
- Last Found Date

- Entity ID (p_id)

Field Level Source Confidence Matrix

In certain instances, adhering to the default confidence matrix may not yield the most accurate outcome for certain attributes. In such situations, it may be necessary for the user to supply an individual confidence matrix for such attributes. A field-level confidence override enables the user to provide a confidence matrix at the attribute level.

It is not mandatory to specify confidence values for every source in the field-level confidence matrix. Instead, the disambiguation task restructures the field-level confidence matrix by combining the overridden confidence with the unmentioned sources from the primary confidence matrix. This can be limited using the RestrictToConfidenceMatrix spec.

Further control over the value selection is controlled by the following specs

- PersistNonNullValue: The toggle feature for persisting and updating. If set to true, the resolver selects the most recent non-null value; otherwise, it proceeds with the most recent value from the high-priority source observed for the entity.
- RestrictToConfidenceMatrix: When set to true, the entity inventory resolves the attribute using only the overriden confidence matrix.
    - When overriding the confidence matrix, and the field values should be selected only from the overridden confidence configuration, the parameter restrictToConfidenceMatrix must be set to true.
    - Setting restrictToConfidenceMatrix = true ensures that the system considers only the sources defined in the overridden confidence matrix for resolving that field.
    - If restrictToConfidenceMatrix is not set to true, and all values for a field from the sources specified in the overridden confidence matrix are null, the system will fall back to the remaining sources defined in the global confidence matrix to resolve the field.
- TemporalConfidenceMatrix: This feature allow users to fetch field values not only based on last_found_date, but also any other fields as needed, such as last_active_date, max of first seen, and max of first found. By default, last_found_date is used, but users can change this by giving a list of fields in the precedence order within the "temporalConfidenceMatrix" attribute. If last_found_date is included in the list of fields, its default nature will be overridden, and the specified fields will be prioritized. However, if last_found_date is not included in the list, it retains precedence by default, last_found_date followed by other fields in the list.
    - The temporal confidence matrix can be overridden to prioritize record selection based only on temporal attributes.
    - Example configuration

```
{
"field":"login_last_user",
"temporalConfidenceMatrix":[
 "last_active_date",
 "last_found_date"
 ],
 "confidenceMatrix":[],
 "restrictToConfidenceMatrix":true
 }
```

    - To ensure records are selected only using the temporal confidence matrix:
        - Set confidenceMatrix to an empty array ([]).
        - Set restrictToConfidenceMatrix to true.
    - The priority of temporal fields is determined by the order defined in temporalConfidenceMatrix.
        - First, select records with the maximum value of last_active_date.
        - If multiple records have the same maximum value, use the next temporal field (last_found_date) to break the tie. If it is also the same, the maximum of p_id is used as the fallback.

Value Based Confidence

In a value-based confidence strategy, confidence is set for field values rather than the source itself. The resolver selects the highest observed value for an attribute as the attribute value of the resolved entity.

Rolling Up Fields

This method enables the user to store every observed value of an attribute in the resolved entity. The resolver accumulates all seen values and stores them in an array within the resolved entity. The observed values are ordered using the primary source confidence matrix.

Aggregation Based

This method enables the user to apply an aggregate function to all observed values of an entity attribute; this is useful when the requirement is to retrieve the minimum and maximum value of an attribute from non-resolved entities.

Frequency Based Confidence

The Frequency Confidence strategy resolves field conflicts by selecting the most frequently occurring value across all data sources. For a given entity, if multiple region values are received from different sources, the system resolves the value using a frequency-based strategy.In this approach, the region value that occurs most frequently occurred is selected for the entity.If multiple region values have the same frequency, the system selects one of the values randomly.

2.4.4. Controlling Persist/Update Behaviour

The field level disambiguation strategy allows the user to specify the persist/update behaviour of a field in disambiguation, The user is required to specify the source from which the value of a field should be taken, given that disambiguation comprises numerous unresolved tables. If the user wants to select the field from a single source, then enable restrictToConfidenceMatrix, which makes the resolver scan the values of the field only from the sources defined in the overriden source matrix. *See Field Level Confidene Matrix Strategy for more details*.

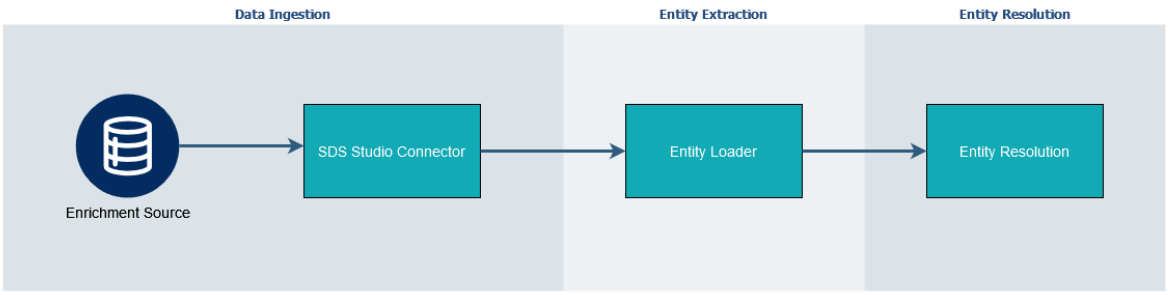
It's important to note that the persist update behaviour set in the global entity config has no impact on disambiguation configuration, the user is required to configure intra and inter disambiguation job for persist/update behavior.

Config Format

```
{
    "strategy": {
        "fieldLevelConfidenceMatrix": [
            {
                "field": "name of the field",
                "confidenceMatrix": [
                    "source 1", "source 2"
                ],
                "persistNonNullValue": false,
                "restrictToConfidenceMatrix": (true/false),
                "temporalConfidenceMatrix": ["List of date fields"]
            }
        ]
    }
}
```

2.4.5. Entity Enrichment

We start by running entity loader jobs to gather data from different enrichment sources, then use Entity Disambiguation to accurately match and link these new entities with existing ones, allowing us to combine information about the same entity from multiple sources.



The following functionalities is incorporated into the entity resolution job:

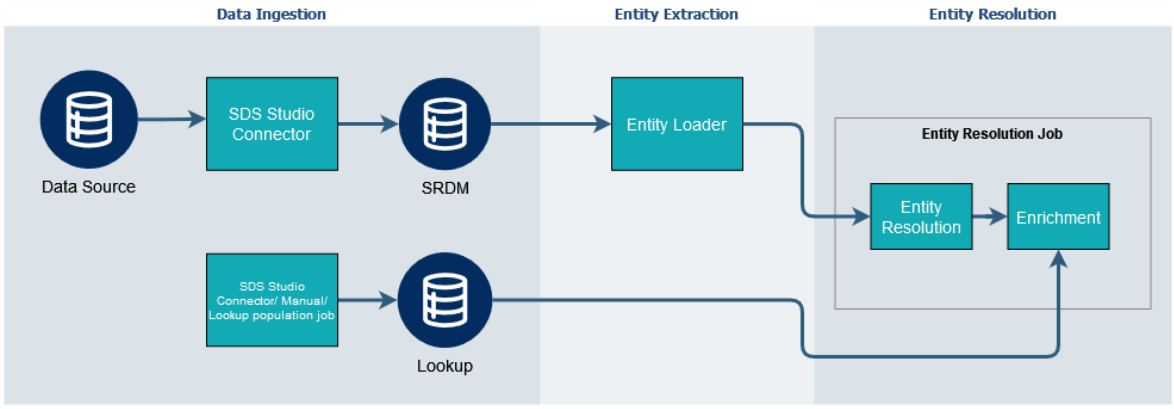
An option to specify the input model is an enrichment source
Ability to remove certain attributes from an inventory model before passing to the disambiguation
A new field called **origin_entity_enrich** to store the entity enrichment origin names (the origin field will have the entity enrichment sources)

Config Format

```
{
"inventoryModelInput": [
    {
      "path": "iceberg table name",
      "name": "an alias name",
      "isEnrichSource" : true,
      "filter": "an optional filter to apply on the table",
      "fieldsToRemove": [list of fields to remove]
    }
}
```

2.4.6. Lookup Enrichment Support in Entity Resolution

This feature enables the users of entity inventory to enhance the attributes of an entity using an external lookup.



The lookup joins for entity resolution doesn't have support for One-to-Many association, as it will duplicate the entity records.

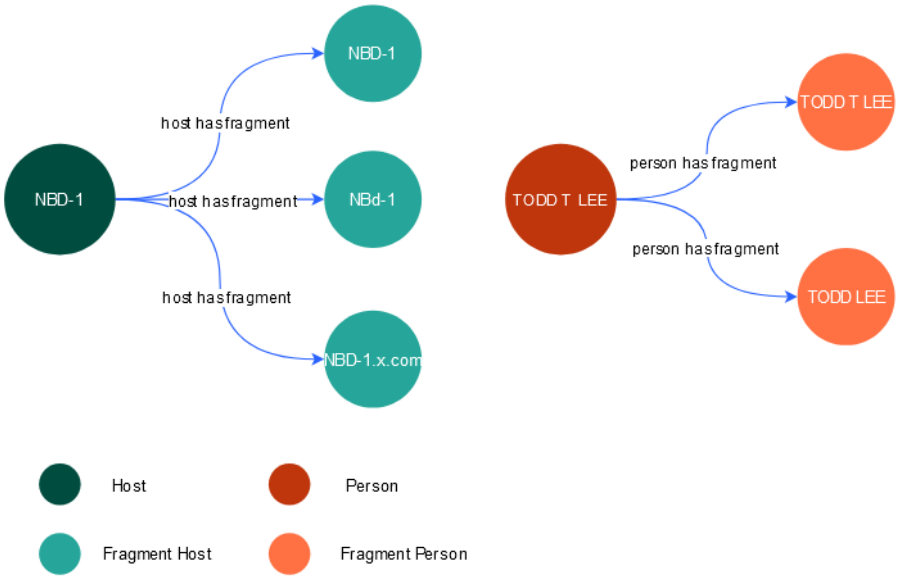
Config Format

```
{
    "enrichments":[
        {
            "lookupInfo":{
                "tableName": "name of the input table",
                "name": "Name of the lookup"
                "filter": "filter to apply on the lookup table",
                "preTransform": [
                    {
                        "colName": "name of the column",
                        "colExpr": "expression"
                    }
                ],
                "enrichmentColumns": ["list of final columns to take from lookup table"]
            },
            "sourcePreTransform": [
                {
                    "colName": "name of the column",
                    "colExpr": "expression"
                }
            ],
            "joinType": "LEFT|INNER",
            "joinCondition": "Join expression"
        }
    ]
}
```

2.4.7. Fragment and Resolver Table

An entity can exist across multiple data sources, and an entity resolution process consolidates these fragments into a single resolved entity. In a knowledge graph, it is important to capture the association between the fragment entities and the resolved entity. This ensures traceability and data integrity within the graph.

Each entity will have its own individual fragment and resolver tables. Fragment tables, named with the prefix "Fragment" (e.g., Fragment Host), will include properties like graph.vertex.name and graph.cache.enabled, with graph IDs based on the fragment class, unique IDs, and updated_at_ts. Resolver tables define the relationships between resolved and nonresolved entities, with properties such as graph.edge.name, graph.edge.source.name, graph.edge.target.name, and graph.cache.enabled. Both fragment and resolver tables will reside in a dedicated Iceberg schema, KG_FRAGMENT_SCHEMA, separate from the main Knowledge Graph schema. Table names and OLAP configurations will be configurable but it is also autogenerated from config merger.



2.4.8. *Job Execution Flow*

- Read Previous inventory
- Loads all the non resolved entity tables
- Add the missing fields
- If disambiguation strategy is **union**, skip the following steps:
  - Finds the disambiguation group using the candidate key
    - Creates a vertex for each of the candidate key and entity id
    - An edge is created between the entity id and the candidate key value
    - Apply the connected component algorithm and creates a group id for each of connected component, this is used as the disambiguation group id.
  - Creates a confidence matrix
    - This modules gives confidence for the sources and fields based on the disambiguation strategy defined in the config, See *Attribute Resolution*
  - Performs the disambiguation of the non resolved inventory on each disambiguation group using the confidence matrix creates earlier.
  - If output written type is view, create a base table ending with __resolved_inv, otherwise skip the write operation
- Computes the common fields defined in the *global entiy config* with computation_phase value set as inter/all.
- Finds the latest updated attributes compared to previous inventory
- Add lookup enrichment columns from lookup table
- Generates the derived fields defined in the config
- Filters the final inventory using the filter condition provided in the configuration.
- Write it as a table or view on top of the base table created based on the configuration.

2.4.9. *Config Details*

Disambiguation configuration

```
{
  "inventoryModelInput": [
    {
      "path": "table name with schema",
      "name": "alias/table name, this name will be used in the confidence matrix",
      "filter": "an optional filter to apply on the table"
```

```
      "isEnrichSource": (true/false)
    }
  ],
  "disambiguation": {
    "candidateKeys": [
      "key 1",
      {
        "name": "key 2",
        "exceptionFilter": "exception filter condition",
        "keyToKeyMatch": (true/false),
       caseSensitive": (true/false)
      },
      {
        "name": "key 3",
        "matchAttributesList": [""]
      },
      {
        "name": "key 4",
        "disable_key": true
      }


    ],
    disambiguationType: "(union/ CandidateKey)",

    "confidenceMatrix": [
      "List of sources in order of higher to lower priority"
    ],
    "excludeValues": [
      "List of low priority values when resolving, eg: Other,Unknown, etc"
    ],
    "strategy": {
      "fieldLevelConfidenceMatrix": [
        {
          "field": "name of the field",
          "confidenceMatrix": [
            "List of sources in order of higher to lower priority"
          ],
          "persistNonNullValue": (true/false),
          "restrictToConfidenceMatrix": (true/false),
          "temporalConfidenceMatrix": ["List of date fields"]
        }
      ],
      "rollingUpFields": [
        "list of field names"
      ],
      "aggregation": [
        {
          "field": "name of the field",
          "function": "aggregation function, eg max"
        }
      ],
      "valueConfidence": [
        {
          "field": "name of the field",
          "confidenceMatrix": [
            "list of values in order of higher to lower priority"
          ]
        }
      ],
      "frequencyConfidence":[
      {
        "field": "The field name to apply frequency-based resolution",
        "fallbackValueConfidence": [
         "Ordered list of preferred values for tie-breaking scenarios"
```

```
            ]
        }
        ]
      }
    },
    "derivedProperties": [
      {
        "colName": "name of the field",
        "colExpr": "expression to derive field."
      }
    ],
    "output": {
      "disambiguatedModelLocation": "resolved entity table name with schema",
      "resolverLocation": "entity resolver table name with schema",
      "resolverGraphLocation": "table captures the granular details of how entities are
linked together during the disambiguation workflow",
      "filter": "an optional filter to apply on the final inventory model."
    }
}
```

Config Element Details

Attribute			Usage	Is Mandatory
inventoryModelInput	Each of the inputs has the name of the non-resolved table name and an alias name used throughout the disambiguation job.			Yes
	path	Non resolved table name with schema.		Yes
	name	Alias name to use in the confidence matrix		Yes
	filter	An optional filter to apply on the non-resolved table		No
	isEnrichSource	An optional field indicates whether the input comes from an enriched source.		No
disambiguation	Holds the core configurations for controlling the disambiguation job			Yes
	candidateKeys	List of candidate keys, user can input the just the key name if no customization is needed		Yes
		name	Name of the candidate Key	No
		exceptionFilter	The condition to disregard the candidate key from grouping	No
		keyToKeyMatch	If set true, the resolver looks for the values only the given key name, it won't search the value in other keys	No
		caseSensitive	If set false, candidate keys are made case insensitive by converting to lower case which results in maximum entity resolution.	No
		disable_key	When enabled, disable_key removes the specified candidate_key from the disambiguation config during the merge.	

		matchAttributesList	Option to invalidate a candidate key if it contains multiple values for a specific attribute across fragments. These attribute values are represented as an array of strings.	No
	confidenceMatrix	A list specifying the order in which the values for each attribute should be taken from input sources.		Yes
	disambiguationType	Specifies the approach used for the disambiguation		No
	excludeValues	A list specifying string values with the least priority (but higher priority than null value) while resolving value of an attribute.		No
	strategy	Specifies the strategies to be used for disambiguation. Each of the strategies have different sub-configs. See *Resolution Field Extraction Strategies*		No
derivedProperties	This section has configurations of existing or new fields which are recalculated/derived using new expressions after performing disambiguation.			No
	name	Name of the field		Yes
	colExpr	Expression to derive the field		Yes
output	As the name suggests, has the table names of output and resolver table.			Yes
	disambiguatedModelLocation	Resolved entity table name with schema		Yes
	fragmentLocation	Non resolved entity table name with schema (autogenerated from config merger api)		Yes
	resolverLocation	Resolver table name with the schema (autogenerated from config merger api)		Yes
	filter	An optional filter to apply on the final inventory		No

2.4.10. Resolution Field Extraction Strategies

Strategy	Description	**Sub-Strategy**	
rollingUpFields	returns an array containing the unique values from a specific field across all fragments (contributing sources).	NA	
fieldLevelConfidenceMatrix	The Field-Level Confidence Matrix strategy allows you to	confidenceMatrix	Custom precedence order for this field, overriding global order

	override global disambiguation rules for specific fields. It provides field level control through multiple sub-strategies.	persistNonNullValue	Whether to prefer any non-null value over a null value from higher precedence source
		restrictToConfidenceMatrix	If true, only accept values from sources listed in confidenceMatrix; reject all others
		temporalConfidenceMatrix	This enables time-based precedence for field resolution, allowing users to prioritize entity data based on multiple temporal fields beyond the default last_found_date.
valueConfidence	The Value Confidence strategy assigns precedence based on specific field values rather than data sources. This allows the system to prioritize certain values over others within the same field, regardless of which source provided the data.	confidenceMatrix	Ordered list of field values from highest to lowest precedence
frequencyConfidence	The Frequency Confidence strategy resolves field conflicts by selecting the most frequently occurring value across all data sources, with intelligent tie-breaking mechanisms to ensure consistent and meaningful results.	fallbackValueConfidence	For values with equal highest frequency, prioritize based on their order in the fallbackValueConfidence list. If no tied values are found in the fallback list, select randomly from the tied values.
aggregation	Applies the specified aggregation function during the disambiguation phase.	function	The aggregation function to apply (max, min, sum, avg, count, etc.)
singleSource	The Single Source strategy enforces that a specific field must always come from one designated data source, regardless of global confidence matrix or other disambiguation strategies.	source	The exact data source name that has authority for this field
		persistNonNullValue	Whether to fall back to other sources if the given source has null

2.5. Entity Fragment Write

Fragment creation is introduced as a standalone analytics job to generate entity fragment tables. Fragment generation is executed independently and writes fragments using the standardized naming pattern sds_ei__fragment__{entity_name}. It is designed to improve overall runtime by decoupling fragment computation and allowing it to run in parallel with the relationship pipeline.

2.5.1. Job Execution Flow

- Read the resolver table and filter it by the current run timestamp.
- Read the previous fragments filtered by the previous update date.
- Read all input tables required for disambiguation.
- Normalize the input tables.
- Recalculate all property fields.
- Compute the last updated attributes.
- Enrich the data with graph details.
- Normalize complex data types (convert structs to JSON and serialize arrays).

Refer

2.6. Relationship Extraction

A relationship defines the association that exists between two entities. To build a relationship, all information about the source entity and target entity must be accessible from a single source.

2.6.1. Relation Input Support

The Knowledge graph supports a more flexible approach to relationship building.The knowledge graph supports three relationship creation methods: traditional resolver-based relationships when disambiguation is needed, direct table-based relationships using join conditions when no resolver is necessary, and purely loader configuration based relationships with no table involvement. New configuration options (configPath, tableName, and joinCondition) under sourceMappingInfo and targetMappingInfo enable these capabilities, making relationship creation more straightforward while reducing unnecessary complexity. This enhancement streamlines the overall solution by removing redundant components and simplifying relationship management. For relationship building from non-SRDM sources, the config keys "dataIntervalTimestampKey", "dataEventTimestampKey" are added under the inputSourceInfo. These keys are added as part of filtering the non-SRDM source. The "uniqueRecordIdentifierKey" also added in the inputSourceInfo that helps to find the unique record of non-SRDM source.

2.6.2. Relation Block Building

There can be multiple occurrences of the relationship between the same source and target object, which can be isolated using a block-building strategy. A single relationship is contained within a single block, and entity inventory uses the record in the block to populate the properties of the relation.

Following are the block-building strategies available in the entity inventory.

Variable Based

This method enables the user to construct relationship blocks using some key columns called block variables, and it groups records with similar values in the same block, which is then used to do relationship extraction. When one of the values in the block variables changes, a new block is created.The user can add any field to the block variable. only restriction on the block variable is that it must always contain the source and target entity ids.

This approach is applied in situations where the source and target entity may have multiple relationships simultaneously. An illustrative example is the vulnerability on a host; it is conceivable for a vulnerability to exist across several hosts concurrently and the host can also have multiple vulnerabilities.

Config Format

```
{
    "variablesBasedRelationBuilderStrategySpec": {
        "blockVariables": [
            "source_p_id",
            "target_p_id",
            "field 1",
            "field 2"
        ]
    }
}
```

Entity Based

This method is used when an entity can only maintain one relationship at a time. This could be the source or the destination entity. The user defines the base entity (the entity that can only have one relation at a time) in the configuration, and the entity inventory groups the records with the base entity together and creates a relationship block with the moving entity based on the order in which they appear in the data source (using event timestamp).

Config Format

```
{
    "entityBasedRelationBuilderStrategySpec": {
        "baseEntity": "targetEntity/sourceEntity"
    }
}
```

2.6.3. Lookup Enrichment / Software Normalisation

Lookup enrichment is a helper function that retrieves information from a reference dataset using a key attribute from the inventory, enhancing relationships with additional contextual data. In Relationship Extraction, lookup-enriched fields are added at the beginning of the extraction process, joining the source data with the lookup table based on a defined condition. If the lookup table contains an updated_at_ts field, it is also included in the condition. Lookup sources contributing to a relationship are tracked using the origin_lookup_enrich field.

For example, Vulnerability Findings relationship extraction works based on host, vulnerability, and software information. However, software information is often missing from the data source. To handle this, a software normalization job calculates the missing software details and enriches them back into the source table through lookup enrichment.

The following details explain the attributes used for data enrichment:

- lookupInfo Inside the enrichment configuration, lookupInfo provides critical information on the data to be extracted from the lookup table.
    - tableName – The name of the lookup table to retrieve data from.
    - name – An identifier for the lookup enrichment which is added to the source dataset as origin_lookup_enrich.
    - filter – An optional condition to filter rows from the lookup table before joining.
    - preTransform – An optional list of column transformations applied to the lookup table before the join. Each entry contains a `colName` (the column name) and `colExpr` (the expression to compute it).
    - enrichmentColumns – The list of columns from the lookup table to be added to the source dataset after the join.
- sourcePreTransform – An optional list of transformations applied to the source dataset before the join. Like `preTransform`, each entry contains a `colName` and `colExpr`. This is useful when the source data needs to be modified to match the lookup table's join key.

- joinCondition – The expression that defines how records from the source dataset and the lookup table are matched. Records are combined based on this condition. When writing the join expression, lookup table fields are referenced using the **e.** prefix and source dataset fields are referenced using the **s.** prefix. For example, `s.uuid= e.uuid`.
- joinType – Type of join(LEFT/INNER)

Config Format

```
{
    "enrichments": [
        {
            "lookupInfo": {
                "tableName": "name of the input table",
                "name": "Name of the lookup",
                "filter": "filter to apply on the lookup table",
                "preTransform": [
                    {
                        "colName": "name of the column",
                        "colExpr": "expression"
                    }
                ],
                "enrichmentColumns": [
                    "list of final columns to take from lookup table"
                ]
            },
            "sourcePreTransform": [
                {
                    "colName": "name of the column",
                    "colExpr": "expression"
                }
            ],
            "joinType": "LEFT|INNER",
            "joinCondition": "Join expression"
        }
    ]
}
```

2.6.4. Job Execution Flow

- Reads previous mini SDM for the relation, filtered using the updated date.
- Load all SRDM and non-SRDM tables specified in the configuration. Apply filters to retrieve only the latest data using theconfig keys dataIntervalTimestampKey,dataEventTimestampKey .
- Added temporary property fields, and for non-SRDM sources, also adds the `uuid` field.
- If the `sourceMappingInfo` or `targetMappingInfo` contains a `tableName` instead of a `configPath`, join the SRDM with the specified table (which has `p_id` and `class`, treated as the final resolved `p_id` source). Enrich SRDM with `source_p_id`, `source_entity_class` (or their target counterparts).
- Adds enrichment data from lookup sources if they are defined in the configuration.
- Converts all the empty string field from SDM to null.
- Load the current intra and inter entity resolver tables. Since inter resolvers are entity-based, union all fragment inter resolver tables.

- For each SDM,if it has Config Path then compute entity IDs for both source and target. If a single SDM defines multiple source/target configurations, compute separate entity IDs for each.
- Merge all the processed SDMs for the current relation into a single unified DataFrame.
- Determine the resolved source entity IDs by joining with both intra and inter resolver tables.
- Similarly, determine the resolved target entity IDs by joining with the resolvers.
- Builds the relationship block and the mini SDM according to the strategy defined. See Relation Block Building
- Finds the optional attributes defined in the configuration
- Prepares the final relationship table

2.6.5. Config Details

Config Format

```
{
    "name": "Relationship name",
    "inverseRelationshipName": "Inverse relationship name",
    "intraSourcePath": "intra resolver table name",
    "interSourcePath": "inter resolver table name of source entity",
    "interTargetPath": "inter resolver table name of target entity",
    "inputSourceInfo": [
        {
            "sdmPath": "sdm table name",
            "origin": "Origin name for the source",
            "dataIntervalTimestampKey": "timestamp column used for scanning data ranges",
            "dataEventTimestampKey": "timestamp column used for filtering scanned data",
            "uniqueRecordIdentifierKey": "column used for uniquely identifing for each
record",
            "sourceMappingInfo": {
                "tableName": "Table containing the p_id column for joining",
                "joinCondition": "Condition to join the table",
                "configPath": [
                    "Locations of source loader configurations"
                ]
            },
            "targetMappingInfo": {
                "tableName": "Table containing the p_id column for joining",
                "joinCondition": "Condition to join the table",
                "configPath": [
                    "Locations of target loader configurations"
                ]
            },
            "temporaryProperties": [
                {
                    "colName": "Column Name",
                    "colExpr": "Column Expression"
                }
            ],
            "enrichments": [
                {
                    "lookupInfo": {
                        "tableName": "lookup table name",
                        "enrichmentColumns": [
                            "list of fields"
```

```
                        ]
                    },
                    "joinCondition": "condition to join"
                }
            ]
        }
    ],
    "output": {
        "outputTable": "output table name",
        "prevMiniSDM": "previous mini sdm table name"
    },
    "optionalAttributes": [
        {
            "name": "field name",
            "exp": "expression to derive the field",
            "occurrence": "FIRST/LAST/COLLECT"
        }
    ],
    "variablesBasedRelationBuilderStrategySpec": {},
    "entityBasedRelationBuilderStrategySpec": {}
}
```

Config Element Details

Attribute		Description
name		The name of the relationship, Only alpha numerals and underscore(_) are allowed.
inverseRelationshipName		Inverse name of the relationship, Only alpha numerals, and underscore(_) are allowed.
intraSourcePath		Intra source resolver table name.
interTargetPath		Inter target resolver table name.
interSourcePath		Inter source resolver table name.
inputSourceInfo	List of data sources and its entity loader config, The EI resolves the entity id from the given config to the intra and inter resolver tables and builds the relationship blocks	
	sdmPath	SRDM/Non SRDM data source table name
	origin	Name of the origin, if the relation is coming from multiple sources after resolution, then EI saves this origin as array of string.
	dataIntervalTimestampKey	timestamp column used for scanning data ranges.

	dataEventTimestampKey	timestamp column used for filtering scanned data	
	uniqueRecordIdentifierKey	column used for uniquely identifing for each record	
	`sourceMappingInfo`	configPath	Specifies the array of path to loader configuration files
		tableName	Identifies the table containing the p_id column that will be used in joins
		joinCondition	Defines the specific condition for table joining
	`targetMappingInfo`	configPath	Specifies the array of path to loader configuration files
		tableName	Identifies the table containing the p_id column that will be used in joins
		joinCondition	Defines the specific condition for table joining
	`temporaryProperties`	Fields that can be temporarily added to the source dataset before relationship extraction begins. Each entry specifies the field to be added in **colName** and its corresponding expression in **colExpr**.	
	`enrichments`	This attribute holds lookup information for software normalization.	
`optionalAttributes`	Contains the list of fields and their corresponding logic to be applied to SDM/datasource relationship block records.		

	`name`	Name of the field
	`exp`	Spark expression which implements the extraction logic
	`occurrence`	Instructs the entity inventory to determine which occurrence of the field within the block should be retrieved for the relationship output. *FIRST* – First occurrence of the field in the block (using event timestamp) *LAST* – The last occurrence of the field in the block (using eventimestamp) *COLLECT* – Collects all the observed values of the field into an array.
`variablesBasedRelationBuilderStrategySpec`	See *3.5.1.1 Variable Based Strategy*	
`entityBasedRelationBuilderStrategySpec`	See 3.5.1.2 Entity Based Strategy	
`output`	Contain information about output table and prev mini sdm	

2.7. Relationship Resolution

Relationship resolution refers to the process of identifying and consolidating references to the same relationship from various sources. When working across several data sources, it is common to encounter variances, inconsistencies, or duplications in the representation of relationships. The objective of relationship resolution is to rectify these inconsistencies and provide an accurate, unified representation for each unique relation. Various attributes or characteristics associated with the relationship are assessed and correlated in the procedure.
The relationship resolution is divided into three major modules they are.

- Disambiguation Group Identification
- Attributes Resolution
- Output Generation enrichment

2.7.1. Disambiguation Group Identification

It is the process of identifying the relationships which falls into the same group, there will be multiple ways we can group relations together depending on the type of relation, this enables us to include the logic of each individual relation into the grouping. Following are the supported group identification available.

- ***Vulnerability Finding Upgrade*** : Here the disambiguation grouping will be determined by utilizing the group variables along with the statusField.
- ***Variable Based Grouping :*** Here the disambiguation grouping will be identified using the group variables. The relations which has the same block variables will fall into the same group.
- ***Union :*** In scenarios where there isn't a defined logic for grouping relationships, this approach can be applied, wherein relations from various sources are consolidated through a union operation.

2.7.2. Attributes Resolution

Once the relations are grouped together then the next step is to extract each attributes from the non-resolved relations to build a unified representation of the relation. Following are the supported strategies available in the relation disambiguation to resolve relation attributes

- Source Based Confidence Matrix
- Value Based Confidence
- Rolling Up Fields
- Aggregation Based
- Field level Confidence

2.7.3. Output Generation

The job needs to provide the following output from the resolution:

- *Final Resolved relation table:* Individual output for each relation
- *Relation Resolver Table:* Resolver table which contains the disambiguated and non disambiguated relationship ids, this table is common for all the relation resolution
- *Non-Resolution Table:* Stores the non resolved relations, this table is common for all the relations.

2.7.4. Fragment and Resolver Table

An relationship can exist across multiple data sources, and an relationship resolution process consolidates these fragments into a single resolved relationship. In a knowledge graph, it is important to capture the association between the fragment relationship and the resolved relationship. This ensures traceability and data integrity within the graph.

Each relationship will have its own individual fragment and resolver tables. Fragment tables, named with the prefix "Fragment" (e.g., Fragment Vulnerability Finding on Host), will include graph-driven properties such as like graph.vertex.name and graph.cache.enabled, with graph IDs based on the fragment unique IDs, and updated_at_ts. Resolver tables define the relationships between resolved and nonresolved relationship, with properties such as graph.edge.name, graph.edge.source.name, graph.edge.target.name, and graph.cache.enabled. Both fragment and resolver tables will reside in a dedicated Iceberg schema, KG_FRAGMENT_SCHEMA, separate from the main Knowledge Graph schema. Table names and OLAP configurations will be configurable but it is also autogenerated from config merger.

2.7.5. Job Execution Flow

- Reads all the relationship tables defined in the config and filters the latest data using parsed interval timestamp.
- Checks if the input tables are empty. In such cases, it generates an empty output table that includes all required fields.
- Converts all the empty string field from relationship tables to null.
- Add missing fields
- Unions the tables defined for the relation resolution to a single data frame and which is subsequently saved to non-resolved relationship table.
- Creates a confidence matrix : This modules gives confidence for the sources and fields based on the disambiguation strategy defined in the config.
- Builds the relationship block accoring to the group strategy defined. See *Disambiguation group Identification.*
- Prepares the final relationship table

2.7.6. Config Details

Config Format

```
{
  "relationshipModels": [
          {
            "tableName": "table name",
```

```
            "name": "name"
          }
  ],
  "disambiguation": {
    "disambiguationGrouping": {
            "type": "VulnerabilityFinding",
            "blockVariables": [
              variables
            ],
            "statusField": "name of the status field"
    },
    "strategy": {
      "rollingUpFields": [
              roll up fields
      ],
      "fieldLevelConfidenceMatrix": [
              {
                "field": "field name",
                "confidenceMatrix": [
                  source names according to the priority
                ]
              }
      ],
       "valueConfidence": [
              {
                "field": "field name",
                "confidenceMatrix": ["Value 1","Value 2"]
              }
      ],
      "aggregation": [
              {
                "field": "field name",
                "function": "aggregation function"
              }
      ]
    }
  },
  "output": {
    "disambiguatedModelLocation": "resolved table location",
    "resolverLocation": "resolver location",
    "nonResolvedModelLocation": "non resolved table location"
  }
}
```

Config Element Details

Attribute		Usage	Is Mandatory
relationshipModels	Each of the inputs has the name of the non-resolved table name and an alias name used throughout the disambiguation job.		Yes
	tableName	Non resolved table name with schema.	Yes
	name	Alias name to use in the confidence matrix	Yes
	filter	An optional filter to apply on the non-resolved table	No
disambiguation	Holds the core configurations for controlling the disambiguation job		Yes
	disambiguationGrouping	Includes list of blockvariables and attributes used in generating group.	Yes

		type	Name of the grouping strategy. See *Disambiguation Group Identification*	Yes
		blockVariables	Includes fields used in resolving relationships.	No
		statusField	It is used in vulnerabilityFinding group building strategy.	No
	confidenceMatrix	A list specifying the order in which the values for each attribute should be taken from input sources.		No
	excludeValues	A list specifying string values with the least priority (but higher priority than null value) while resolving value of an attribute.		No
	strategy	Specifies the strategies to be used for disambiguation. Each of the strategies have different sub-configs.		No
output	As the name suggests, has the table names of output and resolver table.			Yes
	disambiguatedModelLocation	Resolved entity table name with schema		Yes
	resolverLocation		Resolver table name with the schema	Yes
	nonResolvedModelLocation	An optional filter to apply on the final inventory		yes

2.8. Post Analytics Write Back

When an entity/relationship is populated, there will be scenarios where we need to enrich an entity/relationship based on analytics. These enrichments can be derived either by considering attributes from the same entity/relationship, multiple rows of SDM, or attributes from other entities/relationships obtained through relationship traversal or products. The Post analytics write back job helps to bring this enrichment fields from other products back into the Knowledge Graph.

2.8.1. Post Analytics Write Back Job

The new Graph-based OLAP architecture for the Knowledge Graph requires individual tables for entity and relationship tables. The job takes individual entity/relationship tables and their corresponding post-analytics tables from other products and produces the enriched entity table as output.

- The EI team is responsible for preparing entity/relationship enriched tables. These jobs are executed by a parallel pipeline executed at post-analytics write back layer.
  - Prechecking
    Scanning the given post analytics schemas to identify the enrichment tables.
    Verifying the post-analytics table schema to ensure there are no duplicated fields and confirming that the post-analytics table does not contain duplicate p_ids.
  - Join tables
    After taking the information on the entity/relationship tables and the post analytics table from the configuration. The entity table will left join with the post analytics table which matches the regex corresponding to the post analytics schema.
  - Publishing enriched tables
    Finally, any given transformation in the configuration will be populating to the enriched table.

2.8.2. Post Analytics Write Back Configurations

The Post Analytics Write Back configuration includes schemas tailored for the write back process. Input tables are identified from the Knowledge Graph schema, considering table from the config tableInfo.tableName as input table. Post Analytics tables are sourced from the post schema, considering tables matching postTableIdentifierRegex as post analytics tables. Subsequently, a left join is executed on the entity/relationship table and post analytics tables using the unique column ("uniqCol") specified in the configuration.

Publisher Entity Config

```
{
  "transformSpec":{
    "type": "AttributeWriteBack",
    "postSchemas": "Post analytical enrichment table schemas",
    "tableInfo": {
      "tableName": "ei.sds_ei__host__enrich",
      "tableType": "entity",
      "commonColumns": ["p_id", "updated_at", "updated_at_ts"],
      "uniqCol": "p_id"
    }
  },
  "outputTableInfo": {
    "partitionColumns": ["class"],
    "outputTableName": "ei_publish.sds_ei__host__publish"
  },
  "derivedProperties": [
    {
      "colName": "activity_status",
      "colExpr": "CASE WHEN class='Vulnerability' and activity_status IS NOT NULL THEN CASE
WHEN
greatest(associated_hosts_with_open_findings_count,associated_cloud_compute_and_container_w
ith_open_findings_count,0) >= 1 THEN 'Active' ELSE 'Inactive' END ELSE activity_status END"
    },
  "isOLAPTable": true //default= true
}
```

Publisher Relationship Config

```
{
  "transformSpec":{
    "type": "AttributeWriteBack",
    "postSchemas": "ei,ccm,temp",
    "tableInfo": {
      "tableName": "ei.sds_ei__rel__vulnerability_finding_on_host",
      "tableType": "relation",
      "commonColumns": ["relationship_id", "updated_at_partition","updated_at_ts"],
      "uniqCol": "relationship_id"
    }
  },
  "outputTableInfo": {
    "partitionColumns": ["relationship_name"],
    "outputTableName": "ei_publish.sds_ei__rel__vulnerability_finding_on_host__publish"
  },
  "derivedProperties": [
```

```
    {
      "colName": "activity_status",
      "colExpr": "CASE WHEN class='Vulnerability' and activity_status IS NOT NULL THEN CASE
WHENgreatest(associated_hosts_with_open_findings_count,associated_cloud_compute_and_contain
er_with_open_findings_count,0) >= 1 THEN 'Active' ELSE 'Inactive' END ELSE activity_status
END"
    },
  "isOLAPTable": true //default= true
}
```

Config Element Details

Attribute		Description	Is Mandatory
transformSpec.type	Type of the post analytics write back, job for post analytics write back it should be always *AttributeWriteBack*		Yes
transformSpec.postSchemas	Comma separated schema names to look for the post analytics tables.		Yes
transformSpec.tableInfo	Information regarding the Entity tables		Yes
	tableName	The entity/relationship table which should be enriched with the post analytics table.	Yes
	tableType	The type of table whether its entity or relationship.	Yes
	commonColumns	List of common columns between the base and post analytics enrichment tables	Yes
	uniqCol	Unique column to identity a single relationship or entity, (relationship id and entity id)	
outputTableInfo.partitionColumns	List of partition columns to partition the final enriched table.		Yes
outputTableInfo.outputTableName	Name of the output table, which is the enriched entity/relationship table.		Yes
DerivedProperties.colName	Column name of the derived column.		
DerivedProperties.colExpr	The Expression to derive the derived column from the entity/relationship table.		
isOLAPTable	To confirm whether it should be taken to the Graph-based OLAP architecture. Default True.		

2.9. Entity Relationship Enrichment

The Knowledge Graph consists of entities and relationships. While entities contain information about their attributes, the entity relationship count enrichment job adds an attribute to each entity to indicate the number of relationships it has. This was previously implemented using unioned entity and relationship tables. To support the puppy graph, the job now consumes data from resolved entities and relationship tables as input.

2.9.1. Entity Relationship Enrichment Job

Both relationship count enrichment and multihop enrichment are performed within the same job. This is a config-driven process where users can specify relationship count enrichment parameters in the "countEnriches" section and multihop enrichment parameters in the "inheritedPropertyEnrichment" section.

Job Execution flow of Enrichment Job

- Reads entity data.
- Count Enrichment
- Groups enrichment configurations by relationship table.
- For each relationship table, reads and validates relationship data against filter expressions, then determines if entity is source or target based on entity class before calculating the aggregated counts using specified filter conditions.
- Entity data is enriched by joining with calculated relationship counts using a LEFT JOIN to preserve all entities.
- Just before executing multi-hop relationship logic, the configuration is optimized to reduce unnecessary joins and improve performance.
- Recursive multi-hop enrichment is performed by joining with entity or relationship tables, applying aggregations, and resolving nested property dependencies through inherited configurations.
- Derived property fields are computed.
- Fields containing null values are removed.
- The job supports writing back only the enriched fields to the entity table using a configurable flag `writeOnlyEnrichedFields`.

2.9.2. Entity Relationship Enrichment Configurations

Entity Relationship Enrich Config

```
{
  "entityTableName": "entity table name",
  "outputTableName": "enriched output table name”,
  "countEnriches": [
    {
      "colName": "relationship count to enrich",
      "relationshipTableName": "relationship table name"
    },
    {
      "colName": "relationship count to enrich",
      "relationshipTableName": "relationship table name",
      "filter": "filter condition" //default is true
    }
  ],
  " inheritedPropertyEnrichment": [
    {
     "tableName": "",
```

```
      "alias": "",
      "colEnrichments": [
              {
               "colName": "",
               "aggExpr": "",
               "aggFunction": "",
              }
     ]

     }

 ]
"derivedProperties"[
{
  "colName": "column name",
  "colExpr": "expression"
}
]
}
```

Config Element Details

Attribute		Description
entityTableName		Name of the entity table to be enriched .
outputTableName		Output Table Name
countEnriches	Array of count enrichment configurations	
	colName	Name of the new column that will contain the count
	relationshipTableName	Relationship Table Name
	filter	Optional condition to filter relationships before counting. Default value is true
derivedProperties	This section has configurations of existing or new fields which are recalculated/derived using new expressions after performing disambiguation.	
	colName	Name of the field
	colExpr	Expression to derive the field

2.10. Graph Olap Layer

The current publisher layer stores the entire historical data in the Iceberg data lake, and Puppy Graph indexes all the data from the publisher layer. However, the dashboard does not require the full historical dataset for display. For trend analysis,

historical snapshots should be maintained at different granularities based on the data's age. The graph olap layer incorporate a time-based data retention strategy that keeps daily snapshots for the first 30 days (based on updated_at_ts), transitions to weekly snapshots for days 31-180 (with configurable weekday), and maintains only monthly snapshots beyond 180 days. Make this implementation config-driven, allowing for customizable time periods and snapshot days to accommodate varying retention requirements.

2.10.1. Graph Olap Layer Job

The entire Graph OLAP layer pipeline begins with a task of determining both the minimum and maximum updated_at timestamps for each entity, while also calculating the absolute minimum updated_at timestamp across all entities and the maximum updated_at timestamp where data exists for all entities. These time boundary calculations are then posted to an API endpoint that UI teams consume to accurately populate dashboards. The process follows these steps:

- List all available publisher configurations
- Read each publisher config to identify schema and table names where the isOLAP key is set to true
- Use the pyiceberg library to read metadata from each identified table, storing the minimum and maximum updated_at timestamps per entity
- Calculate the global minimum updated_at timestamp and the minimum of all maximum updated_at timestamps from the previous step's results
- Post the calculated timestamp data to the API endpoint: /sds_mgmnt/api/v1/config/entity-timestamps/

The next task involves a Spark job that creates replica tables for each publish table. These replicas reference the original data files by leveraging Iceberg metadata properties, based on the snapshot specified by the user. This selective storage approach achieves an optimal balance between maintaining robust trend analysis capabilities and ensuring storage efficiency. The Spark job task processes an OLAP configuration that defines time-based filters required for each publish table. It creates these time filters by reading the snapshot specification section of the config, which details the daily, weekly, and monthly range of days. The job converts these day ranges to ISO instant format (such as '2011-12-03T10:15:30Z') for UTC-based filtering, then applies these filters to the updated_at_ts field. In addition to time-based filtering, the job also applies custom filters specific to publisher tables when provided in the configuration. For example, if entire dashboard requires only open vulnerabilities for the vulnerability entity, this filter can be applied - with the important note that such custom filters can only be applied to partitioned fields, meaning the status field would need to be partitioned in the vulnerability publisher table.

Another task in the pipeline is the Spark job that creates replicas for fragment and resolver tables. For these tables, maintains only the latest data based on the updated_at_ts field, rather than keeping historical snapshots.

2.10.2. Graph Olap Layer Configurations

Graph Olap Config

```
{
 "analysisPeriodSpec": {
    "weekEnd": "FRIDAY",//Default to FRIDAY
    "monthEnd": "CALENDAR_END" // Default to CALENDAR_END (only supports CALENDAR END)
 },
 "snapShotSpec": [
    {
       "dayRange": "1-30",
       "granularLevel": [
          "DAILY"
       ]
    },
    {
       "dayRange": "31-90",
```

```
        "granularLevel": [
          "WEEKEND",
          "MONTH_END"
        ]
      },
       {
        "dayRange": "91-365",
        "granularLevel": [
          "MONTH_END"
        ]
      }
  ],

  "filterSpec": {
            "filterExpr": [
                    "status='open'"
                      ]
  }
}
```

Graph Olap Config Element Details

Attribute		Description	Is Mandatory
analysisPeriodSpec. weekEnd	Defines which day represents the end of the business week.		No
analysisPeriodSpec. monthEnd	Specifies how to determine the end of a business month.		No
snapShotSpec	Field defines the time-based data retention strategy through an array of configuration. objects.		Yes
	dayRange	Defines the date range for granularity retention.	Yes
	granularLevel	Defines the snapshot frequency or resolution of data retention within each specified time range.	Yes
filterSpec.filterExpr	Custom filter expressions applied to partitioned fields.		Yes

2.10.3. Config Merger for Graph Olap Configs

The graph OLAP configurations are integrated within the config-merger using the config_item_type "olap_configs". Merger provides complete overriding capabilities for OLAP configurations, maintaining all basic-level override functionality. These configurations are handled similarly to entity extraction configs, with a global OLAP config serving as the default. If user don't specify entity/relationship olap configurations, the spark job automatically applies the global OLAP config to each publisher. When overriding OLAP configurations, remember these important points:

- The global OLAP configs give you the ability to override entire configurations.
- At the entity/relationship level, overriding is restricted to the analysisPeriodSpec section.
- The analysisPeriodSpec is always applied according to the global OLAP config.

2.11. Change Detection System(CDS)

The Change Detection System (CDS) in the Knowledge Graph identifies changes that occur between job runs and dynamically determines whether a Spark job needs to be executed again. During a rerun, CDS analyzes different components of the pipeline and detects whether any meaningful change has occurred. Based on this detection, the system decides whether to skip execution or rerun the Spark job.
The types of changes typically detected include:

- Configuration Changes – Modifications in job configuration files.
- Analytical Changes – Changes in the Spark job JAR.
- Input Data Changes – Updates or modifications in input files.

2.11.1. Execution Flow

- The KG pipeline is triggered through an Airflow DAG.
- Airflow retrieves the variable KG_CDS_ENABLED and KG_PREEXEC_CONFIG.
- If the above airflow variables is disabled or not configured, the pipeline proceeds with the normal Spark job execution.
- If airflow variables configured, the system prepares to validate whether any change has occurred.
- Airflow constructs a payload containing:
  - Configuration path (configPath)
  - End epoch of analytical period (parsedIntervalEndEpoch)
  - Current updated timestamp (currentUpdateDate)
  - Spark job JAR location (JarLocation)
- Airflow sends the payload to the Validator API endpoint before executing the Spark job.
- The Validator API determines the relevant input files based on timestamp filters.
- File paths are extracted from Iceberg metadata
- The Validator API generates hashes for Input files, Configuration files, Spark job JAR.
- Reads the last run output table to retrieve previously stored hash values.
- The system compares:
  - Current file hash vs previous file hash
  - Current config hash vs previous config hash
  - Current JAR hash vs previous JAR hash
- Based on the comparison, the validator determines whether:
  - Input data changed
  - Configuration changed
  - Analytical logic (JAR) changed
- The validator returns a response indicating whether any change was detected along with detailed flags.
- If hasChanges is false, Skip the Spark job execution.
- If hasChanges is true, Execute the Spark job.
- If the job runs, the new hash values are stored for use in the next execution cycle.

2.11.2. Configuring Airflow Variables for CDS

To enable the Change Detection System (CDS) in the Knowledge Graph pipeline, specific Airflow variables must be configured.

- Set the Airflow variable KG_CDS_ENABLED to true.
- Set the Airflow variable KG_PREEXEC_CONFIG with the following configuration:

```
                    {
  "cds_task_check": [
    "inventory_models"
  ]
}
```

cds_task_check specifies the list of pipeline modules for which change detection validation should be executed during the Airflow pre-execution phase. At present, change detection is enabled only for the inventory_models module.

2.12. Entity Explorer

2.12.1. Data Dictionary Update Module

Introduced a data dictionary update module that is responsible for validating and updating the data dictionaries for both entity and relationship data dictionaries. It ensures that data dictionary definitions are synchronized with actual data structure in the data lake, verifying attribute availability and maintaining data type consistency. The main purpose of data dictionary update modules is

- Validating that attributes defined in data dictionaries exist in the actual data tables
- Updating data type information for attributes based on their iceberg table type

Following Attributes are added or updated as part of the data dictionary update process

Attribute	Usage	Dictionary Type
is_available_in_datalake	• Added to each attribute in the dictionary to track its availability • Set to True if the attribute is found in the publish table or fragment table • Set to False if the attribute doesn't exist in the tables	• Update in both the relationship/entity data dictionary attributes • Update in the entity attributes section inside the relationship data dictionary
type	• The data type of the attribute as found in the data lake • Mapped from complex data types to simplified types • When the code detects a timestamp field, it always sets the type to "timestamp" for consistent handling of date/time in UI	• Update in both the relationship/entity data dictionary attributes • Update in the entity attributes section inside the relationship data dictionary
nonDisambiguatedType	• The data type of the attribute in the fragment table (before disambiguation) • Only added when both publish and fragment tables exist and the attribute is not marked for enrichment	• Update in both the relationship/entity data dictionary attributes
graph_reference_name	• The reference name is added to the dictionary for use in PG indexing for table reference • Generated from either the caption field (for entities or non-inverse relationships)	• Update in both the config value of relationship/entity data dictionary

	• Or from the inverse_relationship_name (for inverse relationships) • Formatted as lowercase with spaces replaced by underscores	
is_available_in_datalake	• To indicate if the entity/relationship exists in the data lake	• Update in both the config value of relationship/entity data dictionary

Data Type Mapping

The system maps complex data types to simplified types for the data dictionary:

```
data type mapping
```

```
"stringtype": string_type,
"integertype" : "int",
"string": string_type,
"struct": string_type,
"array<struct>": string_type,
"list<struct>": string_type,
"int": "int",
"long":"int",
"double":"float",
"decimal": "float",
"float":"float",
"list<double>":list_float,
"list<float>":list_float,
"list<decimal>":list_float,
"list<int>": list_int,
"list<long>": list_int,
"list<string>": list_string,
"list<array>": string_type,
"array<list>":string_type,
"array<double>": list_float,
"array<float>": list_float,
"array<decimal>": list_float,
"array<int>": list_int,
"array<long>": list_int,
"array<string>": list_string,
"array<array>": string_type,
"list<list>":string_type,
"timestamp": "timestamp",
"timestamptz": "timestamp",
"datetime.date":"timestamp",
"boolean": "boolean"
```

Execution Flow

- The API receives a request to update either all or a specific data dictionary
- The system retrieves the data dictionary configuration(s) to update

- For each configuration:
    - Determine if it's an entity or relationship data dictionary
    - Locate associated publisher and intersource configurations
    - Check if the corresponding tables exist in the data lake
    - Retrieve schema information for existing tables

- For each attribute:
    - Verify its existence in the data lake tables
    - Determine its actual data type
    - Update its availability status
- Add additional metadata like graph reference names
- Post the updated configuration back to the API

2.12.2. Data Dictionary Data Dictionary

The data dictionary contains information about various entities or fields that can be consumed by different clients or users ofa system. It is a JSON schema of the data model. Data Dictionary contains all attributes supported by EI, label to be displayedin UI, whether it should be shown or not in UI, data type, description etc

Expected Format

Entity Data Dictionary

```
{

        "caption": "caption",
        "description": "description",
        "attributes": {
            "field name in druid": {
                "caption": "caption",
                "description": "description on hovering",
                "group": "group",
                "enable_hiding": "True",
                "type": "data type",
                "width": "integer value",
                "step_interval": "integer/float value",
                "range_selection": "None",
                "ui_visibility": "True",
                "solutions": ['EI'],
                "is_active": "True",
                "candidate_key": "False",
                "data_structure": "list"
            }
        }

}
```

Relationship Data Dictionary

```
 {
            "caption": "relationship caption",
            "isInverse": "False",
```

```
            "sorting_columns": [
                {
                    "field": "field name",
                    "type": "data type",
                    "desc": true/false
                }
            ],
            "relationshipCheck": [
                {
                    "field": "field name",
                    "value": [
                        "values"
                    ]
                }
            ],
             "customFilter": [
               {
                 "filterType": "field name,
                 "filterData": [
                   "value"
                 ],
                 "isExtra": true/false,
                 "isDate": true/false
               }
             ],
            "relationship_attributes": {
                "field name in druid": {
                    "caption": "caption",
                    "description": "description",
                    "type": "data type",
                    "is_enrichment": true/false
              }
            },
            "entity_attributes": {
                "field name in druid": {
                    "caption": "caption",
                    "description": "description",
                    "type": "data type"
                }
            }


}
```

Graph Data Dictionary

The Graph Data Dictionary serves as a tool to visually represent the relationship between various attributes within a specific entity. It provides a dendrogram-like view in the user interface (UI), offering clients a quick overview of the assets they own. Currently in EI this has been implemented for Cloud Compute entity only.

Graph Data Dictionary

```
{
  "eiGraph": {
    "entity": {
              "configs": {
              "attributeList": [
                {
                  "id": "General name for the attribute to be shown in UI",
                  "accessorKey": "attribute1",
                  "iconName": "icon name to be shown in UI",
```

```
                "name": "attribute caption as given in data dictionary",
                "children": [
                  {
                    "id": "General name for the child attribute",
                    "accessorKey": "attribute2",
                    "iconName": "child_icon_name",
                    "name": "attribute caption for child as in dictionary",
                    "tooltip": [
                        {
                           "accessorKey": "attribute3”,
                           "id": "name of attribute“
                        }
                     ]
                  }
                ]
              }
            ]
        }}}
```

Attribute Description

Attribute	Usage	Default	Is Mandatory
caption	Caption to be displayed in UI	Underscore removed capitalized field from druid	No
description	Description on hovering		Yes
group	Define whether a particular field is common, entity_specific, source_specific, enrichment		Yes
enable_hiding	Boolean value implies whether a field should be visible in landing page of ui	True	No
type	Implies data type of a particular field with possible values string, integer, timestamp, double		Yes
width	An integer value corresponds to length of caption defined	provided from sds_ei_ui_config.json	No
step_interval	An integer value corresponds to division of values on range selection	provided from sds_ei_ui_config.json	No
range_selection	Boolean value implies whether to add range filter or not	None	No
ui_visibility	Boolean value implies whether a field should be visible to ui	True	No
dashboard_identifier	A nested dictionary where each top-level key (e.g., "EI", "Host", "CCM") represents a solution with specific configuration attributes as key-value pairs. Generated by update_attribute.py, it supports solution-specific overrides within a consistent configuration framework.	{}	No

is_active	This attribute is handled by solution teams (VRA, CAM), implies whether a particular field from EI is to be displayed to their corresponding UI Dashboards	True	No
candidate_key	Boolean value indicates whether this field is candidate key or not	False	No
data_structure	For complex data types, data_structure is set to list	None	No
sorting_columns	Defines sorting criteria for the relationship fragments dropdown in the detailed view. It comprises a dictionary containing the field name, its data type, and the desired sorting order (ascending/descending).	[]	No
is_enrichment	Fields that are enriched in the final relationship table but not present in nondisambiguated relationship tables should have their is_enrichment set to true.	False	No
customFilter	This attribute controls the filtering of relationship data in the detailed view.	[]	No
relationshipCheck	This attribute ensures that only relationships with fields matching the specified values will appear in the left panel on detailed view page.	[]	No

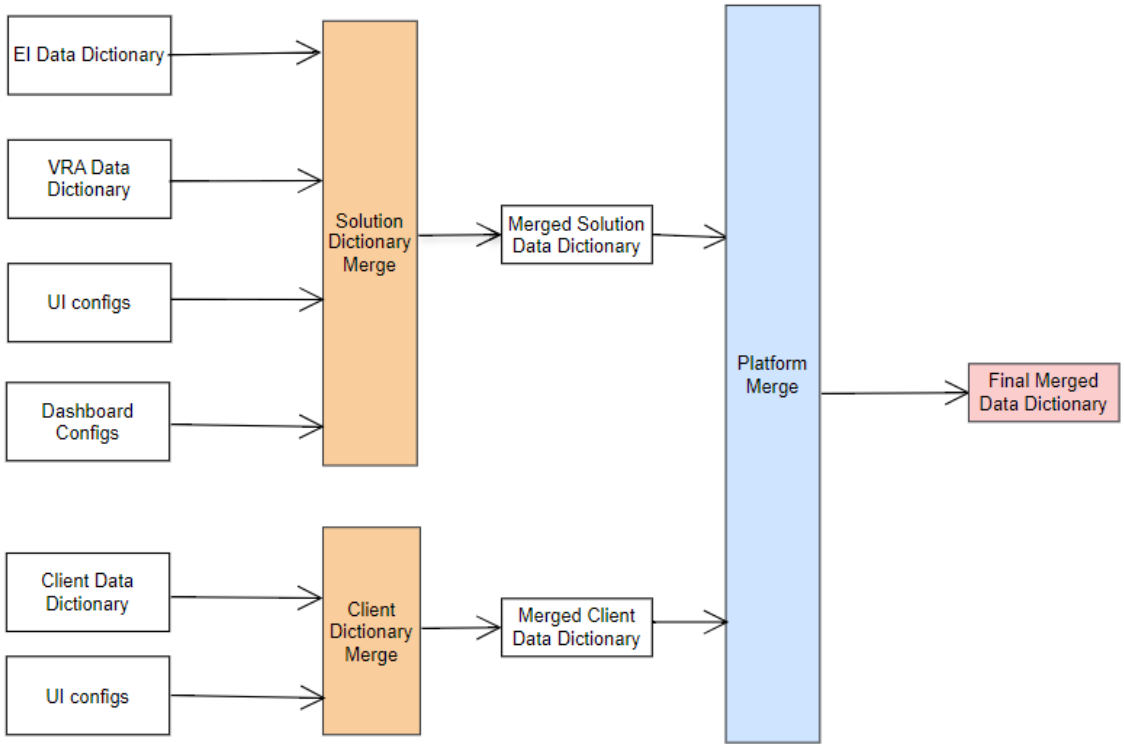
The key at the top level of both data dictionary and relationship data dictionary corresponds to the names of entities. Within the data dictionary, the values associated with these entities include properties such as caption, description, and attributes. The attributes consist of fields retrieved from Druid, where each field is identified by an "accessor_key" in the UI configuration, and the corresponding values represent specific properties of those fields.

In the relationship data dictionary, the values are associated with relationship names and include properties such as caption, isinverse, entity_attributes, and relationship_attributes. The entity_attributes and relationship_attributes mirror the attributes in the entity data dictionary, with the distinction that entity_attributes fields are fetched from the entity table in Druid, while relationship_attributes fields are retrieved from the relationship table in Druid.

The graph data dictionary does not include any analytics with Druid data, so graph additions are directly incorporated into the UI configuration without undergoing any transformations.

2.12.3. Config Merger for data dictionary

The config merger API accepts data dictionaries from various solutions, processes them, and returns a single, unified data dictionary. This transformation includes applying necessary overrides and adding required fields to ensure the output meets the desired specifications.



Transformation Flow Overview:

Input: Multiple data dictionaries from different solutions, their corresponding dashboard configs, solution and client UI configs, and EI data dictionary are passed as input to the API.

Processing:

The Configuration Merger API processes the input and prepared the merged solution data dictionary by:

- Identifying overlapping fields.
- Adding the dashboard identifier field.
- Adding overrides from each solution into its respective key value pair based on the type of merge defined in dashboard config.
- Adding fields from UI config as per requirements.

The Configuration Merger API processes the input and prepared the merged client data dictionary by:

- Adding fields from UI config as per requirements.

Finally the Client and Solution data dictionaries are merged into a single data dictionary using platform merge.

Output: A single, merged data dictionary containing the consolidated and modified data from all input dictionaries.

Dashboard Config Format

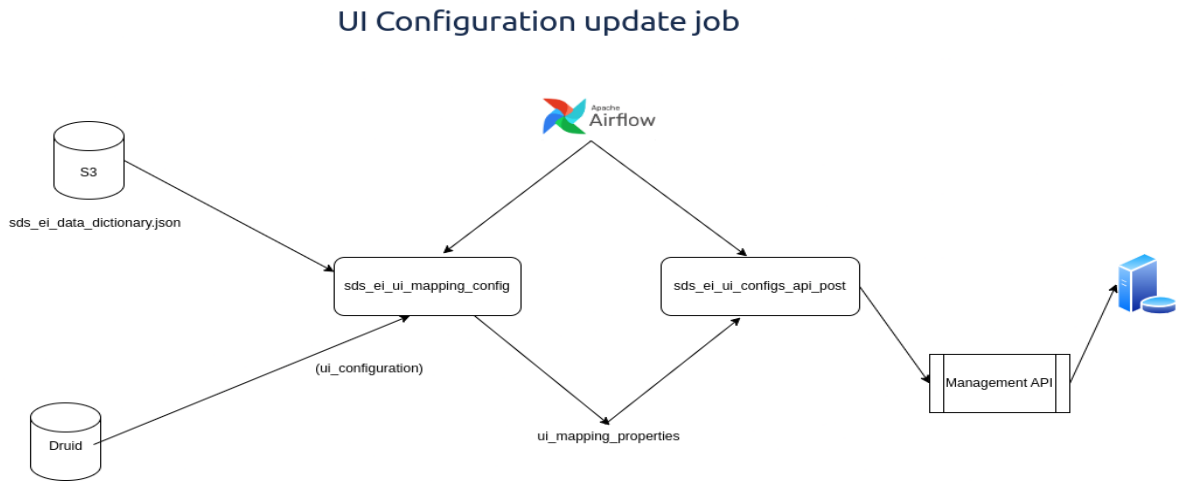
```
{
    "Dashboard_1": {
        "selective_merge": false
```

```
    },
    "Dashoard_2": {
        "selective_merge": true
    }
}
```

Dashboard configs are used to determine the type of merge that is required for each dashboard within a solution.

2.12.4. UI Automation

UI automation plays a pivotal role enabling the UI to directly access and display data from the publish layer. This is achieved using a dictionary mapping provided within the Data Dictionary. UI automation script retrieves the necessary data from the publish layer and generates the UI configuration accordingly combining dictionary information. The generated UI configuration, which includes the mapping of attributes and their display settings, is then posted to a management API endpoint.
In UI automation, UI configuration is posted to management API endpoint at: /sds_mgmnt/api/v1/config/entity_inventory/




```
UI Configuration
```

```
{
    "id": "entity_inventory",
    "name": "Entity Inventory",
    "description": "SDS Entity Inventory UI Configuration",
    "configs": {
        "Common": {
            "properties": [
                {
                    "enableHiding": true,
                    "header": "Entity ID",
                    "description": " Unique ID for the entity ",
                    "accessorKey": "p_id",
```

```
                "type": "string",
                "removable": true,
                "isDate": false,
                "id": "p_id__0",
                "dashboard_identifier": {
                    "dashboard1": {},
                    "dashboard2": {},
                    "dashboard3": {
                        "accessorKey": [
                            {
                                "key": "override_value___p_id"
                            }
                        ]
                    }                    },
                "disableSortBy": true,
                "width": 40,
                "group": "common",
                "nonDisambiguatedType": "varchar"
            }
        },
    "entityConfigs": [
        {
            "category": "Host",
            "config": {
                "entityColumnMaps": [
                    {
                        "category": "Common",
                        "properties": [
                            {
                                "enableHiding": true,
                                "header": "Entity ID",
                                "description": "Unique ID for the entity",
                                "accessorKey": "p_id",
                                "type": "string",
                                "isDate": false,
                                "id": "p_id__0",
                                "dashboard_identifier": {
                                    "dashboard1": {},
                                    "dashboard2": {},
                                    "dashboard3": {
                                        "accessorKey": "override_value__p_id"
                                    }
                                },
                                ],
                                "disableSortBy": true,
                                "width": 40,
                                "group": "common",
                                "column_merge": true
                            }
                        ]
                    }
                ],
                "candidateKeys": [
                    "host_name"
                ],
                "hoverFields": []
            }
```

```
            }
        ],
        "relationshipConfigs": {
            "Host": [
                {
                    "title": "Owned By Person",
                    "relationship": "HOST_OWNED_BY_PERSON",
                   "sorting_columns": [
                      {
                          "field": "field name",
                          "type": "data type",
                          "desc": true/false
                      }
                    ],
                    "relationshipCheck": [
                      {
                          "field": "field name",
                          "value": [
                              "values"
                          ]
                      }
                    ],
                    "customFilter": [
                      {
                        "filterType": "field name,
                        "filterData": [
                          "value"
                        ],
                        "isExtra": true/false,
                        "isDate": true/false
                      }
                    ],
                    "relationshipColumns": [
                        {
                            "header": "Display Label",
                            "isDate": false,
                            "accessorKey": "display_label",
                            "description": "An Identifier",
                            "disableSortBy": true,
                            "isExtra": true,
                            "type": "string",
                            "solutions": [
                                "EI"
                            ],
                            "width": 40,
                            "detailedViewHide": false
                        }
                    ],
                    "queryField": "target_entity_class",
                    "inverse": true
                }
            ]
        }
      "eiGraph": {
        "entity": {
          "configs": {
            "attributeList": [
```

```
                {
                  "id": "General name for the attribute to be shown in UI",
                  "accessorKey": "attribute1",
                  "iconName": "icon name to be shown in UI",
                  "name": "attribute caption as given in data dictionary",
                  "children": [
                    {
                      "id": "General name for the child attribute",
                      "accessorKey": "attribute2",
                      "iconName": "child_icon_name",
                      "name": "attribute caption for child as in dictionary",
                      "tooltip": [
                        {
                          "accessorKey": "attribute3”,
                          "id": "name of attribute“
                        }
                      ]
                    }
                  ]
                }
              ]
            }
          }
        }
    }
}
```

2.12.5. Relevance of each Attribute from Data Dictionary on UI

Following are the field level properties and their relevance from Data Dictionary:

- The "caption" attribute is responsible for what is displayed in the UI. In the case of Druid names, they are separated with underscores and cannot be directly presented in the UI, so it is provided from the data dcitionary . If the "caption" is not specified in the Data Dictionary, the field name from Druid, as provided in the Data Dictionary, will be transformed for UI display. This transformation involves capitalizing the initial letter and removing underscores before being incorporated into the UI configuration.

- The "description" serves as an explanatory element detailing the logic or specifics of a particular field. It becomes visible when hovering over the corresponding fields on dashboards, providing users with additional context or information about the data presented.

- The "group" attribute is utilized for the purpose of structuring and organizing fields based on common, entity specific, source specific, or enrichment. This classification determines where the fields are displayed on a detailed page, with common fields under "General Information," and entity-specific and source-specific fields under "Additional Information." Enrichments are presented in a dedicated "Enrichment" tab. Additionally, the "group" attribute aids in distinguishing between common and entity-specific fields when adding add columns filter on the landing page, streamlining user interaction and navigation based on the nature of the data.

- The "data_type" attribute distinguishes the types of data for a particular field, with possible values being string, timestamp, integer, and double. This classification is pivotal for the calculation of filter types on the UI side. For timestamp data types, the "is_date" attribute is set to true in the UI configuration, while for other types, it is set to false. The "sds_ei_ui_config.json" file contains generic settings such as width and step_interval for specific data types.

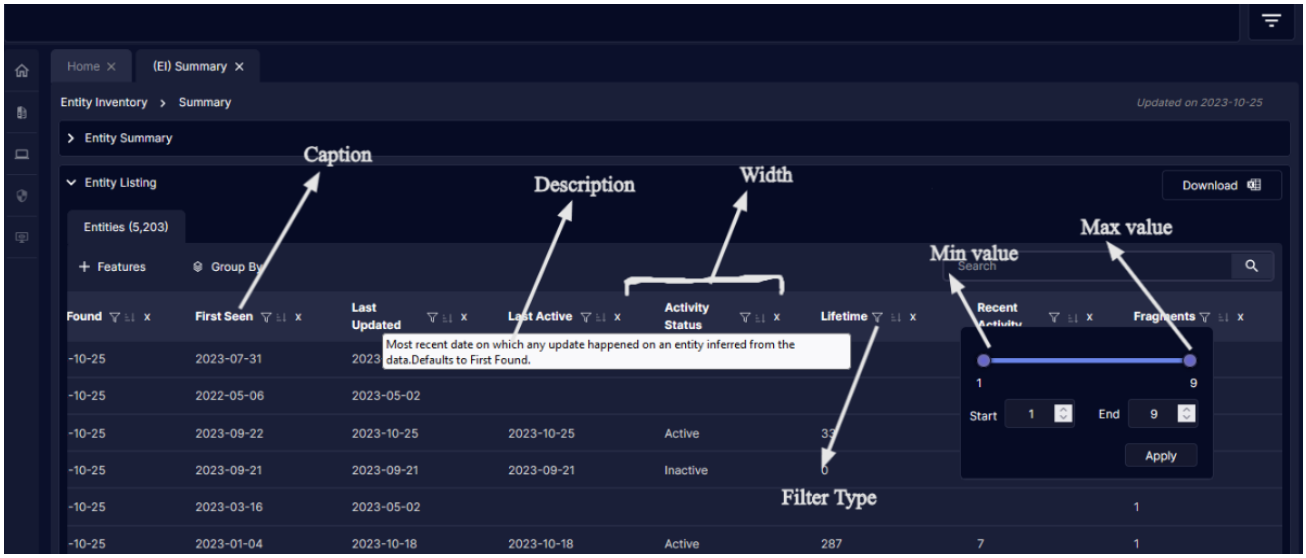
These settings are merged with the data dictionary and applied to corresponding fields based on their data types using update_attribute.py script.

- The "range_selection" attribute is responsible for enabling the range picker filter in the dashboard for data types integer and double. If this attribute is set to true in the data dictionary, the UI automation script will generate a range selection feature. This feature will dynamically provide "min" and "max" values for that specific field, fetching the information from Druid, enhancing the user's ability to filter and analyze data within a specified numerical range.

- The "solutions" attribute is updated through the execution of the update_attribute.py script within the Dockerfile of the client repository. This attribute plays a crucial role in defining the fields or entities to be displayed in the respective dashboards of VRA, AM, and EI. The default value is set to EI. If a particular solution wishes to exclude a specific field from being showcased in its dashboard, the "is_active" attribute can be set to False for that field. In such cases, the solution tag will not be added to the "solutions" attribute, preventing the field from appearing in the corresponding dashboards of that solution.

- The "ui_visibility" attribute serves to control the visibility of a field in the UI while retaining its presence in the data dictionary. When set to false, the field will not be included in the UI automation configuration. The default value is true, indicating that the field is visible in the UI by default.

- The "enable_hiding" attribute decides whether a field should appear on the landing page. Typically, fields under common properties are showcased on the landing page, while others remain hidden. The default value for "enable_hiding" is true, suggesting that, by default, a field will be hidden from the landing page. To make a field visible on the landing page, the "enable_hiding" attribute should be set to false. This setup provides a means to selectively include or exclude fields on the landing page based on specific criteria.

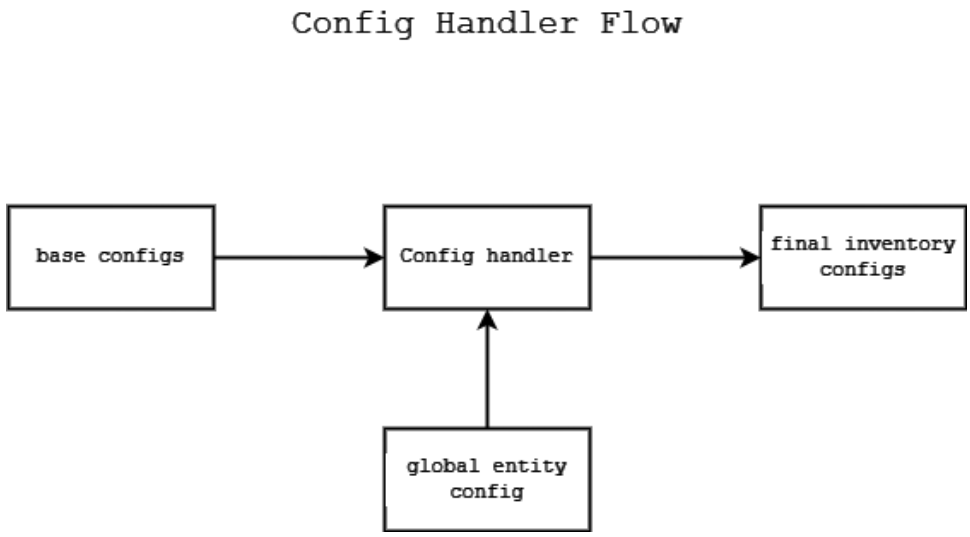


Mandatory attributes from sds_ei_data_dictionary.json are description, type, group and from sds_ei_relationship_data_dictionary.json are caption, type, description.

Attributes of data dictionary are compared with olap data through insight api request from beta 1.2 , so make sure user added necessary environment variables as given below in the corresponding gitops repo. Api configs folder contains payload data used for insight api request.

2.13. Configuration Handler

Entity Inventory as mentioned before has two sets of configurations, global entity and source level configs which are merged together before getting fed into the pipeline run. This "internal" merging is handled by config handler script. The script performs merging for loader and disambiguation separately.



2.13.1. Entity extraction config merge

Merging loader source configs with its corresponding global entity config is done by comparing the column name in both the configurations. Instead of mixing the spec lists during config merging, the field specs are overridden from the global config to the source models. This avoids both unexpected behavior across field specs and specs that are not relevant to a given statement. Attributes/features that are exclusive to global configurations, such as class, last updated attributes, etc., are included straight into the inventory models from global config.

2.13.2. Entity Resolution Config Merge

Entity resolution config merge is done between disambiguation configs and global entity.
Following are the changes made to the disambiguation configuration.

- Adding entity configuration to disambiguation from global configuration.
- Bringing common properties configurations from global config to disambiguation with some expression updates. This manipulation is necessary because these fields won't be calculated until the disambiguation process is complete, which may rename or alter the datatype of some fields. For instance, candidate keys will have the suffix "__resolved," and those introduced during the rollup strategy will be array-type.
- To retain all potential values for those fields, the candidate keys of inter source disambiguation are added to the rollup strategy of intra disambiguation configuration.

2.13.3. Data Dictionary Config Merge

The data dictionary merge occurs between the data dictionaries of Entity Inventory and other solutions.

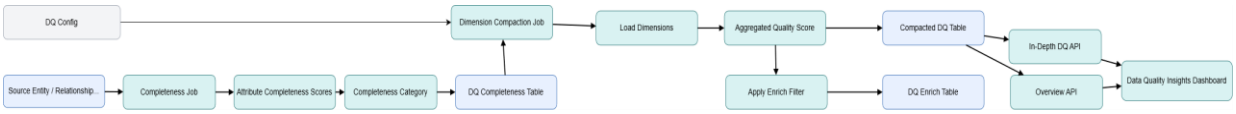
- In this process, dictionaries that are split by entities for entity dictionaries and by relationships for relationship dictionaries from each solution, along with UI configurations from both the client and the solution, is given as input.
- The output is a unified dictionary that includes a dashboard_identifier key, which is a nested dictionary containing solution-specific overrides.

2.13.4. Relationship Config Update

Config handler is capable of more than just merging configurations. Examining the input sources involved in relationship job is one of them. Multiple SDM inputs are supported by the EI relation extractor. It is likely that not all of the loaders/data sources defined in the EI solution are present in the client environment; to address this, the config handler examines the client repo's deployment config and removes loader config names that are not deployed in client from relationship configuration.

3. Configuring Data Quality

This section describes how Data Quality is configured and operationalized in the Knowledge Graph pipeline, including completeness calculation, dimension compaction, and dashboard population paths used by Data Quality Insights.



3.1. Completeness Score Calculation

3.1.1. Calculation Model

Completeness is calculated at attribute level during the DQ completeness Spark job. For each selected attribute, the score is evaluated as binary presence: missing values are scored as 0 and available values are scored as 1.
For array fields, empty arrays and arrays containing only null are treated as missing. For scalar fields, null is treated as missing and any non-null value is treated as present.

3.1.2. Formula and Aggregation Logic

- Attribute-level score formula: completeness_attr_scores.<attribute> = 0 (missing) or 1 (present).
- Record-level aggregation: cqs_total = SUM(attribute scores), number_cq_attributes = total included attributes.
- Final record completeness score: completeness_quality_score = ROUND((cqs_total / number_cq_attributes) * 100).

The score is stored as a percentage scale (0-100), where higher values indicate better completeness.

3.1.3. Field-Level vs Record-Level Completeness

Field-level completeness is persisted as a struct column named completeness_attr_scores, containing one score per attribute. Record-level completeness is persisted as completeness_quality_score for each p_id/disambiguated_p_id and source table context.

3.1.4. Threshold Configuration

Quality bands are configuration-driven in data quality configs using completeness.qualityBands (for example: Low 0-50, Medium 50-75, High 75-100). These thresholds are used to derive completeness_quality_score_category.
Attributes included in scoring are controlled through completeness.skipColumns, which prevents technical columns from skewing score computation.

3.1.5. Persistence and Consumption

Completeness output is persisted per entity/relationship in dq schema tables with pattern sds_ei__{entity}__dq_completeness (or relationship equivalent), partitioned by updated_at_ts day, object type, and table_name.
These outputs are consumed by the dimension compaction job to produce the unified DQ table and enrich table used by downstream APIs and dashboards.

3.2. Compaction Job

3.2.1. Purpose of Compaction

The dimension compaction job consolidates all available DQ dimension outputs into a single record-level DQ table per entity/relationship. This creates a stable serving table for analytics and dashboard queries.

3.2.2. Job Execution Flow of Compaction Job

- Read configured dq dimensions from data quality config (currently completeness in this release).
- For each dimension table, load records for current updated_at_ts snapshot.
- Outer-join dimension outputs on object type(entity/relationship), table_name and id key (p_id/relationship_id).
- Compute aggregated_quality_score as average of available *_quality_score columns.
- Write compacted table to sds_ei__{entity/relationship}__dq .
- Apply enrichFilter and optional enrichRequiredColumns, then write DQ enrich table for publisher/dashboard consumption.

3.2.3. Storage and Query Performance Impact

Compaction reduces multi-table dimension reads at query time by pre-merging dimension scores into one serving table. This lowers query complexity and improves consistency for both overview and in-depth widgets.
Enrich tables further optimize read paths by retaining only required records/columns for downstream consumption.

3.2.4. Integration with DQ Metrics

As additional dimensions are onboarded, compaction automatically merges new *_quality_score columns using the configured dqDimensions list, keeping the serving contract consistent without dashboard redesign.

3.3. Dashboard Population (In-Depth Dashboard)

3.3.1. End-to-End Data Retrieval Flow

- Step 1: UI calls getEntityGraphDQMetrics with entityGraphFilter and dqDetails (dimension, scoreType, entity, module/table context).
- Step 2: KG API normalizes query arguments. If dqTableName is not passed, DQ schema is resolved via Management API context and table name is derived.
- Step 3: Traversal plan builds PID scope from entityGraphFilter.
- Step 4: DQ query builder reads completeness_attr_scores.* metadata and builds completeness aggregation SQL with updated_at/module/table filters.
- Step 5: Query is executed through Spark/Kyuubi loader and mapped with attribute sorting/pagination for dashboard rendering.

3.3.2. Tables and Aggregation Logic

Primary source is the compacted DQ table (for example: {dq_schema}.sds_ei__{entity}__dq). For scoreType=INDIVIDUAL, the API aggregates attribute-level metrics from completeness_attr_scores.<attribute>. For scoreType=OVERALL, it aggregates completeness_quality_score.
PID scoping uses disambiguated_p_id from traversal output to ensure deep-dive metrics match the current entity filters in the graph context.

3.3.3. APIs and Service Interactions

Query entrypoint is getEntityGraphDQMetrics in knowledge-graph-api. Loader factory routes to Spark SQL DQ loader; query execution is delegated to Kyuubi connection.

3.4. Overview Dashboard Population

3.4.1. How It Differs from In-Depth Dashboard

Overview widgets use getEntityGraphMetricsDatalake to return grouped/summary metrics from selected serving tables, while in-depth widgets use getEntityGraphDQMetrics for attribute-level drilldown and dimension-specific diagnostics.

3.4.2. End to End Data Retrieval Flow

- UI sends getEntityGraphMetricsDatalake with entityAttributeMetrics (dimensions + metrics) and kg iceberg table.
- API resolves dq schema name(if not provided) from context.
- DataLake query builds base table query, applies updated_at and optional filters.
- If dq schema name is provided, graph filter is enforced by mapping traversal disambiguated_p_id to dq table p_id, then filtering base query.
- Aggregations are applied and results are returned for overview cards/charts.

3.5. Config Details

Config Format

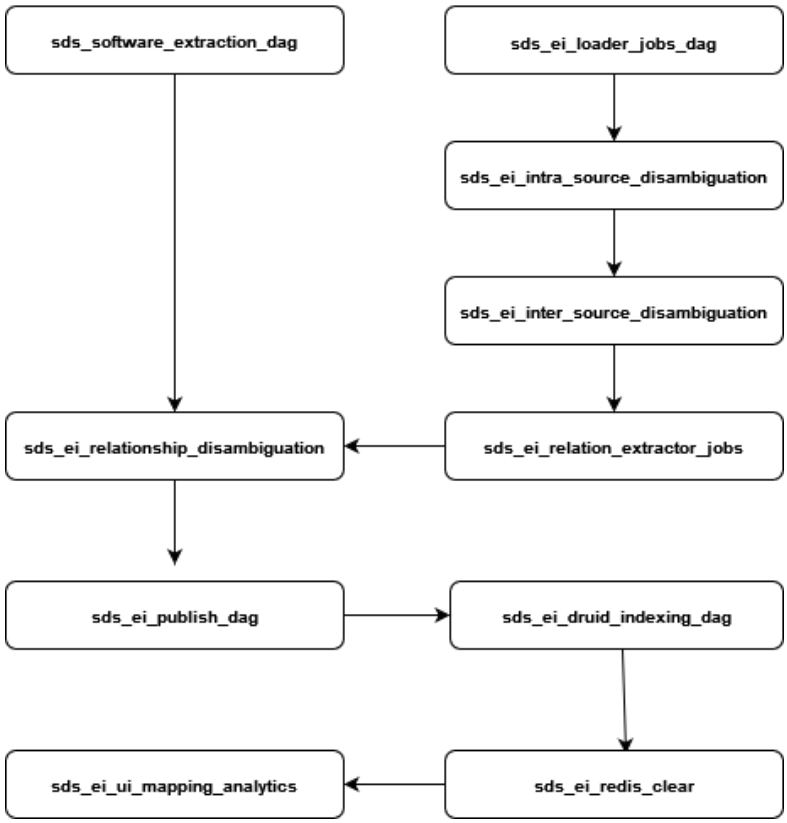
```
  {
    "dqDimensions":
    [
      " Lists quality dimensions that are enabled for evaluation (currently completeness)."
    ],
    "enrichFilter": " Filter for selecting records to enrich ",
    "enrichRequiredColumns":
    [
      “columns that are written back for enrichment"
    ],
    "completeness":
    {
      "skipColumns":
      [
          Columns excluded from completeness evaluation
      ],
      "qualityBands":
      [
        “Score ranges used to classify completeness as Low, Medium, or High.”
      ]
    }
  }
}
```

4. Orchestration Pipeline

The Entity Inventory analytical jobs involves multiple stages, each serving a specific purpose in the overall process. The division of tasks into multiple phases serves to reduce complexity and streamline the analytical workflow, facilitating easy configuration.

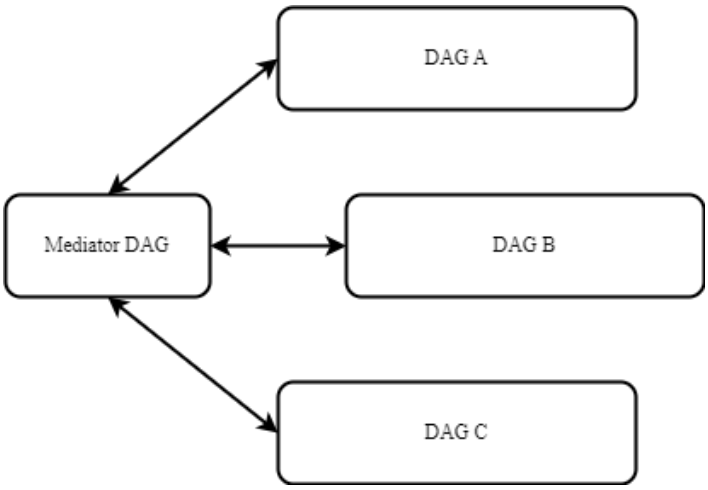
Entity inventory implemented its pipeline using Apache Airflow dags. The EI pipeline serves as the foundation for all other solutions. Other solution pipelines are started after the EI pipeline (majority of dags) finishes. When configured to work with other solutions, other dags might come in-between the flow but even then, the order of execution of EI dags will still be the same. A mediator dag is responsible for regulating the initiation of the subsequent dag upon completion. This dag continuously evaluates the status of each dag and initiates the subsequent dag upon completion.

4.1. Design



4.1.1. Mediator Pattern: Orchestrating Solution Pipelines

The orchestration of dag triggering is done by the mediator dag. Mediator dag is a utility in the platform orchestration component that manages and orchestrates DAGs in a pipeline that are interdependent. The inter dag dependency configuration, which is kept in airflow variables with the key " SDS_ORCHESTRATOR_CONFIG " lists the dags that are required to be a part of the pipeline.



The config has the below format:

Mediator Config Format

```
{
  "SDS_EI_PIPELINE": {
    "head_dag_id": "starting dag of the pipeline",
    "tail_dag_id": "ending dag of the pipeline",
    "sequential_pipeline_run": "true",
    "dependencies": [
      {
        "dependent_dag": "dag2",
        "dependencies": [
          "dag1"
        ]
      },
      {
        "dependent_dag": "dag3",
        "dependencies": [
          "dag2"
        ]
      }
    ]
  }
}
```

- According to the config format mentioned, “SDS_EI_PIPELINE” is the name given to the pipeline. This pipeline id should be given as an argument while creating dags to specify which pipeline the DAG should be a part of.
- The head dag should be an instance of SDSHeadDAG and all other dags including the tail dag should be an instance of SDSPipelineDAG.
- Head and tail dags mark the start and end of the pipeline.
- The param “sequential_pipeline_run” specifies whether to run the pipeline sequentially or not.
- The dependencies param has a list of dags with their dependent dags. For example, in the format given dag2 depends on dag1 which means dag1 should complete its run successfully to run dag2. Also, multiple dependencies can be mentioned in dependencies for a single dag

4.1.2. *Backfill DAG: Filling the missing dag runs*

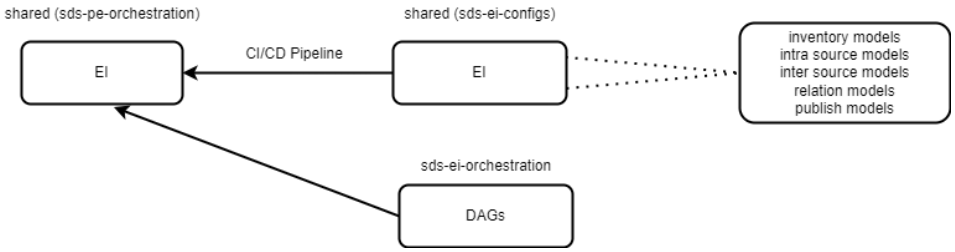
There can be times when we want to run pipeline between certain days past the current day. Especially in EI, entity extraction is based on many time-based parameters (like start epoch etc), therefore getting dag runs with those dates are unavoidable. To do this airflow provides backfill feature to create dag runs between two dates. To do this, we can get into the airflow scheduler and type the command as mentioned in the airflow documentation or we can use the backfill dag from the platform orchestration. For the backfill dag to work, we need to first set values of a variable called "SDS_BACKFILL_CONFIG".

Backfill config format

```
{
    "dag_id":"start dag name",
    "end_date":"end date (yyyy-mm-dd format) for the dag to run(exclusive)",
    "pipeline_id":"name of the pipeline",
    "start_date":"start date (yyyy-mm-dd format) of the dag to run(inclusive)"
}
```

4.1.3. *Generating dag tasks*

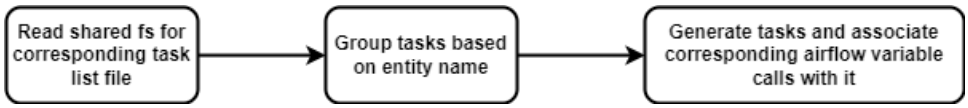
Most of the EI dags whose tasks can vary are dynamically generated. For example, the tasks in loader jobs depend on the client to client and solution to solution. Hence tasks need to be generated accordingly. This became possible by using the shared filesystem of airflow.



The shared space has multiple folders under sds-ei-configs folder for each set of configs models (shown in the diagram). Each of these folders has a json file having a list of task names. This list is generated by ci/cd pipeline. For example, the loader dag generates its tasks from inventory_models.json file which looks as shown in next page.

Inventory models task list in shared folder

The generated tasks are also grouped by the entities for all the dags possible. This allows us to selectively rerun a particular entity and also makes it easier to interact with airflow ui. Below is a flow of task generation in EI dags.



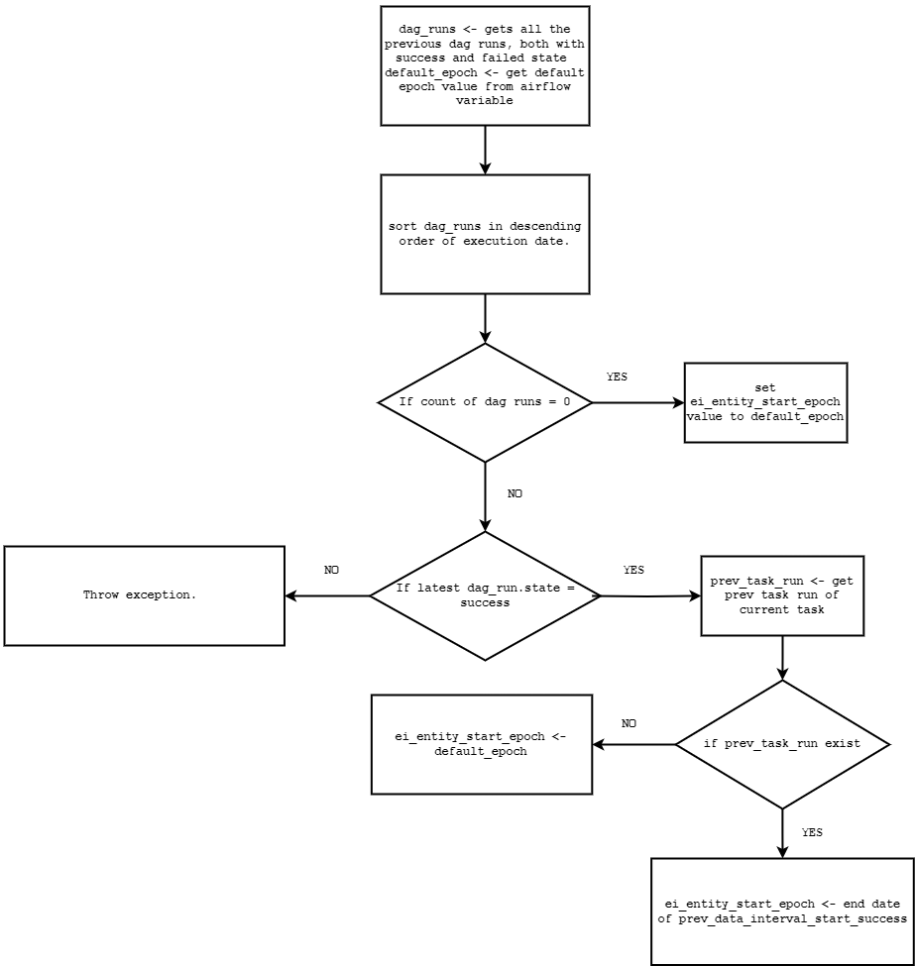
The variables for each of these jobs are kept in the airflow meta database (via CI/CD). The task list file is generated from the filename of entity extraction and entity resolution configs. The same name is also given as the key for its corresponding orchestration variables. For instance, when extracting a host entity from the active directory, the loader configuration file is named "sds_ei__host__active_directory__object_guid__job_config.json" and its orchestration variables file sds_ei__host__active_directory__object_guid__vars.json has configurations with key “sds_ei__host__active_directory__object_guid__job_config”. Eliminating the ".json" extension will result in identical names for both. This enables EI to retrieve the orchestration variables kept in the airflow database by iterating through the task lists and initializing an airflow variable call with the same key. Variables are linked to their task in this way.

4.2. Entity Extraction

Loader dag, sds_ei_loader_dag is the first dag in ei pipeline. The dag has two sets of tasks, calculating the parsed-interval-start parameter of the orchestration config and running the spark job for entity extraction.

4.2.1. Calculating epoch parameter

The find_task_run task is the one responsible for calculating parsed-interval-start. This parameter determines the filtration of input data for building the loader tables. The task calls the function find_task_run which has a logical flow as show in next page.



find_task_run flow

By default, the value of default_entity_epoch is set to 0000000000000, we can change that to any other value in common variables file under orchestration variables folder. If an exception is thrown, the whole pipeline stops, and its state is set to failed state.

4.2.2. Submitting spark job

After calculating ei_entity_start_epoch, spark job task is triggered. Following are the orchestration variables specific to spark job task. For loaders, explicitly set "spark.sql.caseSensitive" to "true".

Orchestration variable attributes	Usage	Is Mandatory
parsed interval start	Start epoch for input filtration	Yes
parsed interval end	End epoch for input filtration	Yes
event timestamp end	Source field. Used for filtration	Yes
previous end epoch	Last successful dag run’s end date	Yes

4.3. Entity Resolution

Entity resolution phase has 2 dags, sds_ei_intra_source_disambiguation and sds_ei_inter_source_disambiguation. Both of the dags have the same dag template, and their orchestration variable configs are also similar.

Orchestration variable attributes	Usage	Is Mandatory
config-path	Path to disambiguation config file	Yes
current-updated-date	Latest run's epoch date	Yes
previous-updated-date	Last successful dag run's end date	Yes

4.4. Entity Fragment Write

Fragment write operations occur in the sds_ei__inter_fragment_models_dag DAG, where fragment write tasks are generated corresponding to each entity inter-disambiguation. The execution of these fragment write tasks is controlled using the pre_execute operation when the task runs in Airflow. During the pre_execute step, the corresponding disambiguation configuration is checked, and if isFragmentOLAPTable is set to false, the fragment write task is skipped.

Orchestration variable attributes	Usage	Is Mandatory
config-path	Path to disambiguation config file	Yes
current-updated-date	Latest run's epoch date	Yes
previous-updated-date	Last successful run's end date	Yes

4.5. Relationship Extraction

Relationships are built by sds_ei_relation_extractor dag. Following are the spark job related arguments passed from orchestration variable:

Orchestration variable attributes	Usage	Is Mandatory
parsed interval start	Start epoch for input filtration	Yes
parsed interval end	End epoch for input filtration	Yes
event timestamp end	source field. Used for filtration	Yes
previous-end-epoch	Last successful dag run's end date	Yes
config-path	Path to relationship config file	Yes
output-path	Name of output table	Yes
prev-mini-sdm	Name of mini SDM table.	Yes

4.6. Relationship Resolution

Relationships are resolved using sds_ei_relationship_disambiguation dag. Following are the spark job related arguments passed from orchestration variable:

Orchestration variable attributes	Usage	Is Mandatory
config-path	Path to relationship disambiguation config file	Yes
current-updated-date	Latest run's epoch date	Yes

4.7. Dynamic DAG Generator (Entity Relationship Enrichment & Publisher)

A Dynamic Auto DAG Generator is a flexible and reusable pipeline framework that creates DAGs dynamically based on configurations. Instead of hardcoding DAGs for every task, this approach allows you to define workflows through configuration files.

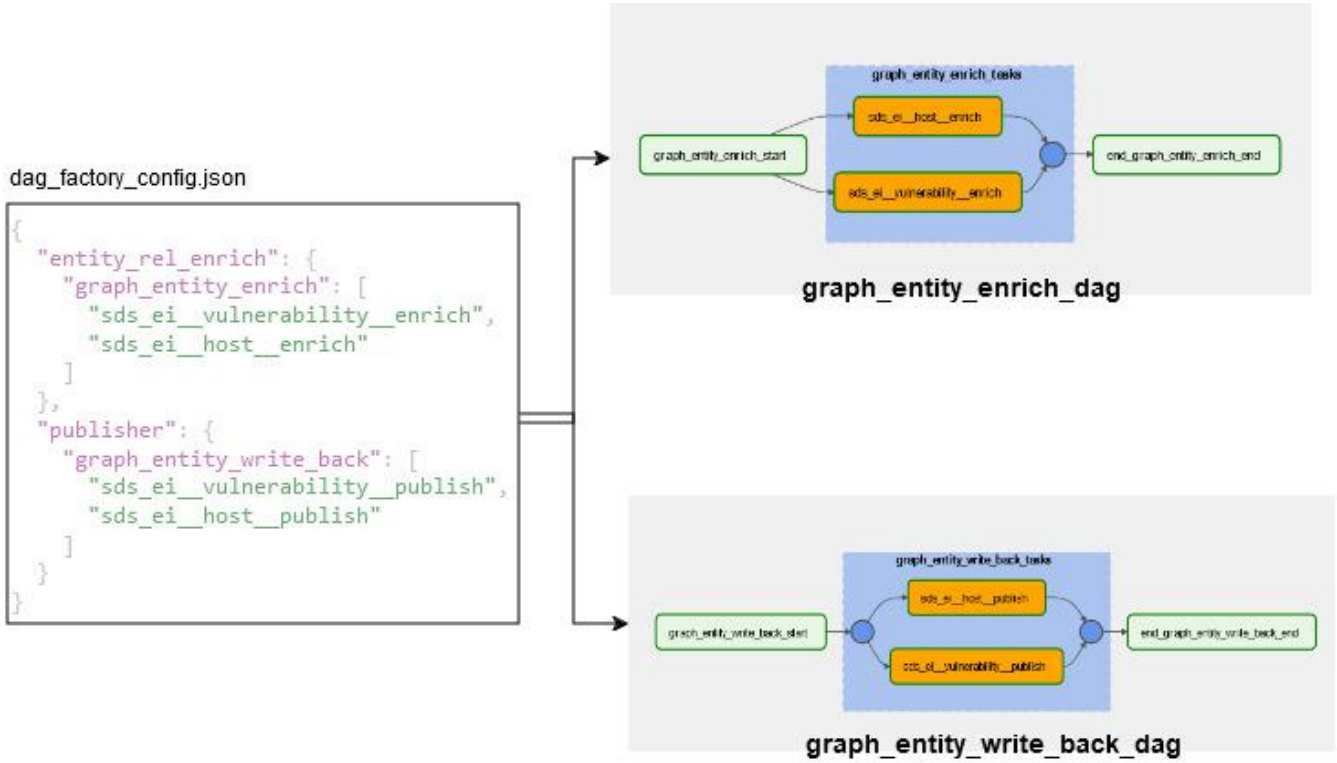
How It Works

Configuration File (dag_factory_config): A JSON file defines the DAG structure by specifying parent templates, DAG names, and task lists.

Dynamic DAG Creation: The framework dynamically generates DAGs by combining parent templates, DAG names, and tasks from the configuration file.

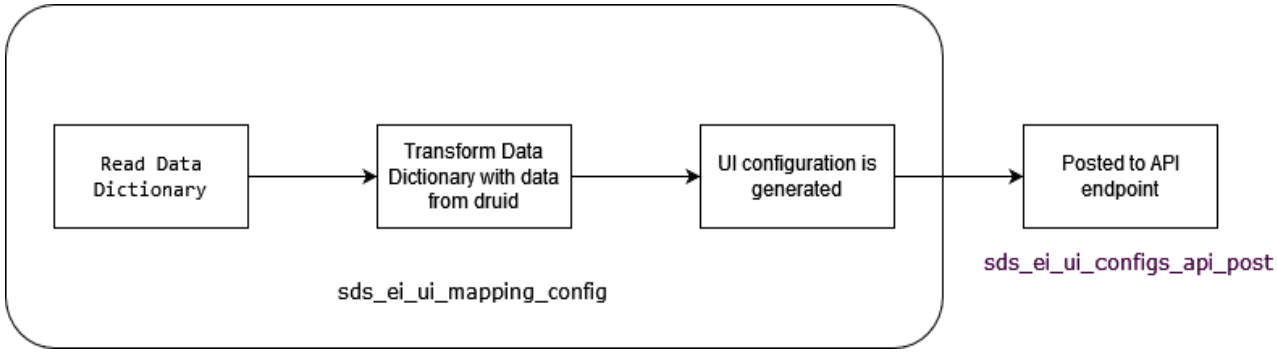
Template Integration: Each DAG uses a standard structure with start/end tasks, grouped task organization, and dynamic task creation using SparkOperatorFactory.

Template Variable Rendering: Variables are dynamically generated based on the parent key, enabling auto-configuration for tasks.



4.8. Data Dictionary Mapping

UI automation DAG is the final step in orchestration pipeline which is used to power ui with the help of a data dictionary. The Dag's initial step involves reading data dictionary and performing transformations with data fetched from Druid. Subsequently, the transformed data, referred to as the UI configuration, is sent to the API endpoint. Check the *Entity Explorer* section for a detailed explanation of how UI automation works.



The sds_ei_ui_mapping_config script encompasses the logic outlined earlier, involving the process of reading data dictionaries, and transforming them using information retrieved from the Druid database. Connection to Druid is established through the Druid broker API, utilizing a POST request to execute queries. The specified field values are then extracted from the resulting JSON response and returned as a list. The necessary SSL verification and connection details are managed internally. The Druid connection information is configured in the orchestration GitOps YAML file under the environment variable "AIRFLOW_CONN_DRUID_CONN," specifying the broker's URL and port.

In sds_ei_ui_configs_api_post, the "GET" method is employed to verify the existence of an API endpoint. If the endpoint exists, data is patched to that specific endpoint; otherwise, if it doesn't exist, the data is posted to create the endpoint.

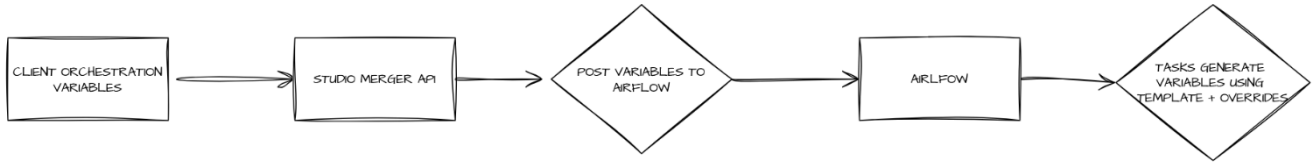
Method Used	End Point
POST	/sds_mgmnt/api/v1/config
PATCH	/sds_mgmnt/api/v1/config/entity_inventory
GET	/sds_mgmnt/api/v1/config/entity_inventory
PUT	Not allowed

4.9. EI Auto Orchestration Config Generation

The proposed method leverages the similar structure shared by various types of EI tasks. In the case of loader orchestration variables, most attributes are consistent across all loaders, while unique attributes for each task can be generated dynamically as the tasks are created or executed.

In this approach, a base template for each job type is maintained within the solution repository and made accessible through Airflow variables. Task-specific orchestration configurations are then generated from these templates. For instance, loader orchestration variables previously required manual configuration by data engineers or analysts, who had to create and name

orchestration files individually for each loader or disambiguation task added to the pipeline. These configurations were then posted to Airflow variables during deployment. While this manual method provided precise control over each variable, it had a significant drawback: manually created file names could deviate from EI's intended naming conventions, causing tasks to fail.



The proposed method reduces this risk by using standardized templates, ensuring consistency and simplifying task configuration across the pipeline.

Loader Orchestration Template

```
{
  "args": [
    "--spark-service",
    "spark",
    "--parsed-interval-end",
    "{{
str(pendulum.parse(dag_run.conf['data_interval_end']).timestamp()*1000).split('.')[0] }}",
    "--event-timestamp-end",
    "{{ get_end_date(data_interval_start, 'day','utc') }}",
    "--previous-end-epoch",
    "{{ get_end_date(prev_data_interval_start_success, 'day','utc') if
prev_data_interval_start_success is not none else -1 }}"
  ],
  "base_config_path": "<%CONFIG_ARTIFACTORY_URI%><%EI_LOADER_CONFIG_BASE_PATH%>",
  "class_name": "ai.prevalent.entityinventory.loader.Loader",
  "conf": {
    "spark.checkpoint.dir": "<%CONFIG_ARTIFACTORY_URI%>/ei-checkpoint/",
    "spark.blacklist.decommissioning.timeout": "600s",
    "spark.dynamicAllocation.enabled": "false",
    "spark.yarn.heterogeneousExecutors.enabled": "false",
    "spark.sql.shuffle.partitions": "100",
    "spark.driver.maxResultSize": "200M",
    "spark.executor.heartbeatInterval": "10000",
    "spark.sql.caseSensitive": "true"
  },
  "driver_cores": "<%LOADER_DRIVER_CORES%>",
  "driver_memory": "<%LOADER_DRIVER_MEMORY%>",
  "executor_cores": "<%LOADER_EXECUTOR_CORES%>",
  "executor_memory": "<%LOADER_EXECUTOR_MEMORY%>",
  "executor_instances": "<%LOADER_EXECUTOR_INSTANCES%>",
  "application_file": "<%ARTIFACTORY_URI%>/sds/data-analytics/lib/latest/sds-ei-
analytics_2.12.jar"
}
```

5. Repository Management

5.1. EI Configs

The sds-ei-configs repository is well-organized with dedicated folders for various aspects of the EI system, particularly focusing on configuring Spark jobs and related processes. This modular structure simplifies managing configurations for different components of the system. The repository is structured with the following folders:

- config_merger_api: Previously, config transformation involves two components: the EI config merger and config manipulator scripts. In Beta 1.1, each configuration folder has its own dedicated script. The code for the EI config merge API, along with all its related scripts, test cases, and lineage, is organized within the config_merger_api folder.
- olap_configs: The "druid_indexing_configs" folder has been renamed to "olap_configs" in the platform release 3.4, this change was made to accommodate the addition of RDBMS OLAP engine support. This folder contains configuration files for the Druid indexing process. These files define the data sources, dimensions, and metrics that are used to index the entity inventory data.
- orchestration_shared_fs: This folder contains configurations or files related to shared file systems used in the orchestration process.
- orchestration_variables: This folder holds variables or configuration files used in the orchestration process.
- scripts: This folder contains various scripts, potentially used for automation or other tasks in EI.
- spark_job_configs: This folder contains configurations for Spark jobs, a distributed computing system.
    - global_entity_config: This subfolder holds global configurations specific to entity-related processing in Spark jobs.
    - intrasource_disambiguated_models: This subfolder contains configurations related to intrasource disambiguation.
    - inventory_models: This subfolder contains configurations related to inventory building in Spark jobs.
    - intersource_disambiguated_models: This subfolder contains configurations related to intersource disambiguation.
    - relationship_models: This subfolder contains configurations specific to models handling relationships between entities.
    - relationship_disambiguation: This subfolder contains configurations for relationship disambiguation.
    - publisher: This subfolder holds configurations related to publishing data.
    - entity_rel_enrich: This subfolder potentially contains configurations for enriching data related to entity relationships.

5.2. Config Merging

Clients may want to add extra sources or modify logic specific to their environment while consuming EI . They might choose only certain sources or adjust certain logic based on their data behavior. To facilitate this customization without disrupting the logic or functionalities of the EI jobs, config merging, also referred to as config patching is introduced. This process enables clients to customize EI configurations while ensuring that the core functionality of the EI is not compromised. It also prevents clients from misplacing objects within the configurations.

For config merging, we keep config merger files for each folder (path: sds-ei-configs\config_merger_api\utils)

To customize, clients need to make a folder in their repository (path: (client repo)/sds-ei-configs/) and create files mirroring those in the sds-ei-configs, maintaining the same folder structure. Any modifications or additions to logic should be done in

accordance with the hierarchy of folders in the EI. Simply put, replicate the structure and content of the sds-ei-configs folder in your client repository for any changes or additions.

To handle custom configuration sections, the script utilizes a deployment configuration file (path: (client repo)/sds-ei-configs/deployment_config.json). This file defines a list of configurations to be extracted from the solution repository (sds-ei-configs). The script processes and carries forward only the folders and files specified in the deployment configuration file.

- To include specific files or folders from the solution repository, list them explicitly in the deployment configuration file. For instance, if you only need one relationship model from the solution, you would add the following to the deployment configuration file:

```
{
  "spark_job_configs": {
    "source_models": {
      "relationship_models": [
        "sds_ei__rel__host_has_identity__job_config"
      ]
    }
  }
}
```

- To omit a folder from the deployment, leave it blank in the deployment configuration file. For example, to exclude the sds_ei_druid_indexing_config folder, you would add the following to the deployment configuration file:

```
{
  "sds_ei_druid_indexing_config": []
}
```

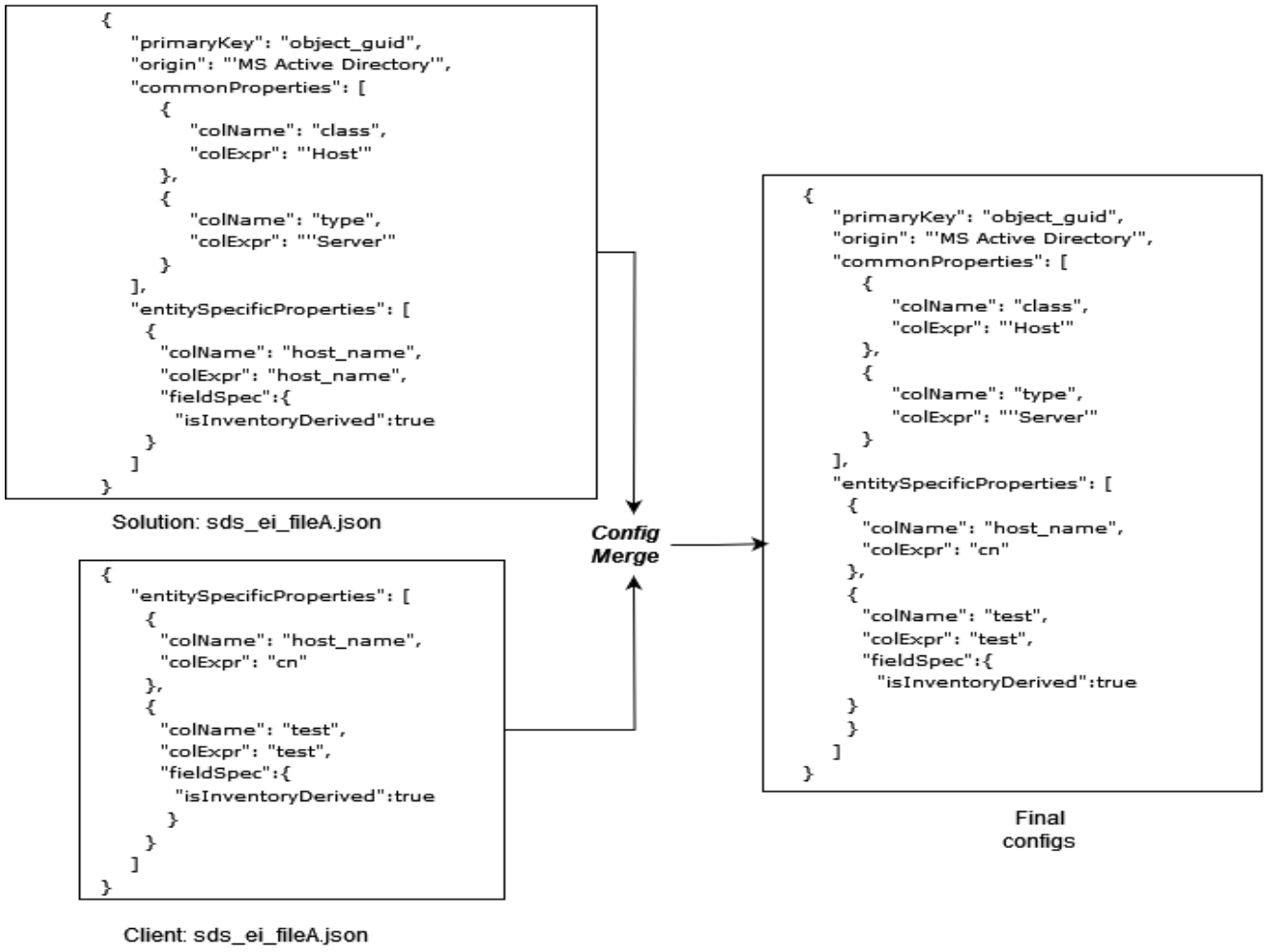
- If a folder is not mentioned in the deployment configuration file, all files within that folder will be included in the deployment. This implies that if you don't want a particular folder, you should explicitly indicate it as empty in the deployment configuration file.

deployment_config.json

```
{
    "sds_ei_druid_indexing_config": [],
    "spark_job_configs": {
        "source_models": {
            "global_entity_config": [
                "host_entity_config",
                "identity_entity_config"
            ],
            "intrasource_disambiguated_models": [
                "sds_ei__host__globalprotect_vpn",
                "sds_ei__identity__ms_defender"
            ],
            "inventory_models": [
                "sds_ei__host__active_directory__object_guid",
                "sds_ei__host__globalprotect_vpn__client_hostname",
                "sds_ei__host__globalprotect_vpn__device_hostname",
                "sds_ei__host__ms_defender_device_events__device_id",
                "sds_ei__identity__ms_defender_device_list__id",
                "sds_ei__identity__ms_defender_device_software__device_id"
            ],
            "relationship_models": [
                "sds_ei__rel__host_has_identity"
            ],
```

```
            "relationship_disambiguation":[] ,
            "intersource_disambiguated_models": [
                "sds_ei__host",
                "sds_ei__identity"
            ],
            "publisher": [
                "sds_ei__publish_transformer__entity_inventory",
                "sds_ei__publish_transformer__non_disambiguated_",
                "sds_ei__publish_transformer__relationship"
            ]
        }
    }
}
```

After filtering the solution and client repositories based on the deployment_config.json, the subsequent step involves merging configurations based on each folder merging logic.



5.2.1. Handling Array of objects

In the typical merging process, values with the same key are overridden by the client's values. However, when dealing with arrays of objects, the entire array is replaced by the client's array. To address this, a key is specified in the script to determine merging based on the values inside the objects. The specified keys for merging are as follows:

Properties	Merging Key
commonProperties	colName
entitySpecificProperties	colName
sourceSpecificProperties	colName
temporaryProperties	colName
inventoryModelInput	path
aggregation	field
fieldLevelConfidenceMatrix	field
valueConfidence	field
optionalAttributes	name
derivedProperties	colName
inputSourceInfo	sdmPath

For confidence-related properties like "confidenceMatrix," appending is not considered during merging; instead, the client's value should override the existing one.

5.2.2. Shift Right Feature and Column Expression Merge Strategy of Config Merger

Shift Right Feature

The config-merger shift right feature provides control over when entity properties are calculated in the Knowledge Graph pipeline. This feature optimizes performance by allowing developers to specify exactly which phase should process specific properties, reducing redundant calculations across the pipeline.
Previously, field level; calculations were performed in all phases (loader, intra, inter), which could lead to:

- Unnecessary recomputation of properties
- Higher processing time and cost.
- Developers need to run the entire pipeline even for minor changes.

The computation_phase attribute has been introduced in the Global Entity Config to specify when properties should be calculated. This attribute can be added to both common properties and entity-specific properties under their fieldsSpec section.

Available Computation Phase Values

Value	Description
loader	Property is calculated only during the loader phase. It will not be recalculated in later phases
inter	Property is calculated only during the inter phase. This is the default if not specified.
all	Property is calculated in both loader and inter phases.

When merging configurations, properties are filtered based on their computation phase before being included in the relevant configuration object.

Global Entity Config With shift right feature

```
{
 "entityClass": "Host",
 "commonProperties": [
   {
     "colName": "first_seen_date",
     "colExpr": "LEAST(last_active_date,first_found_date)",
     "fieldsSpec": {
       "isInventoryDerived": true,
       "computation_phase": "loader"
     }
   },
   {
     "colName": "last_active_date",
     "colExpr": "cast(null as bigint)",
     "fieldsSpec": {
       "isInventoryDerived": true,
       "computation_phase": "inter"
     }
   }
 ],
 "entitySpecificProperties": [
   {
     "colName": "ip",
     "colExpr": "cast(null as string)",
     "fieldsSpec": {
       "isInventoryDerived": false,
       "computation_phase": "all"
     }
   }
 ]
}
```

Column Expression Merge Strategy

Traditionally, the client configuration would completely override the solution's configuration, forcing clients to reimplement complex expressions even when they only need to make small modifications.
The merge strategy feature enables intelligent combination of column expressions from both solution and client configurations. This allows clients to extend or modify expressions without having to completely rewrite them.

Merge Strategies

Strategy	Description
OVERRIDE	Client configuration takes complete precedence (default)
CLIENT_FIRST	Client expression is used first, falling back to solution expression if null
PRODUCT_FIRST	Solution expression is used first, falling back to client expression if null

The merge strategy is specified in the fieldsSpec section of a property.

Merge Strategy Scenario

```
// Solution Configuration
{
 "colName": "device_status",
 "colExpr": "CASE WHEN last_seen_date IS NULL THEN 'Unknown' WHEN datediff(current_date(),
last_seen_date) > 60 THEN 'high' ELSE 'low' END"
}

// Client Configuration
{
 "colName": "device_status",
 "colExpr": "CASE WHEN datediff(current_date(), last_seen_date) between 20 and 30 THEN
'moderate' ELSE NULL END",
  "fieldsSpec": {
    "merge_strategy": "CLIENT_FIRST"
  }
}
```

In the above scenario

- First check if the condition falls in the 20–30-day range then return 'moderate'
- If not, fall back to the solution's logic for 'Unknown', 'high', or 'low' statuses

The client only needed to specify their addition (the 'moderate' case) while preserving all the solution's logic, significantly simplifying configuration and maintenance.

5.2.3. Config Merging for Each Folder

This section delves into the unique scenarios related to config merging for individual folders within spark_job_configs/source_models. Apart from the general merging process applicable across all folders, the following use cases are specific to each folder:

- *Global Entity Configurations*:
  - In this folder, each file from the solution is meticulously examined in comparison to the corresponding file from the client to identify any misplacement of properties.
  - The script proceeds by considering the solution as the source of truth and cross-references each field in the client to identify any misplacement across properties.
  - If a field (let's say, Field A) is common between the solution and the client, and it is intended to be overridden in the client, users must ensure that they place it under common properties.
  - Failure to adhere to this structure results in raising an exception to ensure data integrity and proper configuration alignment.
- *Intersource/Intrasource disambiguation Configurations:*
  - The folders "intersource_disambiguated_models" and "intrasource_disambiguated_models" share identical configurations. If a source, added in "intersource" disambiguation from the solution, is having "intra" on the client side, the corresponding entry in the "inter" configurations is eliminated. This streamlines the output and reduces redundancy.
  - In the disambiguation process, each field must adhere to a single strategy, either "valueConfidence," "fieldLevelConfidenceMatrix," "aggregation," or "rollingUpFields." If the client specifies a different strategy for a field already defined in the solution, the entry is removed from the solution configurations. This ensures that each field maintains a consistent strategy.
- *Inventory Configurations (Loader):*
  - In loader configurations, if a field from the solution exists in the client's configuration for the same file, fieldSpec corresponding to the field is removed from solution.
- *Relationship models:*

- In relationships, apart from the usual merging of configurations, sourceLoaderConfPath and targetLoaderConfPath inside inputSourceInfo is checked against the loader configs and if any loaders aren't present then it will be removed from inputSourceInfo. After this removal, if either sourceLoaderConfPath or targetLoaderConfPath becomes an empty list, the entire sdmPath entry will be removed from inputSourceInfo.

- *Relationship Disambiguation:*
  - Usual merging logic is applicable.
- *Entity Relationship Enrich*
  - For entity_rel_enrich, both sourceFieldName and targetFieldName are compared with client configs for overriding.

For studio merger api documentation refer EI Config Merge API confluence page.

5.3 Config Context Variables

A config context, as illustrated in the provided example, is a JSON file containing key-value pairs specifying various parameters and attributes. This file acts as a set of configuration settings that can be customized by the client to influence the behavior of a system or application during deployment. The values within the config context file serve as placeholders for dynamic information that can be replaced during the deployment process.

context.json

```
{
  "RECON_DB_USERNAME":"db name",
  "RECON_DB_JDBC_URL":"jdbc url",
  "DATALAKE_URI":"s3a://pai-sol-integration-datalake",
  "ARTIFACTORY_URI":"s3a://pai-sol-integration-apps",
  "CONFIG_ARTIFACTORY_URI":"https://sds3.solution.prevalent.ai",
  "EI_LOADER_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
  "EI_INTRASOURCE_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-
job-configs/",
  "EI_INTERSOURCE_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-
job-configs/",
  "EI_RELATION_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
  "EI_RELATION_DIS_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-
job-configs/",
  "EI_PUBLISHER_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
  "EI_SPARK_CONFIGS_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
  "EI_PUBLISHER_INTERSOURCE_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-
item/?config_item_type=intersource_disambiguated_models",
  "LOG_URI":"s3a://pai-sol-integration-logs",
  "AWS_ARN_ROLE":"aws arn role",
  "AWS_REGION_NAME":"ap-south-1",
  "KUBE_NAMESPACE":"dev",
  "SRDM_SCHEMA_NAME":"srdm",
  "SDM_SCHEMA_NAME":"sdm",
  "KEYCLOAK_HOSTNAME":"https://sds3.solution.prevalent.ai",
  "SDM_IMAGE_TAG":"prevalentai/sol-dbt:sds-sol-sdm-dbt-1-0-0-92",
  "EI_SCHEMA_NAME":"ei",
  "EI_PUBLISH_SCHEMA_NAME":"ei_publish",
  "EI_POST_SCHEMA_NAMES": "ei,ei_post,cam,vra",
```

```
"AM_DBT_IMAGE_TAG":"1-0-0-51",
"AM_SCHEMA_NAME":"cam",
"VM_SCHEMA_NAME":"vra",
"AM_ANALYTICS_DBT_IMAGE_TAG":"prevalentai/am-dbt:sds-am-analytics-dbt-1-1-0-16",
"DRUID_INDEXING_TYPE":"s3",
"DRUID_DATALAKE_URI":"s3://pai-sol-integration-datalake",
"SPARK_IMAGE_NAME":"spark image name",
"HIVE_METASTORE_HOST":"10.125.1.191",
"LOADER_DRIVER_CORES":"1",
"LOADER_DRIVER_MEMORY":"3g",
"LOADER_EXECUTOR_CORES":"2",
"LOADER_EXECUTOR_INSTANCES":"1",
"LOADER_EXECUTOR_MEMORY":"3g",
"LOADER_SHUFFLE_PARTITIONS":"100",
"INTRASOURCE_DRIVER_CORES":"1",
"INTRASOURCE_DRIVER_MEMORY":"3g",
"INTRASOURCE_EXECUTOR_CORES":"2",
"INTRASOURCE_EXECUTOR_MEMORY":"3g",
"INTRASOURCE_EXECUTOR_INSTANCES":"1",
"INTRASOURCE_SHUFFLE_PARTITIONS":"100",
"INTERSOURCE_DRIVER_CORES":"1",
"INTERSOURCE_DRIVER_MEMORY":"3g",
"INTERSOURCE_EXECUTOR_CORES":"2",
"INTERSOURCE_EXECUTOR_MEMORY":"6g",
"INTERSOURCE_EXECUTOR_INSTANCES":"1",
"INTERSOURCE_SHUFFLE_PARTITIONS":"100",
"PUBLISHER_DRIVER_CORES":"2",
"PUBLISHER_DRIVER_MEMORY":"3g",
"PUBLISHER_EXECUTOR_CORES":"2",
"PUBLISHER_EXECUTOR_MEMORY":"3g",
"PUBLISHER_EXECUTOR_INSTANCES":"2",
"PUBLISHER_SHUFFLE_PARTITIONS":"100",
"RELATIONSHIP_DRIVER_CORES":"1",
"RELATIONSHIP_DRIVER_MEMORY":"10g",
"RELATIONSHIP_EXECUTOR_CORES":"2",
"RELATIONSHIP_EXECUTOR_MEMORY":"12g",
"RELATIONSHIP_EXECUTOR_INSTANCES":"2",
"RELATIONSHIP_SHUFFLE_PARTITIONS":"100",
"SECRETS_MANAGER_NAME": "secret name",
"PYTHON_SPARK_3_IMAGE":"python spark image",
"VM_VUL_RELATION_TBL_NAME": "sds_ei__rel__vulnerability_finding_on_host",
"CCM_SCHEMA_NAME":"ccm",
"ADMIN_API_BASE_PATH":"https://sds3.solution.prevalent.ai",
"CCM_CEI_METRIC_EXEC_DRIVER_CORES":"1",
"CCM_CEI_METRIC_DRIVER_MEMORY":"1g",
"CCM_CEI_METRIC_EXECUTOR_CORES":"2",
"CCM_CEI_METRIC_EXECUTOR_INSTANCES":"2",
"CCM_CEI_METRIC_EXECUTOR_MEMORY":"4g",
"CCM_CEI_METRIC_SUMMARY_EXEC_DRIVER_CORES":"1",
"CCM_CEI_METRIC_SUMMARY_DRIVER_MEMORY":"1g",
"CCM_CEI_METRIC_SUMMARY_EXECUTOR_CORES":"2",
"CCM_CEI_METRIC_SUMMARY_EXECUTOR_INSTANCES":"2",
"CCM_CEI_METRIC_SUMMARY_EXECUTOR_MEMORY":"2g",
"KEYCLOAK_REALM":"dev",
"CEI_CONFIGS_RECORD_LIMIT":100,
"GROUP_SIZE": 100,
"CONTINUE_ON_CEI_PARTIAL_SUCCESS": false,
```

```
  "SDS_SOFTWARE_EXTRACTION_DRIVER_CORES": "1",
  "SDS_SOFTWARE_EXTRACTION_DRIVER_MEMORY": "1g",
  "SDS_SOFTWARE_EXTRACTION_EXECUTOR_CORES": "2",
  "SDS_SOFTWARE_EXTRACTION_EXECUTOR_INSTANCES": "6",
  "SDS_SOFTWARE_EXTRACTION_EXECUTOR_MEMORY": "3g",
  "VRA_CONSEQUENCE_ENRICHMENT_DRIVER_CORES": "1",
  "VRA_CONSEQUENCE_ENRICHMENT_DRIVER_MEMORY": "1g",
  "VRA_CONSEQUENCE_ENRICHMENT_EXECUTOR_CORES": "3",
  "VRA_CONSEQUENCE_ENRICHMENT_EXECUTOR_INSTANCES": "4",
  "VRA_CONSEQUENCE_ENRICHMENT_EXECUTOR_MEMORY": "8g",
  "VRA_HOST_CONSEQUENCE_ROLLUP_DRIVER_CORES": "1",
  "VRA_HOST_CONSEQUENCE_ROLLUP_DRIVER_MEMORY": "1g",
  "VRA_HOST_CONSEQUENCE_ROLLUP_EXECUTOR_CORES": "3",
  "VRA_HOST_CONSEQUENCE_ROLLUP_EXECUTOR_INSTANCES": "4",
  "VRA_HOST_CONSEQUENCE_ROLLUP_EXECUTOR_MEMORY": "8g",
  "VRA_RISK_SCORE_DRIVER_CORES": "1",
  "VRA_RISK_SCORE_DRIVER_MEMORY": "1g",
  "VRA_RISK_SCORE_EXECUTOR_CORES": "3",
  "VRA_RISK_SCORE_EXECUTOR_INSTANCES": "4",
  "VRA_RISK_SCORE_EXECUTOR_MEMORY": "8g",
  "VRA_DATA_ANALYTICS_GENERATION_DRIVER_CORES": "1",
  "VRA_DATA_ANALYTICS_GENERATION_DRIVER_MEMORY": "2g",
  "VRA_DATA_ANALYTICS_GENERATION_EXECUTOR_CORES": "3",
  "VRA_DATA_ANALYTICS_GENERATION_EXECUTOR_INSTANCES": "4",
  "VRA_DATA_ANALYTICS_GENERATION_EXECUTOR_MEMORY": "8g",
  "LOOKUP_SCHEMA_NAME": "lookup"
}
```

In the given context, the config context file (client_repo)/config_context/context.json) is stored in the client's repository. This file includes attributes such as AWS settings, Spark configurations, data lake URIs, Docker image tags, and various parameters related to the deployment and execution of different components in EI data processing or analytics system.. Clients can modify these attributes based on their specific requirements.

The implementation involves a script (config_template_renderer.py mentioned in respective gitops repository) that uses the Jinja2 templating engine to replace placeholders in JSON files with values from the config context file. This script takes two arguments: the path to the config context file (context_path) and the path to the input files (input_path) where the placeholders need to be replaced. It recursively searches for JSON files in the specified paths, applies the template rendering, and overwrites the original files with the updated values.

5.4 Update Attribute in Data Dictionary

The script is designed to update a merged data dictionary with additional attributes specific to different solutions (e.g., VRA, AM, EI). It adds solution attribute to each field based on the keys found in the vm_configs_file and am_configs_file and uses a UI config file (sds_ei_ui_config.json) that provides attributes like width, step_interval, etc., for different data types (string, integer, timestamp, double) inside data dictionary. The purpose of the solution attribute is to identify the entities or attributes that are relevant or consumed by each individual solution. This attribute serves as a reference or indicator to determine which entities, relationships or attributes should be displayed or made available to a particular solution. The UI config file contains common information for certain data types, so it is kept in a single file and added when the script is executed.

Flow of update_attribute.py Script:

- The script is initiated with the following command inside docker file of client repository:

```
RUN python /opt/airflow/ei_configs/scripts/update_attribute.py --ei_configs_file
<merged data dictionary path> --am_configs_file <base am configs data dictionary
path (without merging)> --vm_configs_file <base vm configs data dictionary path
(without merging)> --ui_configs_file
/opt/airflow/final_configs/orchestration_shared_fs/sds-ei-configs/ui-
configs/sds_ei_ui_config.json

RUN python /opt/airflow/ei_configs/scripts/update_attribute.py --ei_configs_file
<merged data dictionary path> --vm_configs_file <base vm configs relationship data
dictionary path (without merging)> --ui_configs_file
/opt/airflow/final_configs/orchestration_shared_fs/sds-ei-configs/ui-
configs/sds_ei_ui_config.json
```

The script is executed with arguments to specify the paths to the merged data dictionary (ei_configs_file), VM dictionary (vm_configs_file), AM dictionary (am_configs_file), and UI configurations (ui_configs_file).

- The script processes data dictionary and updates fields based on keys (key will be entities from dictionary point of view) found in AM and VM configs. For example, if the VM configs have keys like host and vulnerability, the script adds the solution attribute "VRA" to all fields in the corresponding entities inside the merged data dictionary. The script then merges the updated merged data dictionary with the ui config file.

  Sample Data Dictionary Field Before:

```
"host_name": {
        "caption": "Hostname",
        "description": "Hostname as inferred from data source.",
        "examples": ["HYD-JOHDOE-MOB"],
        "group": "entity_specific",
        "type": "string",
        "source": ["MS Intune", "MS Active Directory", "Windows Security Logs"],
        "derived_field": false,
        "candidate_key": true,
        "data_structure": "list"
}
```

sds_ei_ui_config.json

```
{
  "string": {"width": 40},
  "integer": {"width": 10, "step_interval": 1},
  "timestamp": {"width": 15},
  "double": {"width": 10, "step_interval": 0.1}
}
```

  Sample Data Dictionary Field After:

```
"host_name": {
         "caption": "Hostname",
         "description": "Hostname as inferred from data source.",
         "examples": ["HYD-JOHDOE-MOB"],
         "group": "entity_specific",
         "type": "string",
         "source": ["MS Intune", "MS Active Directory", "Windows Security Logs"],
         "derived_field": false,
         "candidate_key": true,
         "data_structure": "list",
         "width": 40, //added from ui config file
         "solution": ["CAM", "VRA", "EI", "Host"] //EI and entity name is added as
                                                  default
```

```
}
```

Fields can be excluded from specific solutions by adding an “is_active” attribute. For example, setting "is_active": false for a particular field in the merged data dictionary ensures it won't be displayed in the dashboard for that solution.

Example:

- Description field should be removed from VRA dashboard, so vra data dcitionary will have "is_active" set to false for description

```
"description": {
       "is_active": false
}
```

After the script is executed, description in final data dictionary will look like

```
"description": {
         "caption": "Description",
         "description": "A description of the entity.",
         "group": "common",
         "examples": "",
         "ui_visibility": true,
         "enable_hiding": true,
         "type": "string",
         "width": 40,
         "solution": ["CAM", "EI", "Host"] // VRA is removed
}
```

6. Upgrade from beta 1.2 to 2.0

The upgrade from Beta 1.2 to 2.0 enhances Puppy Graph support, enables loading of non-SRDM sources into the Knowledge Graph, and improves enrichments of entity and relationships with multi-level hops.

6.1. Entity Extraction

Spark Job:

The entity extraction now offers two modes of operation: one that permanently keeps all entities once they've been loaded, and another that only retains entities found in the current day's data source. Additionally, it can now incorporate entities from non-SRDM sources into the knowledge graph.

Configuration:

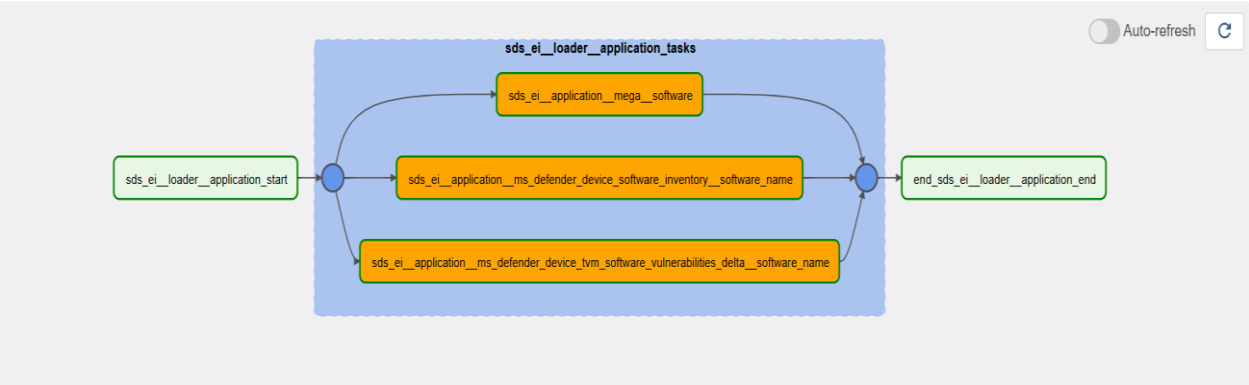
- The config structure has been updated with a new key 'operationalMode' to control whether entities are carried over or refreshed daily. By default, it is set to ‘carryForward’
- Introduced three new keys “dataIntervalTimestampKey”, “dataEventTimestampKey”, ”uniqueRecordIdentifierKey” under the datasource section to support non-SRDM source integration. By default, the values respectively for these keys are “parsed_interval_timestamp_ts”, “event_timestamp_ts” and “UUID”

```
{
  "primaryKey": "concat_WS(':',assessment_id,finding_primary_key)",
  "origin": "'Prevalent'",
  "outputTable" : "<%EM_SCHEMA_NAME%>.sds_em__finding",
  "operationalMode": "dailyReset",
  "dataSource":{
    "name":"Prevalent",
    "feedName":"Finding",
    "srdm":"<%EM_SCHEMA_NAME%>.sds_em__finding_evidence",
    "dataIntervalTimestampKey":"updated_at_ts",
    "dataEventTimestampKey": "updated_at_ts",
    "uniqueRecordIdentifierKey":"uuid"
  },
```

For more details, refer to

Orchestration:

- Each entity now has a separate DAG instead of a single DAG (sds_ei_loader_jobs_dag); for example, if the application entity has three inventory models, a DAG named sds_ei__loader__application is automatically created, containing three Spark job tasks one for each inventory model. For more details refer *EI Auto Orchestration Pipeline*.



- To support these changes, the folder structure inside the repository has been updated. New folders are introduced under sds_ei_configs, spark_job_configs, and inventory_models with the format sds_ei__loader__{entity_name}, where all inventory models corresponding to the entity are stored.

Directory Structure:

- The new version introduces a standardized directory structure requirement for client repositories. All entity-specific loader configurations must now be organized within dedicated directories named according to the pattern sds_ei__loader__{entity_name}. For example, entity configurations would be stored at paths like configs/spark_job_configs/source_models/inventory_models/sds_ei__loader__{entity_name}/ This structural organization enables the system to automatically generate DAGs based on the directory hierarchy, removing the need for manual DAG generation.

- Use the ---------- script to migrate the existing client repository folder structure to the new format.

```
configs
└── spark_job_configs
    └── source_models
        └── inventory_models
            └── sds_ei__loader__host
                ├── sds_ei__host__tenable_io_assets__id__job_config.json
                └── sds_ei__host__ms_intunes__device_id__job_config.json
```

6.2. Entity Resolution

Spark Job :

- In the updated version, fragments and resolvers are no longer combined in the publisher layer before indexing to the OLAP layer. Instead, to support Puppy Graph OLAP, each entity now has its own separate fragments and resolvers. The inter-disambiguation job now writes these entity-specific tables using a standardized naming convention: "sds_ei__fragment__{entity_name}" and "sds_ei__resolver__{entity_name}". Additionally, the fragment tables now include vertex name properties, while resolver tables include edge name properties to properly support vertex and edge definitions in Puppy Graph.
- A new feature has been added to the disambiguation job that allows for the union of fragments. This is particularly useful when there are no common candidate keys or when fragment union is specifically required during the disambiguation process.

Configuration:

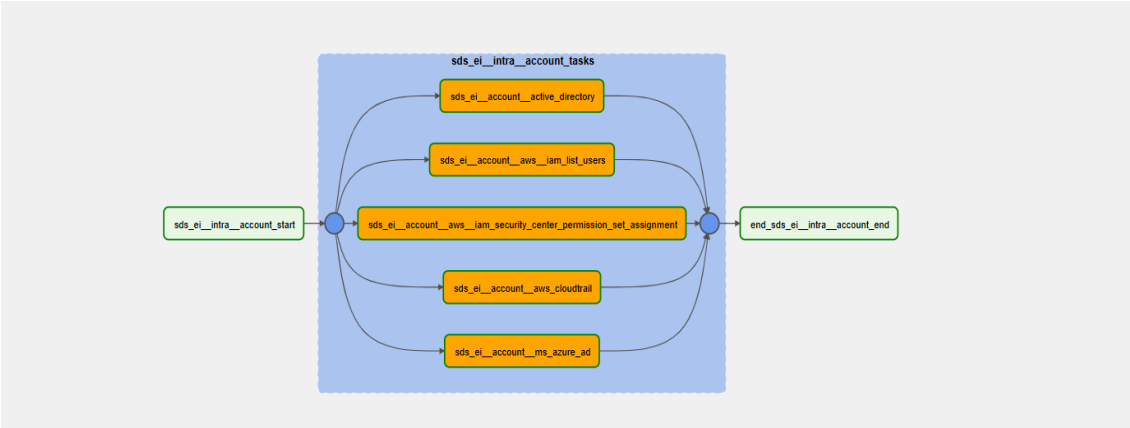
- The configuration now includes new keys to specify fragment and resolver locations, along with a new "isFragmentOLAPTable" key that determines whether fragments and resolvers should be indexed into the Puppy Graph. For intrasource disambiguation configs, "isFragmentOLAPTable" defaults to false. These additional keys are automatically handled during the config merge operation, so they're added to the necessary configurations without manual intervention.
- A new disambiguation approach has been introduced, controlled by the "disambiguationType" key. The default value is "candidateKey" which maintains the current behavior of disambiguation based on the values in the "candidateKeys" array. The alternative option supports union-type disambiguation when needed.

```
    "disambiguation": {
        "candidateKeys": [...],
        "confidenceMatrix": [...],
        "excludeValues": [...],
        "strategy": {...},
        "disambiguationType": "Union"
    },
    "derivedProperties": [...],
    "output": {
        "disambiguatedModelLocation": "<%EI_SCHEMA_NAME%>.sds_ei__account",
        "fragmentLocation": "<%KG_FRAGMENT_SCHEMA%>.sds_ei__fragment__account",
        "resolverLocation": "<%KG_FRAGMENT_SCHEMA%>.sds_ei__resolver__account",
        "isFragmentOLAPTable": true
    },
```

For more details refer *Entity Extraction Config*

Orchestration:

- Each entity now has its own dedicated disambiguation DAG, replacing the previous approach that used just two DAGs (sds_ei_intra_source_disambiguation_dag and sds_ei_inter_source_disambiguation_dag). The new DAGs follow a standardized naming convention, "sds_ei__intra__{entity_name}" for intra-source disambiguation and "sds_ei__inter__{entity_name}" for inter-source disambiguation. This means all intra-source disambiguation tasks for a specific entity are created under its dedicated intra DAG, while all inter-source disambiguation tasks for that entity are created under its corresponding inter DAG.
- For the DAG population, no manual intervention is required, as DAGs and spark job tasks are automatically created based on the directory structure. Therefore, client teams must follow the specified directory structure in their client repository.



Directory Structure

- The new version requires client repositories to follow a specific directory structure where all entity-specific inter-source disambiguation configs are stored under sds-ei-configs/spark_job_configs/source_models/intersource_disambiguated_models/sds_ei__inter__{entity_name}/, and similarly, all intra-source disambiguation configs for each entity are stored under sds-ei-configs/spark_job_configs/source_models/intrasource_disambiguated_models/sds_ei__intra__{entity_name}/. This structured approach enables automatic DAG generation based on the directory organization, eliminating the need for manual DAG configuration. For more details refer *EI Auto Orchestration Pipeline*.

```
sds-ei-configs
└── spark_job_configs
    └── source_models
        ├── intrasource_disambiguated_models
        │   └── sds_ei__intra__account
        │       ├── sds_ei__account__active_directory__job_config.json
        │       ├── sds_ei__account__aws__iam_list_users__job_config.json
        │       ├── sds_ei__account__aws__iam_security_center_permission_set_assignment__job_config.json
        │
        └── intersource_disambiguated_models
            └── sds_ei__inter__account
                └── sds_ei__account__job_config.json
```

6.3. Relationship Extraction

Spark Job:

- Users can now use temporary properties in the relationship extractor to create custom columns, which can then be used to define block variables. If you want to derive `source_p_id` and `target_p_id` within temporary properties, note that the calculated `p_id` will correspond to the loader level, not the resolved `p_id`.
- In previous versions, relationship creation required resolver dependency to generate connections between source and target entities. This made creating resolver tables unnecessary when disambiguation wasn't needed. The updated version provides two new approaches when disambiguation isn't required:
    - Building relationships directly using table join conditions - In this scenario, both the base and joining tables already contain the necessary p_id and class information, eliminating the resolver table requirement.
    - Creating relationships using loader configurations alone - These configurations contain all essential details including mandatory attributes, p_id, and class identifiers, which are then built across the base table.
- In the new version, the system has been enhanced to support creating relationships from non-SRDM sources.

Configuration:

- To support these changes new configuration options have been introduced under sourceMappingInfo and targetMappingInfo sections. "configPath": Specifies the path to loader configuration files, "tableName": Identifies the table containing the p_id column that will be used in joins, "joinCondition": Defines the specific condition for table joining.
- For supporting non srdm base table support “dataIntervalTimestampKey", "dataEventTimestampKey", "uniqueRecordIdentifierKey" are introduced under "inputSourceInfo". By default, the values respectively for these keys are “parsed_interval_timestamp_ts”, “event_timestamp_ts” and “UUID”

```
"temporaryProperties": [
        {
            "colName": "last_reopened_temp",
            "colExpr": "greatest(UNIX_MILLIS(TIMESTAMP(to_timestamp(first_reopened_datetime))))"
        }
    ],
"inputSourceInfo": [
    {
        "sdmPath": "<%EM_SCHEMA_NAME%>.sds_em__finding_evidence",
        "origin": "'Assessment Evidence'",
        "dataIntervalTimestampKey":"updated_at_ts",
        "dataEventTimestampKey": "updated_at_ts",
        "uniqueRecordIdentifierKey":"uuid",
        "sourceMappingInfo": {
            "tableName": "<%EM_SCHEMA_NAME%>.sds_em__assessment",
            "joinCondition": "s.assessment_id=e.assessment_id"
        },
        "targetMappingInfo": {
            "configPath": ["<%CONFIG_ARTIFACTORY_URI%><%EI_LOADER_CONFIG_BASE_PATH%>sds_em__finding"]
        }
    }
],
```

For more details refer the *relationship extraction config*

Orchestration

To support auto DAG generation, the relationship extractor DAG name is updated as "sds_ei__relationship_models", with all relationship extractor tasks grouped under this single DAG.

Directory Structure

The folder structure has been updated to include a new subfolder named "sds_ei__relationship_models" under the path sds-ei-configs/spark_job_configs/source_models/relationship_models/

```
sds-ei-configs
└── spark_job_configs
    └── source_models
        └── relationship_models
            └── sds_ei__relationship_models
                ├── sds_ei__rel__active_directory__host_has_identity__job_config.json
                └── sds_ei__rel__active_directory__person_has_identity__job_config.json
```

6.4. Relationship Resolution

Configuration

The entity/relationship fragment unioning job from the publisher has been integrated into the relationship disambiguation job. Unlike in beta 1.2 which created a single unioned table, the system now generates individual resolver and fragment tables for each task. The fragmentLocation and resolverLocation are automatically generated by the config merger script, though custom names can be specified if needed.

Orchestration

To support auto DAG generation, the relationship disambiguation DAG name is updated as " sds_ei__relationship_disambiguation", with all relationship disambiguation tasks grouped under this single DAG.

Directory Structure

The folder structure has been updated to include a new subfolder named "sds_ei__relationship_disambiguation" under the path sds-ei-configs/spark_job_configs/source_models/ relationship_disambiguation/

```
sds-ei-configs
└── spark_job_configs
    └── source_models
        └── relationship_disambiguation
            └── sds_ei__relationship_disambiguation
                ├── sds_ei__rel__host_has_identity__job_config.json
                └── sds_ei__rel__identity_has_account__job_config.json
```

6.5. Enrichments

Spark Job:

- The count enrichment functionality, which was previously part of the publisher layer, has now been moved to a dedicated Enrichments module within the knowledge graph. The latest version introduces Multi Hop Relationship Enrichment alongside relationship count enrichment in the knowledge graph. This powerful new feature allows for the creation, enhancement, or updating of entity attributes by traversing through multiple connected relationships between entities, supporting traversal across any number of relationship levels.
- For example, imagine a scenario where a Host is owned by a Person who has an Identity with administrative privileges. This critical admin privilege information isn't directly stored in the Host entity. Traditionally, to discover this, users would need to manually trace connections from Host → Person → Identity → Account to determine if any Identity associated with the Person has admin privileges. Similarly, identifying if a Host has an Active Critical Vulnerability that cannot be patched would require additional relationship traversal.
- With Multi Hop Relationship Enrichment, these complex multi-step relationship traversals can be automated, allowing the spark job to directly enrich the Host entity with this valuable information from distant, connected entities by writing the config.

Configuration:

- The entity enrichment configuration structure has two key sections, multihop relationship enrichments are defined in the “inheritedPropertyEnrichment” section, while relationship count enrichments are specified in the “countEnriches” section.
- The new version includes a configuration key called “writeOnlyEnrichedFields” that controls output behavior. When set to true, only the enriched fields are written to the output. When false (the default setting), all base table columns are written alongside the enriched columns.


```
{
  "entityTableName": "<%EI_SCHEMA_NAME%>.sds_ei__host",
  "output": { ...
  },
  "countEnriches": [ ...
  ],
  "inheritedPropertyEnrichment": [
    {
      "tableName": "ei_edm_sit_v5.sds_ei__rel__person_owns_host",
      "alias": "person_owns_host",
      "inheritedPropertyEnrichment": [
        {
          "tableName": "ei_edm_sit_v5.sds_ei__rel__person_has_identity",
          "alias": "person_has_identity_1",
          "colEnrichments": [
            {
              "colName": "owned_by",
              "aggExpr": "person_has_identity_1.source_display_label",
              "aggFunction": "collect_set"
            }
          ],
          "inheritedPropertyEnrichment": [
            {
              "tableName": "ei_edm_sit_v5.sds_ei__rel__identity_has_account",
              "alias": "id_hs_accnt",
              "inheritedPropertyEnrichment": [
                {
                  "tableName": "ei_edm_sit_v5.sds_ei__account",
                  "alias": "account",
                  "colEnrichments": [
                    {
                      "colName": "is_global_admin",
                      "aggExpr": "cast(account.privilege_roles as string) like '%Global Administrator%'",
                      "aggFunction": "min"
                    }
                  ]
                }
              ]
            }
          ]
        }
      ]
    },
  ]
}
```


Orchestration:

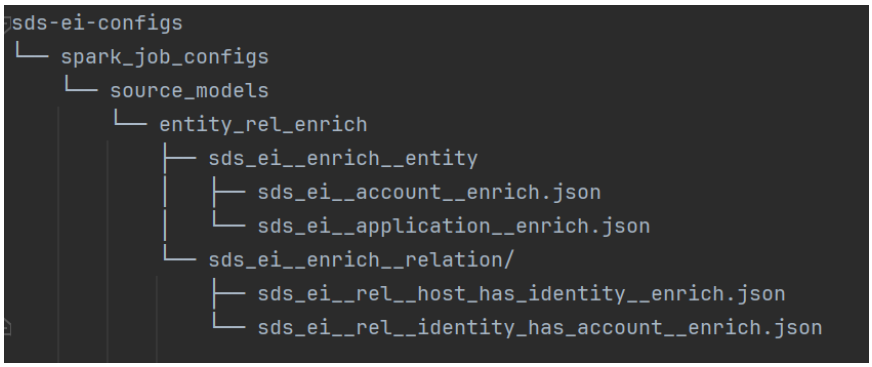
- In the orchestration layer, entity and relationship enrichment processes are handled by separate, auto-generated DAGs named "sds_ei__enrich__entity" and "sds_ei__enrich__relationship". All corresponding enrichment tasks are created and organized under these specific DAGs.

Orchestration Variables:

- Spark jobs now run automatically with default orchestration variable template for the enriched job also, and users only need to make manual overrides if they want to add job specific runtime configurations. In short, for every new client only configs, there is no need to add orchestration variables unless user requires any custom configurations

Directory Structure:

- To create or override enrichments, the client repository must follow a specific directory structure with two new subfolders introduced under the path sds-ei-configs/spark_job_configs/source_models/entity_rel_enrich/, sds_ei_enrich_entity for entity enrichment configurations and sds_ei_enrich_relationship for relationship enrichment configurations, each dedicated to organizing their respective enrichment configuration files.



6.6. Publisher

Spark Job :

- In the publisher layer, there's a new feature to determine whether a publisher table should be indexed to the OLAP layer (Puppy graph). This config-driven capability is useful when user don't want certain tables to appear in the puppy graph.
- This feature is particularly valuable for creating intermediate tables that contain additional field enrichments from other solution tables. These intermediate tables enable further analytics by combining data from multiple sources to generate new enrichments that can be written back to the final publish table.For writing back data to the final publish table from any intermediate, enrichment, or other solutions table, users have two options to specify which tables should be written back to the publisher:
    - Provide regex patterns to identify specific tables
    - Follow a standardized table structure provided by the Knowledge Graph team, which enables automatic write-back to the publisher layer
- The publisher layer now enhances support for the graph OLAP layer by including additional graph metadata properties as iceberg table properties. For entity publishers, the graph vertex name is added as the entity name. For relationship publishers, several edge properties are included, graph.edge.name as relationship name, graph.edge.source.name as relationship source class, and graph.edge.target.name as relationship target class.

Configuration :

- The new configuration key isOLAPTable that determines whether a table should be moved to the Knowledge Graph OLAP layer, defaulting to true.
- For enhancing publisher tables with additional fields, new configuration keys have been introduced, inputTableIdentifierRegex identifies the base table where fields will be added, while enrichTableIdentifierRegex identifies enrichment tables whose data should be written back to the base table. Default pattern configurations are provided for both entities and relationships, with entity patterns using inputTableIdentifierRegex: "^\w++[.]sds_ei__${name}enrich" and enrichTableIdentifierRegex: "^\w++.sds[a-z]+?entity${name}enrich", while relationship patterns use inputTableIdentifierRegex: "^\w++[.]sds_ei__rel${name}" and enrichTableIdentifierRegex: "^\w++[.]sds_[a-z_]+?rel${name}__enrich"

Orchestration:

- In the orchestration layer, entity and relationship publisher jobs are handled by separate, auto-generated DAGs named "sds_ei__publisher__entity" and "sds_ei__publisher__relation". All corresponding publisher spark job tasks are created and organized under these specific DAGs.

Directory Structure:

```
sds-ei-configs
└── spark_job_configs
    └── source_models
        └── publisher
            ├── sds_ei__publisher__relation
            │   ├── sds_ei__rel__finding_on_cloud_account__publish.json
            │   └── sds_ei__rel__host_has_identity__publish.json
            └── sds_ei__publisher__entity
                ├── sds_ei__host__publish.json
                └── sds_ei__finding__publish.json
```

6.7. Entity Explorer

Configuration

Dictionary changes involve the introduction of a graph data dictionary and enhancements to sorting orders, including entity-specific sorting (logical sort) and source-specific sorting (alphabetical sort), along with the addition of new attributes to support detailed view of relationships (Refer Entity Explorer). Some changes are given below:

Attribute	Dictionary Type
sorting_columns	Relationship Data Dictionary
is_enrichment	Relationship Data Dictionary
customFilter	Relationship Data Dictionary
relationshipCheck	Relationship Data Dictionary

Client Data dictionary Format

Expected Format:

The dictionaries will be split based on entities and relationships and kept as separate files.

host__data_dictionary.json

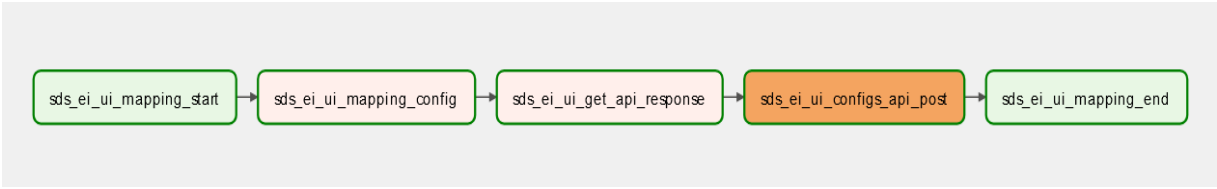
```
{
    "caption": "Host"
    "attributes": {
      "project": {
        "caption": "Project",
        "description": "Field enriched from solution",
        "examples": "test",
        "group": "enrichment",
        "ui_visibility": true,
        "dashboard_identifier":{
            "EI":{},
            "Host": {},
            "VRA": {
                "accessorKey": "source__project"
             },
            "IDRA": {
                "accessorKey": "subject__project"
             }
        }
      },
      "domain_score": {
        "caption": "Host Domain Risk Score",
        "description": "Description of the field",
        "group": "enrichment",
        "type": "double",
        "range_selection": true
      }
    }
  }
```

host_has_vulnerability_finding.json

```
{
    "relationship_attributes": {
        "source_display_label": {
            "caption": "Target Display Label",
            "description": "Name of target entity as obtained from source data.",
            "type": "string",
            "detailed_view_hide": true,
            "hideFeaturesDropdown": true,
            "dashboard_identifier": {
                "EI": {},
                "Host": {},
                "VRA": {
                        "is_active": false
                 }
            }
        }
    }
}
```

Orchestration

A new task “sds_ei_ui_get_api_response” has been introduced to handle patching failures of newly added endpoints. Insight api querying is introduced Refer section *3.7.3*



6.8. Orchestration Variables

Configuration

In beta 1.1, EI tasks need manual orchestration configurations written by DEs or DAs for each Spark job task. This process often causes naming inconsistencies that can prevent tasks from running. In beta 1.2, the solution uses a base template stored in Airflow variables for each job type, which reduces the need for manual setup. This template holds common attributes across tasks, and task-specific details are generated programmatically during task creation or runtime. This approach ensures consistent naming, reduces errors, and improves efficiency by automatically creating task configurations. As a result, it makes deployment simpler and more reliable across the pipeline. (Refer *EI Auto Orchestration Pipeline*). One thing to note is that there should not be any orchestration variables of previous versions (< 1.1). Therefore, it should be ensured that all the entity-based orchestration variables are removed.

6.9. Context Variable

The mandatory context variables for running EI include database credentials, storage URIs, configuration paths, schema names, AWS settings, and infrastructure parameters that are essential for connecting to data stores, managing configurations, and controlling the EI pipeline's execution environment.

Context Variable

```
  {
    "DATALAKE_URI": "<datalake_uri>",
    "ARTIFACTORY_URI": "<artifactory_uri>",
    "CONFIG_ARTIFACTORY_URI": "<config_artifactory_uri>",
    "EI_LOADER_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-configs/",
    "EI_INTRASOURCE_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
    "EI_INTERSOURCE_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
    "EI_RELATION_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
    "EI_RELATION_DIS_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
    "EI_PUBLISHER_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
    "EI_RELATIONSHIP_WRITE_BACK_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-
item/spark-job-configs/",
    "EI_SPARK_CONFIGS_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-configs/",
    "EI_PUBLISHER_LIST_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/list-configs-
meta/?config_item_type=publisher&deployed=true",
    "EI_INTER_LIST_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/list-configs-
meta/?config_item_type=intersource_disambiguated_models&deployed=true",
    "EI_ENTITY_REL_ENRICH_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
    "EI_OLAP_LIST_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/list-configs-
meta/?config_item_type=olap_configs&deployed=true",
    "EI_SCHEMA_NAME": "<ei_schema_name>",
    "EI_PUBLISH_SCHEMA_NAME": "<ei_publish_schema_name>",
    "EI_POST_SCHEMA_NAMES": "<ei_post_schema_names>",
    "KG_FRAGMENT_SCHEMA": "<kg_fragment_schema>",
    "EI_FRAGMENT_PUBLISH_SCHEMA_NAME": "<ei_fragment_publish_schema_name>",
```

```
    "EI_FRAGMENT_RELATIONSHIP_SCHEMA_NAME": "<ei_fragment_relationship_schema_name>",
    "LOOKUP_SCHEMA_NAME": "<lookup_schema_name>",
    "EI_LOOKUP_SCHEMA_NAME": "<ei_lookup_schema_name>",
    "KG_OLAP_SCHEMA": "<kg_olap_schema_name>",
    "KG_OLAP_FRAGMENT_SCHEMA": "<kg_fragment_olap_schema_name>",
    "LOADER_DRIVER_CORES": "1",
    "LOADER_DRIVER_MEMORY": "5g",
    "LOADER_EXECUTOR_CORES": "2",
    "LOADER_EXECUTOR_INSTANCES": "3",
    "LOADER_EXECUTOR_MEMORY": "4g",
    "LOADER_SHUFFLE_PARTITIONS": "200",
    "INTRASOURCE_DRIVER_CORES": "1",
    "INTRASOURCE_DRIVER_MEMORY": "3g",
    "INTRASOURCE_EXECUTOR_CORES": "2",
    "INTRASOURCE_EXECUTOR_MEMORY": "6g",
    "INTRASOURCE_EXECUTOR_INSTANCES": "3",
    "INTRASOURCE_SHUFFLE_PARTITIONS": "200",
    "INTERSOURCE_DRIVER_CORES": "1",
    "INTERSOURCE_DRIVER_MEMORY": "4g",
    "INTERSOURCE_EXECUTOR_CORES": "2",
    "INTERSOURCE_EXECUTOR_MEMORY": "6g",
    "INTERSOURCE_EXECUTOR_INSTANCES": "3",
    "INTERSOURCE_SHUFFLE_PARTITIONS": "200",
    "PUBLISHER_DRIVER_CORES": "1",
    "PUBLISHER_DRIVER_MEMORY": "3g",
    "PUBLISHER_EXECUTOR_CORES": "2",
    "PUBLISHER_EXECUTOR_MEMORY": "4g",
    "PUBLISHER_EXECUTOR_INSTANCES": "3",
    "PUBLISHER_SHUFFLE_PARTITIONS": "200",
    "RELATIONSHIP_DRIVER_CORES": "1",
    "RELATIONSHIP_DRIVER_MEMORY": "8g",
    "RELATIONSHIP_EXECUTOR_CORES": "2",
    "RELATIONSHIP_EXECUTOR_MEMORY": "12g",
    "RELATIONSHIP_EXECUTOR_INSTANCES": "5",
    "RELATIONSHIP_SHUFFLE_PARTITIONS": "200",
    "SDS_SOFTWARE_EXTRACTION_DRIVER_CORES": "1",
    "SDS_SOFTWARE_EXTRACTION_DRIVER_MEMORY": "1g",
    "SDS_SOFTWARE_EXTRACTION_EXECUTOR_CORES": "2",
    "SDS_SOFTWARE_EXTRACTION_EXECUTOR_INSTANCES": "6",
    "SDS_SOFTWARE_EXTRACTION_EXECUTOR_MEMORY": "7g",
    "ENRICHER_DRIVER_CORES": "2",
    "ENRICHER_DRIVER_MEMORY": "9g",
    "ENRICHER_EXECUTOR_CORES": "3",
    "ENRICHER_EXECUTOR_MEMORY": "16g",
    "ENRICHER_EXECUTOR_INSTANCES": "4",
    "ENRICHER_SHUFFLE_PARTITIONS": "50",
    "DATA_PARTITION_KEY": "updated_at_ts",
    "DATA_PARTITION_AMOUNT": "380",
    "PUPPY_GRAPH_STORAGE_TYPE": "<specify the storage type>",
    "LOG_URI": "<log_uri>",
    "AWS_ARN_ROLE": "<aws_arn_role>",
    "AWS_REGION_NAME": "<region_name>",
    "KUBE_NAMESPACE": "<namespace>",
    "SRDM_SCHEMA_NAME": "<srdm_name>",
    "SDM_SCHEMA_NAME": "<sdm_name>",
    "KEYCLOAK_HOSTNAME": "<keycloak_hostname>",
    "SDM_IMAGE_TAG": "<sdm_image_tag>",
    "SRDM_SCHEMA_NAME_INGESTION": "srdm_edm_sit_copy",
    "SDM_SCHEMA_NAME_INGESTION": "sdm",
    "SPARK_IMAGE_NAME": "<spark_image_name>",
    "EI_SPARK_IMAGE_NAME": "<ei_spark_image_name>",
    "HIVE_METASTORE_HOST": "<hive_metastore_host>",
    "SECRETS_MANAGER_NAME": "<secrets_manager_name>",
    "PYTHON_SPARK_3_IMAGE": "docker.io/prevalentai/spark:3.2.3-2.12-corretto8-bullseye11.6-sds-
iceberg-v1-0-pyspark-nodeps",
    "ADMIN_API_BASE_PATH": "<admin_api_base_path>",
    "KEYCLOAK_REALM": "<keycloak_realm_name>",
    "SERVICE_ACCOUNT": "spark",
    "ICEBERG_CATALOG_TYPE": "<iceberg_catalog_type>",
    "ICEBERG_CATALOG_URI": "<iceberg_catalog_uri>",
    "HADOOP_FS_IMPL": "<hadoop_fs_impl>",
    "HADOOP_FS_ENDPOINT": "<hadoop_fs_endpoint>",
    "SPARK_IMAGE_NAME_3_5": "<spark_image_name_3_5>"
}
```

6.10. Dockerfile

Dockerfile

```
ARG SDS_PLATFORM_APPS
ARG SDS_SOLUTION_EI
ARG SDS_PRODUCT_EM
ARG SDS_SOLUTION_STUDIO



ARG PE_CONFIG_BASE_IMAGE="prevalentai/sds-platform-apps-configs:"
ARG EI_CONFIG_BASE_IMAGE="prevalentai/sds-solution-ei-configs:"
ARG EM_CONFIG_BASE_IMAGE="prevalentai/sds-product-em-configs:"
ARG STUDIO_CONFIG_BASE_IMAGE="prevalentai/sds-solution-studio-configs:"


FROM ${STUDIO_CONFIG_BASE_IMAGE}${SDS_SOLUTION_STUDIO} AS studioconf

FROM ${EM_CONFIG_BASE_IMAGE}${SDS_PRODUCT_EM} AS emconf

FROM ${EI_CONFIG_BASE_IMAGE}${SDS_SOLUTION_EI} AS eiconf

FROM ${PE_CONFIG_BASE_IMAGE}${SDS_PLATFORM_APPS} AS peconf



COPY --from=eiconf /opt/airflow/ei_configs /opt/airflow/ei_configs
COPY --from=emconf /opt/airflow/em_configs/ /opt/airflow/em_configs/
COPY --from=studioconf /opt/airflow/studio_configs/ /opt/airflow/studio_configs/

COPY  ./sds_ei_configs /opt/airflow/sds_ei_configs
COPY  ./sds_ei_configs /opt/airflow/sds_ei_configs_temp
COPY  ./sds_em_configs /opt/airflow/sds_em_configs
COPY  ./sds_pe_configs /opt/airflow/sds_pe_configs
COPY  ./scripts /opt/airflow/scripts/
COPY  ./sds_data_dictionary /opt/airflow/sds_ei_configs_temp/sds_data_dictionary
COPY  ./sds_relationship_data_dictionary
/opt/airflow/sds_ei_configs_temp/sds_relationship_data_dictionary


USER root
RUN chown -R airflow:0 /opt/airflow/ \
&& ln -s /usr/local/bin/python /bin/python
USER airflow
RUN python -u /opt/airflow/sds_ei_configs/scripts/merge_orchestration_vars.py
/opt/airflow/sds_ei_configs/
RUN python -u /opt/airflow/sds_ei_configs/scripts/merge_orchestration_vars.py
/opt/airflow/sds_ei_configs_temp/
RUN python /opt/airflow/pe_configs/scripts/platform_config_merge.py
/opt/airflow/pe_configs \
/opt/airflow/sds_pe_configs /opt/airflow/pe_final_configs true
RUN python /opt/airflow/pe_configs/scripts/platform_config_merge.py
/opt/airflow/ei_configs \
/opt/airflow/sds_ei_configs /opt/airflow/ei_final_configs true
RUN python /opt/airflow/pe_configs/scripts/platform_config_merge.py
/opt/airflow/em_configs \
/opt/airflow/sds_em_configs /opt/airflow/em_final_configs true
RUN python /opt/airflow/pe_configs/scripts/platform_config_merge.py
/opt/airflow/pe_final_configs \
/opt/airflow/ei_final_configs /opt/airflow/pe_ei_final_configs true
```

```
RUN python /opt/airflow/pe_configs/scripts/platform_config_merge.py
/opt/airflow/pe_ei_final_configs \
/opt/airflow/em_final_configs /opt/airflow/pe_ei_em_final_configs true
RUN python /opt/airflow/pe_configs/scripts/platform_config_merge.py
/opt/airflow/pe_ei_em_final_configs \
/opt/airflow/studio_configs /opt/airflow/final_configs true



RUN python /opt/airflow/ei_configs/scripts/platform_merger_extended.py
/opt/airflow/ei_configs/orchestration_shared_fs/sds-ei-configs/ui-
configs/sds_data_dictionary \
    /opt/airflow/sds_data_dictionary /opt/airflow/ei_dict_configs
RUN python /opt/airflow/ei_configs/scripts/platform_merger_extended.py
/opt/airflow/ei_configs/orchestration_shared_fs/sds-ei-configs/ui-
configs/sds_relationship_data_dictionary \
    /opt/airflow/sds_relationship_data_dictionary /opt/airflow/ei_rel_configs
RUN python /opt/airflow/ei_configs/scripts/dictionary_combine.py --input_folder
/opt/airflow/ei_dict_configs    --output_filename
/opt/airflow/final_configs/orchestration_shared_fs/sds-ei-configs/ui-
configs/sds_ei_reference_dd.json
RUN python /opt/airflow/ei_configs/scripts/dictionary_combine.py --input_folder
/opt/airflow/ei_rel_configs    --output_filename
/opt/airflow/final_configs/orchestration_shared_fs/sds-ei-configs/ui-
configs/sds_ei_reference_relation_dd.json




COPY ./config_context /opt/airflow/final_configs/config_context
COPY ./config_context /opt/airflow/sds_ei_configs_temp/config_context

ENTRYPOINT ["/usr/bin/dumb-init", "--", "/entrypoint"]

CMD []
```

EUI Dockerfile

```
ARG SDS_PLATFORM_APPS
ARG SDS_SOLUTION_EI
ARG SDS_SOLUTION_CCM
ARG SDS_SOLUTION_VRA
ARG SDS_SOLUTION_CAM
ARG SDS_SOLUTION_CSPI
ARG SDS_SOLUTION_IDRA

ARG NGINX_BASE_IMAGE="prevalentai/nginx:unprivileged-debian-11.7"
ARG EUI_BASE_IMAGE="prevalentai/workbench:sds-enterprise-ui-1-2-1-7"
ARG PATH_TO_CODE='code/enterprise-ui'
ARG WEBAPP_ROOT_DIR="/usr/share/nginx/html/eui"

ARG CSPI_CONFIG_BASE_IMAGE="prevalentai/sds-solution-cspi-configs:"
ARG IDRA_CONFIG_BASE_IMAGE="prevalentai/sds-solution-idra-configs:"
ARG EI_CONFIG_BASE_IMAGE="prevalentai/sds-solution-ei-configs:"
ARG PE_CONFIG_BASE_IMAGE="prevalentai/sds-platform-apps-configs:"
ARG VM_CONFIG_BASE_IMAGE="prevalentai/sds-solution-vra-configs:"
ARG CCM_CONFIG_BASE_IMAGE="prevalentai/sds-solution-ccm-configs:"

FROM ${EUI_BASE_IMAGE} AS euibase
FROM ${CSPI_CONFIG_BASE_IMAGE}${SDS_SOLUTION_CSPI} AS cspiconf
FROM ${IDRA_CONFIG_BASE_IMAGE}${SDS_SOLUTION_IDRA} AS idraconf
```

```
FROM ${CCM_CONFIG_BASE_IMAGE}${SDS_SOLUTION_CCM} AS ccmconf
FROM ${VM_CONFIG_BASE_IMAGE}${SDS_SOLUTION_VRA} AS vmconf
FROM ${EI_CONFIG_BASE_IMAGE}${SDS_SOLUTION_EI} AS eiconf
FROM ${PE_CONFIG_BASE_IMAGE}${SDS_PLATFORM_APPS} AS peconf
FROM prevalentai/python:3.10.4-bullseye11.7 AS builder

COPY --from=eiconf /opt/airflow/ei configs /opt/airflow/ei configs
COPY --from=vmconf /opt/airflow/vm configs /opt/airflow/vm configs
COPY --from=cspiconf /opt/airflow/cspi_configs /opt/airflow/cspi_configs
COPY --from=idraconf /opt/airflow/idra_configs /opt/airflow/idra_configs
COPY --from=ccmconf /opt/airflow/ccm configs /opt/airflow/ccm configs
COPY --from=peconf /opt/airflow/pe configs /opt/airflow/pe configs
COPY  config-files /opt/airflow
RUN ls /opt/airflow/configs

COPY --from=euibase /tmp/enterprise-ui/ /tmp/enterprise-ui/
COPY custom-files/ /tmp/enterprise-ui/code/enterprise-ui/src/web/config/dashboard-solution-
schema/
COPY custom/config/ /tmp/enterprise-ui/code/enterprise-ui/customer-repo/
COPY custom/code/ /tmp/enterprise-ui/code/enterprise-ui/src/
COPY custom/images/ /tmp/enterprise-ui/code/enterprise-ui/public/
RUN mkdir -p /opt/airflow/final_configs_temp
COPY solution-config.json /tmp/enterprise-ui/code/enterprise-ui/src/

RUN python /opt/airflow/ei_configs/scripts/platform_merger_extended.py
/opt/airflow/ei_configs/orchestration_shared_fs/sds-ei-configs/ui-configs/sds_data_dictionary \
    /opt/airflow/sds_data_dictionary /opt/airflow/ei_dict_configs \
    && python opt/airflow/ei_configs/scripts/dictionary_combine.py --input_folder
/opt/airflow/ei_dict_configs  \
    --output filename /opt/airflow/final configs temp/sds ei data dictionary.json \
    && python /opt/airflow/pe configs/scripts/platform config merge.py
/opt/airflow/final_configs_temp \
    /opt/airflow/vm_configs/orchestration_shared_fs/sds-ei-configs/ui-configs/vra
/opt/airflow/final configs true \
    && python /opt/airflow/ei configs/scripts/eui configs update.py
/opt/airflow/final configs/sds ei data dictionary.json \
    /tmp/enterprise-ui/code/enterprise-ui/src/web/config/dashboard-solution-schema/entity-
inventory/config \
    && python /opt/airflow/ei_configs/scripts/eui_configs_update.py
/opt/airflow/final_configs/sds_ei_data_dictionary.json \
    /tmp/enterprise-ui/code/enterprise-ui/src/web/config/dashboard-solution-
schema/vulnerability-management/config



WORKDIR /tmp/enterprise-ui/code/enterprise-ui/
# RUN python3 ./eui_merge_config.py

FROM ${EUI_BASE_IMAGE} AS yarnbuilder

ARG PATH TO CODE
ARG build version=$build version
ARG WEBAPP ROOT DIR



COPY --from=builder /tmp/enterprise-ui/code /tmp/enterprise-ui/code
COPY --from=builder /tmp/enterprise-ui/default.conf /tmp/enterprise-ui/default.conf
COPY --from=builder /tmp/enterprise-ui/.npmrc /tmp/enterprise-ui/.npmrc

USER root

RUN npm install --ignore-scripts --global yarn \
    && cd /tmp/enterprise-ui/${PATH_TO_CODE}/ \
    && if [ $build_version != "null" ] ; then  yarn version --new-version ${build_version} --
no-git-tag-version ; fi\
    && yarn --ignore-scripts install \
    && yarn run build \
    && yarn pack \
    && build=`ls *.tgz` \
```

```
    && tar -xvf $build \
    && mv ./build/* ${WEBAPP_ROOT_DIR}
USER 101
```

7. Upgrade from beta 2.0 to 2.1

The upgrade from Beta 2.0 to 2.1 introduces view support in the entity extraction layer of the Knowledge Graph.

7.1. Entity Extraction

Spark Job:

By default, this version supports Iceberg view generation in the extraction table. To enable view support, Spark version 3.5.5 and Iceberg version 1.9 are required.

Configuration:

To override the default behavior of creating views and instead generate tables, update the configuration by adding a separate section for output table details. A new configuration key, outputTableInfo, has been introduced, which includes outputTableName and outputWrittenMode.

For more details regarding the view implementation refer the *Entity_Extraction_Using_View_Based_Strategy*

7.2. Entity Resolution

Spark Job:

Introduced a frequency-based field resolution strategy with fallback confidence, aimed at resolving conflicting attributes across multiple data fragments. This enables selecting the most representative value based on how frequently it appears across sources, ensuring the Knowledge Graph reflects the most consistent and meaningful information. The resolution process begins by counting the occurrence of each distinct value. The value with the highest frequency is selected. In cases where there is a tie, the system refers to a predefined fallbackValueConfidence list (e.g., ["Server", "Endpoint"]) to determine which value to prefer. If none of the tied values match the fallback list, one is randomly selected. This approach ensures a balance between data-driven decisions and domain-specific preferences. The configuration supporting this logic is defined under the frequencyConfidence strategy in the config file.

Configuration:

To support this feature, the configuration file should include a frequencyConfidence section inside the strategy block with a list. Each entry in this list specifies the field to be resolved as key “field” and an key “associated

fallbackValueConfidence" list that defines the preferred values.

7.3. Publisher

The Drop Column feature lets you exclude specific columns from the final published entity or relationship publisher schema. This helps improve schema efficiency, reduce storage usage, and enhance indexing performance by removing unnecessary fields that were only used for intermediate processing or analytical tasks. The feature drops the specified columns from the DataFrame before the data is written.

Configuration

To use this feature, add the "dropColumns" array under the "outputTableInfo" section in your publisher configuration:

7.4. Context Variable

The mandatory context variables for running EI include storage URIs, configuration paths, schema names, and infrastructure parameters that are essential for connecting to data stores, managing configurations, and controlling the EI pipeline's execution environment.

Context Variable

```
{
    "DATALAKE_URI": "<datalake_uri>",
    "ARTIFACTORY_URI": "<artifactory_uri>",
    "CONFIG_ARTIFACTORY_URI": "<config_artifactory_uri>",
```

```
    "DATAHUB_GMS_SERVER":"http://datahub-gms.datahub.svc.cluster.local:8080",
    "EI_LOADER_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-configs/",
    "EI_INTRASOURCE_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
    "EI_INTERSOURCE_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
    "EI_RELATION_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-configs/",
    "EI_RELATION_DIS_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
    "EI_PUBLISHER_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-configs/",
    "EI_RELATIONSHIP_WRITE_BACK_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-
job-configs/",
    "EI_SPARK_CONFIGS_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-configs/",
    "EI_PUBLISHER_LIST_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/list-configs-
meta/?config_item_type=publisher&deployed=true",
    "EI_INTER_LIST_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/list-configs-
meta/?config_item_type=intersource_disambiguated_models&deployed=true",
    "EI_ENTITY_REL_ENRICH_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
    "EI_SCHEMA_NAME": "<ei_schema_name>",
    "EI_PUBLISH_SCHEMA_NAME": "<ei_publish_schema_name>",
    "EI_POST_SCHEMA_NAMES": "<ei_post_schema_names>",
    "KG_FRAGMENT_SCHEMA": "<kg_fragment_schema>",
    "EI_FRAGMENT_PUBLISH_SCHEMA_NAME": "<ei_fragment_publish_schema_name>",
    "LOOKUP_SCHEMA_NAME": "<lookup_schema_name>",
    "EI_LOOKUP_SCHEMA_NAME": "<ei_lookup_schema_name>",
    "EI_OLAP_LIST_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/list-configs-
meta/?config_item_type=olap_configs&deployed=true",
    "KG_OLAP_SCHEMA": "<kg_olap_schema_name>",
    "KG_OLAP_FRAGMENT_SCHEMA": "<kg_fragment_olap_schema_name>",
    "LOADER_DRIVER_CORES": "1",
    "LOADER_DRIVER_MEMORY": "5g",
    "LOADER_EXECUTOR_CORES": "2",
    "LOADER_EXECUTOR_INSTANCES": "3",
    "LOADER_EXECUTOR_MEMORY": "4g",
    "LOADER_SHUFFLE_PARTITIONS": "200",
    "INTRASOURCE_DRIVER_CORES": "1",
    "INTRASOURCE_DRIVER_MEMORY": "3g",
    "INTRASOURCE_EXECUTOR_CORES": "2",
    "INTRASOURCE_EXECUTOR_MEMORY": "6g",
    "INTRASOURCE_EXECUTOR_INSTANCES": "3",
    "INTRASOURCE_SHUFFLE_PARTITIONS": "200",
    "INTERSOURCE_DRIVER_CORES": "1",
    "INTERSOURCE_DRIVER_MEMORY": "4g",
    "INTERSOURCE_EXECUTOR_CORES": "2",
    "INTERSOURCE_EXECUTOR_MEMORY": "6g",
    "INTERSOURCE_EXECUTOR_INSTANCES": "3",
    "INTERSOURCE_SHUFFLE_PARTITIONS": "200",
    "PUBLISHER_DRIVER_CORES": "1",
    "PUBLISHER_DRIVER_MEMORY": "3g",
    "PUBLISHER_EXECUTOR_CORES": "2",
    "PUBLISHER_EXECUTOR_MEMORY": "4g",
    "PUBLISHER_EXECUTOR_INSTANCES": "3",
    "PUBLISHER_SHUFFLE_PARTITIONS": "200",
    "RELATIONSHIP_DRIVER_CORES": "1",
    "RELATIONSHIP_DRIVER_MEMORY": "8g",
    "RELATIONSHIP_EXECUTOR_CORES": "2",
    "RELATIONSHIP_EXECUTOR_MEMORY": "12g",
    "RELATIONSHIP_EXECUTOR_INSTANCES": "5",
    "RELATIONSHIP_SHUFFLE_PARTITIONS": "200",
    "DATAHUB_GMS_SERVER":"http://datahub-gms.datahub.svc.cluster.local:8080",
    "EI_DQ_CONFIG_BASE_PATH":"/sds_mgmnt/config-manager/api/v1/config-item/",
    "DQ_SCHEMA_NAME":"kg_dq_4_1_2",
    "KG_SRDM_MINI_SCHEMA":"kg_srdm_mini_rel_4_1_2",
    "DATAHUB_ENV":"PROD",
    "DATAHUB_PLATFORM_INSTANCE":"prod_4_1_2",
    "DATA_QUALITY_DRIVER_CORES": "1",
    "DATA_QUALITY_DRIVER_MEMORY": "4g",
    "DATA_QUALITY_EXECUTOR_CORES": "2",
    "DATA_QUALITY_EXECUTOR_INSTANCES": "5",
    "DATA_QUALITY_EXECUTOR_MEMORY": "6g",
    "DQ_COMPACTION_EXECUTOR_INSTANCES": "6",
    "DQ_COMPACTION_SHUFFLE_PARTITIONS": "100",
    "DQ_COMPACTION_DRIVER_CORES": "1",
    "DQ_COMPACTION_DRIVER_MEMORY": "6g",
    "DQ_COMPACTION_EXECUTOR_CORES": "4",
    "DQ_COMPACTION_EXECUTOR_MEMORY": "12g",
```

```
    "DQ_COMPLETENESS_EXECUTOR_INSTANCES": "6",
    "DQ_COMPLETENESS_SHUFFLE_PARTITIONS": "100",
    "DQ_COMPLETENESS_DRIVER_CORES": "1",
    "DQ_COMPLETENESS_DRIVER_MEMORY": "6g",
    "DQ_COMPLETENESS_EXECUTOR_CORES": "4",
    "DQ_COMPLETENESS_EXECUTOR_MEMORY": "12g",
    "SDS_EM_ASSET_ENRICH_DRIVER_CORES": "1",
    "SDS_EM_ASSET_ENRICH_DRIVER_MEMORY": "3g",
    "SDS_EM_ASSET_ENRICH_EXECUTOR_CORES": "2",
    "SDS_EM_ASSET_ENRICH_EXECUTOR_MEMORY": "4g",
    "SDS_EM_ASSET_ENRICH_EXECUTOR_INSTANCES": "4",
    "SDS_EM_ASSET_ENRICH_SHUFFLE_PARTITIONS": "200",
    "SDS_EM_FINDING_ENRICH_SHUFFLE_PARTITIONS": "200",
    "SDS_SOFTWARE_EXTRACTION_DRIVER_CORES": "1",
    "SDS_SOFTWARE_EXTRACTION_DRIVER_MEMORY": "1g",
    "SDS_SOFTWARE_EXTRACTION_EXECUTOR_CORES": "2",
    "SDS_SOFTWARE_EXTRACTION_EXECUTOR_INSTANCES": "6",
    "SDS_SOFTWARE_EXTRACTION_EXECUTOR_MEMORY": "7g",
    "KUBE_NAMESPACE": "<namespace>",
    "SRDM_SCHEMA_NAME": "<srdm_schema_name>",
    "EI_SPARK_IMAGE_NAME": "docker.io/prevalentai/spark:4-1-0-3.5.5-2.13-iceberg-v1-9-bookworm-12.10-
20250428-slim",
    "EI_PYSPARK_IMAGE_NAME": "prevalentai/spark:4-1-0-3.5.5-2.12-python-v3-12-bookworm-12.10-20250428-
slim",
    "KEYCLOAK_REALM": "<keycloak_realm_name>",
    "SERVICE_ACCOUNT": "<spark_service_account>",
    "SOL_QA_AUTOMATION_SELENIUM_IMAGE": "prevalentai/qa-automation:selenium-standalone-chrome-4-15-0-1-
0-0-590",
    "SOL_QA_AUTOMATION_PYSPARK": "prevalentai/qa-automation:pyspark-1-0-0-590",
    "ICEBERG_CATALOG_TYPE": "org.apache.iceberg.spark.SparkCatalog",
    "ICEBERG_CATALOG_URI": "<svc_uri_of_hivemetastore>",
    "HADOOP_FS_IMPL": "org.apache.hadoop.fs.s3a.S3AFileSystem",
    "HADOOP_FS_ENDPOINT": "",
    "ENRICHER_DRIVER_CORES": "1",
    "ENRICHER_DRIVER_MEMORY": "3g",
    "ENRICHER_EXECUTOR_CORES": "2",
    "ENRICHER_EXECUTOR_MEMORY": "4g",
    "ENRICHER_EXECUTOR_INSTANCES": "4",
    "ENRICHER_SHUFFLE_PARTITIONS": "200",
    "SDS_OS_EXTRACTION_DRIVER_MEMORY": "1",
    "SDS_OS_EXTRACTION_DRIVER_CORES": "1g",
    "SDS_OS_EXTRACTION_EXECUTOR_CORES": "2",
    "SDS_OS_EXTRACTION_EXECUTOR_MEMORY": "7g",
    "SDS_OS_EXTRACTION_EXECUTOR_INSTANCES": "4",
    "ARCHIVAL_THRESHOLD_FACTOR": "1.5"
}
```


8. Upgrade from 2.1 to 4.2

The upgrade from 2.1 to 4.2 introduces several architectural and pipeline improvements to enhance scalability, performance, and deployment flexibility. These updates include view-based disambiguation outputs, separation of fragment processing into a dedicated pipeline, and the introduction of a separate pipeline for generating Mini SRDM models that capture the latest SRDM values. Additionally, the release refactors configuration deployment and orchestration, enabling independent deployment of KG product configurations and improving the scalability of entity resolution, resolver, and data quality processing workflows.

8.1. Entity Resolution

Spark Job:

- Previously, inter-source disambiguation materialized the final disambiguated output directly as physical tables.

- With the new update, disambiguation introduces a view-based output type. The pipeline now generates a base resolved table (with the suffix __resolved_inv) and exposes the disambiguated model as a view when outputWrittenMode is set to viewType.
- Union-type disambiguation continues to be written as tables.
- Reduced write time as recalculated fields, last-updated fields, enrichment fields, and derived fields are no longer written.
- Results in faster disambiguation job execution.

Configuration:

- The output.outputWrittenMode is introduced for disambiguation (default viewType, with tableType override supported for compatibility).

```
"output": {
    "disambiguatedModelLocation": "<%EI_SCHEMA_NAME%>.sds_ei__host",
    "outputWrittenMode": "tableType"
},
```

Validator Schema Cleanup:

- After upgrading to version 4.2, the disambiguation output in the validator schema is created as a view. However, the validator schema from the previous version may already contain a table with the same name for the disambiguation output.
- This naming conflict can cause an error during view creation. To prevent this issue, it is recommended to drop the validator schema with CASCADE before running the studio, ensuring that the new views can be created successfully by running from studio.

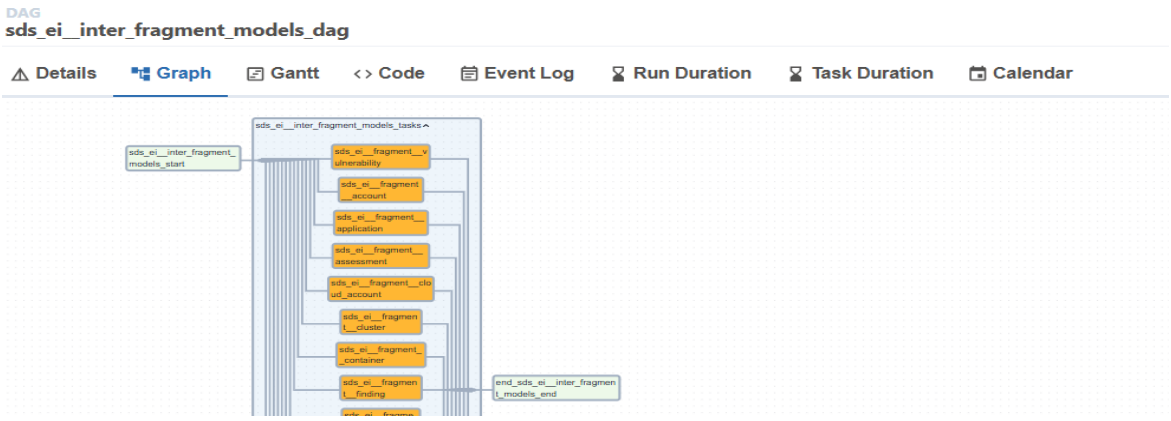
8.2. Fragment Separation

Spark Job:

- Fragment creation is now separated from the main disambiguation job. Fragment creation will run independently and in parallel after the inter-disambiguation job.
- Faster pipeline execution as the fragment writes job runs independently in parallel with relationship jobs.

Orchestration:

- Fragment write job orchestration is handled in a separate DAG called “sds_ei__inter_fragment_models_dag”. The task automatically skips execution if isFragmentOLAPTable in the disambiguation configuration is set to false.



For more details refer *EntityFragmentWrite*

8.3. Mini SRDM Models

Spark job:

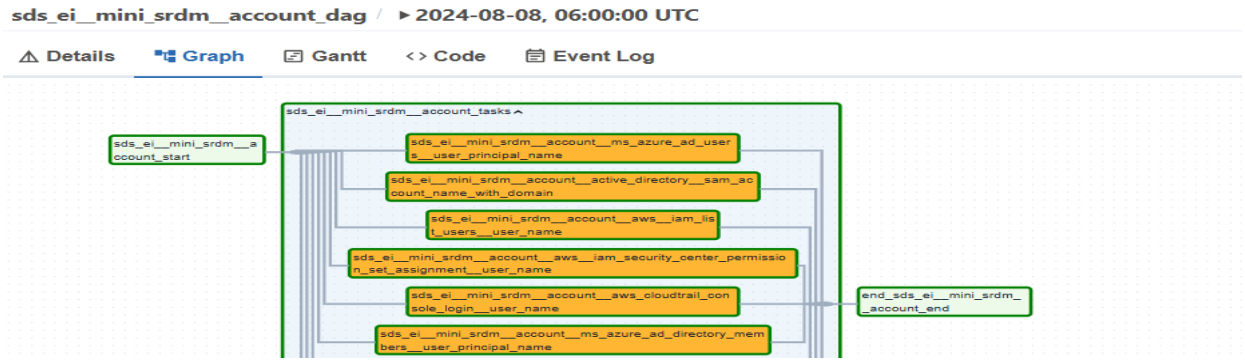
- In version 4.2, mini SRDM creation is a separate Spark job (MiniSRDMLoader) instead of being part of the main Loader. The Loader no longer creates mini SRDM schema.

Configuration:

- A new template, sds_ei_mini_srdm_models_orchestration_config_template, is used for mini SRDM jobs.
- Mini SRDM jobs use the same loader configurations as the entity loader.
- The spark.kg.mini_srdm_schema is still required and is defined in the mini SRDM orchestration template.

Orchestration:

- Mini SRDM models have their own DAGs. The DAGs follow the pattern sds_ei__mini_srdm__{entity_name}_dag, with one task for each inventory model.



For more details refer *minisrdm*

8.4. Independent Deployment of KG Product Configurations

- Previously, both product-level and client-level configurations were posted and deployed through the ei_config_client_deploy_job in the client repository. This approach created a tight coupling between product configuration dependencies and the client repository, requiring the client image to be rebuilt even when there were no changes to other client configurations or solutions.
- Starting with version 4.2, the KG product configuration load has been moved from the client repository to sds-solution-ei as the kg-config-load job. This change allows KG product configurations to be loaded and deployed independently, eliminating the need to rebuild the client repository when updating the solution-ei version.

8.5. Context

```
Context Variable
```

```
{
  "DATALAKE_URI": "<datalake_uri>",
  "ARTIFACTORY_URI": "<artifactory_uri>",
  "CONFIG_ARTIFACTORY_URI": "<config_artifactory_uri>",
  "EI_LOADER_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-configs/",
  "EI_INTRASOURCE_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
  "EI_INTERSOURCE_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
  "EI_RELATION_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-configs/",
  "EI_RELATION_DIS_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
  "EI_PUBLISHER_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-configs/",
  "EI_RELATIONSHIP_WRITE_BACK_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-
job-configs/",
  "EI_SPARK_CONFIGS_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-configs/",
  "EI_PUBLISHER_LIST_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/list-configs-
meta/?config_item_type=publisher&deployed=true",
  "EI_INTER_LIST_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/list-configs-
meta/?config_item_type=intersource_disambiguated_models&deployed=true",
  "EI_ENTITY_REL_ENRICH_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/spark-job-
configs/",
  "EI_OLAP_LIST_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/list-configs-
meta/?config_item_type=olap_configs&deployed=true",
  "PUPPY_GRAPH_STORAGE_TYPE": "<>",
  "PLATFORM_QA_IMAGE_VERSION": "4-1-0-20250725-0705754807-APPS",
  "KG_QA_IMAGE_VERSION": "4-2-0-20260203-1002701441-KG",
  "LOG_URI": "",
  "AWS_ARN_ROLE": "",
  "AWS_REGION_NAME": "ap-south-1",
  "KUBE_NAMESPACE": "<namespace>",
  "SRDM_SCHEMA_NAME": "<srdm_schema_name>",
  "SDM_SCHEMA_NAME": "sdm",
  "SRDM_SCHEMA_NAME_INGESTION": "srdm_edm_sit",
  "SDM_SCHEMA_NAME_INGESTION": "sdm",
  "KEYCLOAK_HOSTNAME": "<key_cloak_hostname>",
  "EI_SCHEMA_NAME": "<ei_schema_name>",
  "EI_UI_SCHEMA_NAME": "<ei_schema_name>",
  "EI_PUBLISH_SCHEMA_NAME": "<ei_publish_schema_name>",
  "EI_ENRICH_SCHEMA_NAME": "<ei_schema_name>",
  "EI_POST_SCHEMA_NAMES": "<ei_post_schema_names>",
  "KG_FRAGMENT_SCHEMA": "<kg_fragment_schema>",
  "KG_OLAP_SCHEMA": "<kg_olap_schema_name>",
  "KG_OLAP_FRAGMENT_SCHEMA": "<kg_fragment_olap_schema_name>",
  "EI_FRAGMENT_PUBLISH_SCHEMA_NAME": "<kg_fragment_schema>",
  "DATAHUB_GMS_SERVER": "<datahub_service_name>",
  "ENABLE_DATAHUB_INGESTION": "true",
  "EI_DQ_CONFIG_BASE_PATH": "/sds_mgmnt/config-manager/api/v1/config-item/",
  "QA_REPORT_URI": "azure://",
  "SPARK_IMAGE_NAME": "prevalentai/spark:4-2-0-R-3.5.5-2.13-py-iceberg-v1-9-bookworm-12.13-20260112-
slim",
  "EI_SPARK_IMAGE_NAME": "docker.io/prevalentai/spark:4-2-0-R-4.0.1-2.13-iceberg-v1-10-bookworm-
12.13-20260112-slim",
  "EM_SPARK_IMAGE_NAME": "docker.io/prevalentai/spark:4-2-0-R-4.0.1-2.13-iceberg-v1-10-bookworm-
12.13-20260112-slim",
  "HIVE_METASTORE_HOST": "<hive_metastore_host_name>",
  "LOADER_DRIVER_CORES": "1",
  "LOADER_DRIVER_MEMORY": "5g",
  "LOADER_EXECUTOR_CORES": "2",
  "LOADER_EXECUTOR_INSTANCES": "3",
  "LOADER_EXECUTOR_MEMORY": "10g",
  "LOADER_SHUFFLE_PARTITIONS": "200",
  "INTRASOURCE_DRIVER_CORES": "2",
  "INTRASOURCE_DRIVER_MEMORY": "8g",
  "INTRASOURCE_EXECUTOR_CORES": "2",
  "INTRASOURCE_EXECUTOR_MEMORY": "12g",
  "INTRASOURCE_EXECUTOR_INSTANCES": "3",
  "INTRASOURCE_SHUFFLE_PARTITIONS": "200",
  "INTERSOURCE_DRIVER_CORES": "1",
  "INTERSOURCE_DRIVER_MEMORY": "6g",
  "INTERSOURCE_EXECUTOR_CORES": "2",
  "INTERSOURCE_EXECUTOR_MEMORY": "12g",
  "INTERSOURCE_EXECUTOR_INSTANCES": "5",
  "INTERSOURCE_SHUFFLE_PARTITIONS": "200",
  "INTERFRAGMENT_DRIVER_CORES": "1",
  "INTERFRAGMENT_DRIVER_MEMORY": "3g",
  "INTERFRAGMENT_EXECUTOR_CORES": "2",
```

```
  "INTERFRAGMENT_EXECUTOR_INSTANCES": "5",
  "INTERFRAGMENT_EXECUTOR_MEMORY": "12g",
  "INTERFRAGMENT_SHUFFLE_PARTITIONS": "200",
  "PUBLISHER_DRIVER_CORES": "1",
  "PUBLISHER_DRIVER_MEMORY": "3g",
  "PUBLISHER_EXECUTOR_CORES": "2",
  "PUBLISHER_EXECUTOR_MEMORY": "12g",
  "PUBLISHER_EXECUTOR_INSTANCES": "5",
  "PUBLISHER_SHUFFLE_PARTITIONS": "200",
  "RELATIONSHIP_DRIVER_CORES": "2",
  "RELATIONSHIP_DRIVER_MEMORY": "4g",
  "RELATIONSHIP_EXECUTOR_CORES": "2",
  "RELATIONSHIP_EXECUTOR_MEMORY": "12g",
  "RELATIONSHIP_EXECUTOR_INSTANCES": "7",
  "RELATIONSHIP_SHUFFLE_PARTITIONS": "200",
  "SECRETS_MANAGER_NAME": "",
  "PYTHON_SPARK_3_IMAGE": "docker.io/prevalentai/spark:4-2-0-R-3.5.5-2.13-py-iceberg-v1-9-bookworm-
12.13-20260112-slim",
  "EI_PYSPARK_IMAGE_NAME": "prevalentai/spark:4-2-0-R-3.5.5-2.12-py-iceberg-v1-9-bookworm-12.13-
20260112-slim",
  "ADMIN_API_BASE_PATH": "",
  "KEYCLOAK_REALM": "<keycloak_realm_name>",
  "SDS_SOFTWARE_EXTRACTION_DRIVER_CORES": "1",
  "SDS_SOFTWARE_EXTRACTION_DRIVER_MEMORY": "1g",
  "SDS_SOFTWARE_EXTRACTION_EXECUTOR_CORES": "2",
  "SDS_SOFTWARE_EXTRACTION_EXECUTOR_INSTANCES": "4",
  "SDS_SOFTWARE_EXTRACTION_EXECUTOR_MEMORY": "7g",
  "LOOKUP_SCHEMA_NAME": "<lookup_schema_name>",
  "EI_LOOKUP_SCHEMA_NAME": "<ei_lookup_schema_name>",
  "ENRICHER_DRIVER_CORES": "1",
  "ENRICHER_DRIVER_MEMORY": "3g",
  "ENRICHER_EXECUTOR_CORES": "2",
  "ENRICHER_EXECUTOR_MEMORY": "12g",
  "ENRICHER_EXECUTOR_INSTANCES": "4",
  "SPARK_IMAGE_NAME_3_5": "docker.io/prevalentai/spark:4-1-4-R-3.5.1-2.12-iceberg-v1-6-bookworm-
12.10-20250428-slim",
  "ENRICHER_SHUFFLE_PARTITIONS": "200",
  "DATA_QUALITY_DRIVER_CORES": "1",
  "DATA_QUALITY_DRIVER_MEMORY": "4g",
  "DATA_QUALITY_EXECUTOR_CORES": "2",
  "DATA_QUALITY_EXECUTOR_INSTANCES": "5",
  "DATA_QUALITY_EXECUTOR_MEMORY": "10g",
  "GRAPH_SCHEMA": "<kg_olap_schema_name>,<kg_fragment_olap_schema_name>",
  "DATA_PARTITION_KEY": "updated_at_ts",
  "DATA_PARTITION_AMOUNT": "380",
  "DQ_COMPACTION_EXECUTOR_INSTANCES": "6",
  "DQ_COMPACTION_SHUFFLE_PARTITIONS": "200",
  "DQ_COMPACTION_DRIVER_CORES": "1",
  "DQ_COMPACTION_DRIVER_MEMORY": "6g",
  "DQ_COMPACTION_EXECUTOR_CORES": "4",
  "DQ_COMPACTION_EXECUTOR_MEMORY": "12g",
  "DQ_COMPLETENESS_EXECUTOR_INSTANCES": "6",
  "DQ_COMPLETENESS_SHUFFLE_PARTITIONS": "200",
  "DQ_COMPLETENESS_DRIVER_CORES": "1",
  "DQ_COMPLETENESS_DRIVER_MEMORY": "6g",
  "DQ_COMPLETENESS_EXECUTOR_CORES": "4",
  "DQ_COMPLETENESS_EXECUTOR_MEMORY": "12g",
  "SDS_OS_EXTRACTION_DRIVER_MEMORY": "12g",
  "SDS_OS_EXTRACTION_DRIVER_CORES": "2",
  "SDS_OS_EXTRACTION_EXECUTOR_CORES": "2",
  "SDS_OS_EXTRACTION_EXECUTOR_MEMORY": "16g",
  "SDS_OS_EXTRACTION_EXECUTOR_INSTANCES": "4",
  "DATAHUB_ENV": "PROD",
  "DATAHUB_PLATFORM_INSTANCE": "<datahub_platform_instance>",
  "DQ_SCHEMA_NAME": "<dq_schema_name>",
  "KG_SRDM_MINI_SCHEMA": "<mini_srdm_schema_name>"
}
```

9. Troubleshooting

9.1. General Spark Job Failures

9.1.1. Config path does not exist.

- This occurs mainly because the config is not accessible by the job. This can be due to the following
- Wrong s3 bucket or path given as argument
- CI/CD syncing issue. Sometimes the sync might not work or maybe It is still in progress. Please make sure the job is only triggered after the sync process is complete.

9.2. General Orchestration Failures

Apart from the mentioned causes of common orchestration errors, CI/CD pipeline issue and wrong build number are also the most common cause for these errors. So please check those before proceeding to further investigation.

9.2.1. DAG import error in Airflow UI

DAG import errors occurs when airflow parses all the dag files under /dags folder and finds python files which are not of the instance of DAG object. This error can also occur if the same version of dag file is not in sync with all the scheduler, executer and webserver.

9.2.2. Variable missing when pipeline starts running.

Sometimes when the loader or any of the dags of ei are triggered, the task fails because of missing variable when variable call happens. This can happen because of the following.

1. Key name of the variable is not same as the spark job config file name. The names (without extension) must be the same.
2. CI/CD deployment issue.

9.2.3. Task(s) not getting generated in airflow UI after deployment.

After deployment, the task(s) does not get generated. This can be due to the ***following.***

Missing ei job model file in sds-ei-configs.

The name of the config model is not added into the deployment_config file.

9.2.4. "None" value rendered for parsed-interval-start in orchestration variables.

Cases might occur where the rendered values of params like parsed-interval-start is rendered "None". This occurs due to wrong task_id given for the xcom_pull. In case of relationship tasks, make sure "include_prior_dates" param is set to true for xcom_pull.

9.2.5. Mediator dag: dag id not found.

This occurs due to incorrect dag name given in the sds_orchestrator_config variable. Check the dag names listed in the config along with the dependencies.

9.2.6. Mediator dag: Multiple days run triggered simultaneously.

Check if "sequential_pipeline_run" param is set to true. By default, this param is false.

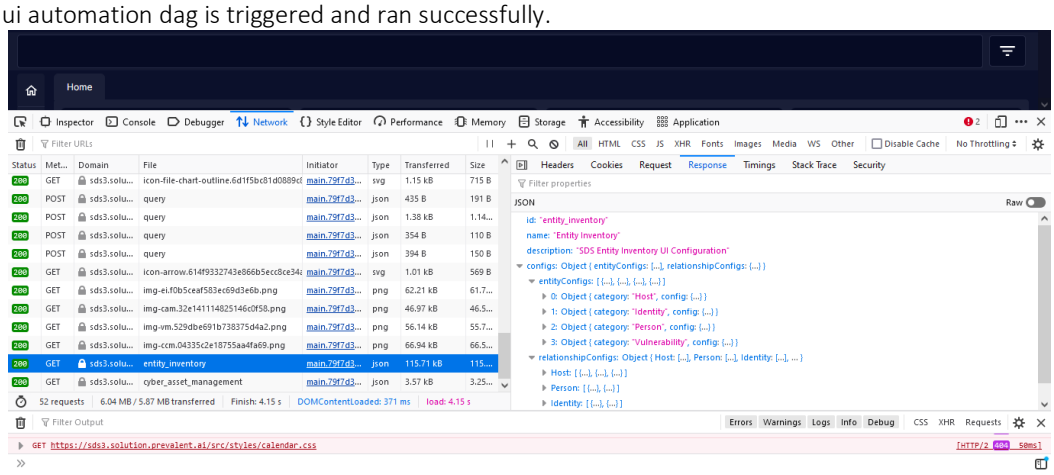
9.3. General UI Automation failures

If UI automation encounters issues, explore *Entity Explorer* for comprehensive insights into the formatting and functionality of UI automation with respect to data dictionary.

9.3.1. UI Rendering Issues

Troubleshooting Data Dictionary Updates and UI Reflection:

If you make changes to the data dictionary, but those changes are not reflected in the UI, the first thing to do is open the developer tools by pressing Ctrl+Shift+I, go to the network tab, click on "entity_inventory," and check the response. Look for the recent changes in the configuration. If they're not present, there might be an issue with the deployment. Verify the correctness of build number and data dictionary commits were merged. Also check whether ui automation dag is triggered and ran successfully.



1. Ensuring Correct Attributes on UI:

   If the attributes for a field don't look right on the UI, first, review the data dictionary to ensure the expected values for that field are correctly defined.

Resolving Entities or Fields Missing on the UI:

If you notice entities or fields missing on the UI, start by ensuring that the entity names and field names in the data dictionary match what is published in Druid. Check if the "ui_visibility" is set to false, as this could be a reason for items not showing up. If the issue persists, delve into the Druid database. Examine the "sds_ei_entity_inventory" and "sds_ei_non_disambiguated_entity_inventory" tables to confirm that the fields are present in both. For additional details such as entities, enrichment fields, and value ranges, refer to the "sds_ei_entity_inventory" table. Also, for relationships, check the "sds_ei_relationship" table. This approach helps identify and resolve any issues related to missing or incorrect information on the UI.

9.3.2. UI Automation Dag Failures

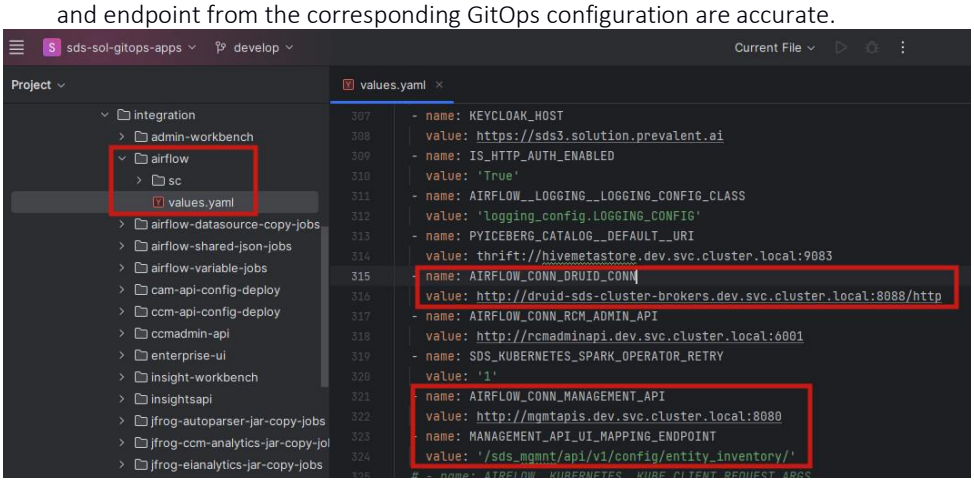
KeyError - Missing Mandatory Attributes

If there are failures in the DAG, it could be because of some essential attributes are missing from the data dictionary. Check Section *3.6.3* for mandatory attributes and ensure they are present in the data dictionary. If the DAG logs indicate a KeyError, inspect the logs. Look at the field mentioned in the last log entry. In the data dictionary, the field following the one mentioned in the log is likely missing some required attributes.

ConnectionError - Ensuring Connection Stability for UI Automation

Connection issues may arise in UI automation due to the reliance on data from Druid and posting UI configurations to an API endpoint. To troubleshoot:

i. Environment Variable Verification
- Check if environment variables are correctly set.
- Ensure the Druid connection is using the correct broker's URL and the provided management API and endpoint from the corresponding GitOps configuration are accurate.



ii. Connection Check
- Ensure that the corresponding environments for Druid are operational.
- Check if the connection is functioning by testing the broker's URL and the provided management API endpoint.